Decoders

Garrett Goon

Abstract Notes on various aspects of decoder models (and related topics). Conventions are in the appendix, Appendix A.

Contents

1. Architecture 5
1.1. Decoder-Only Fundamentals 5
1.1.1. Embedding Layer and Positional Encodings 6
1.1.2. Layer Norm 7
1.1.3. Causal Attention 7
1.1.4. MLP 10
1.1.5. Language Model Head 11
1.1.6. All Together 11
1.1.7. The Loss Function 12
1.2. Architecture and Algorithm Variants 13
1.2.1. RMS Norm 13
1.2.2. GLU Variants 13
1.2.3. Multi-Query Attention 13
1.2.4. Grouped Attention 13
1.2.5. Parallel `MLP` and `CausalAttention` Layers 14
1.2.6. RoPE Embeddings 14
1.2.6.1. RoPE Implementation Notes 15
1.2.6.2. YaRN 16
1.2.7. Flash Attention 17
1.2.7.1. The Details 19
1.2.8. Implementation Notes 21
2. Linear Attention 21
2.1. Intro 21
2.2. General Structure 22
2.2.1. Simple Linear Attention 22
2.2.2. $S_{sd'd}$ as a Look-up Table 22
2.2.3. Gating 23

2.2.4. Delta Rule 23
2.3. Efficient Implementations 23
2.3.1. Flash Linear Attention 24
2.3.2. Gated 25
2.3.3. Delta 26
2.4. Mamba2 28
2.4.1. Architecture Overview 28
2.4.2. Architecture Details 28
2.4.3. Mamba2 Duality with Attention 30
2.4.4. Details: Cumsums, Chunking, and the Mamba2 Scan 30
2.4.4.1. The $c = c'$ Cases 31
2.4.4.2. The $c > c'$ Cases 32
2.4.4.3. Chunked Solution 34
2.4.4.4. The Non-Chunked Solution 34
2.4.4.5. Notes On the `mamba-ssm` Implementation 34
2.4.4.6. Context Parallel Mamba2 Scan 36
2.4.5. Aren't These Just RNNs? 37
3. Pretraining 37
3.1. Memory 37
3.1.1. No Sharding 38
3.1.1.1. Parameters, Gradients, Optimizer States, and Mixed Precision 38
3.1.1.2. Gradients 39
3.1.1.3. Activations 40
3.1.1.3.1. Attention Activations 40
3.1.1.3.2. MLP Activations 40
3.1.1.3.3. LayerNorm, Residual Connections, and Other Contributions .40
3.1.1.3.4. Total Activation Memory 40
3.1.1.4. When does mixed-precision reduce memory? 41
3.2. Training FLOPs 42
3.2.1. No Recomputation 43
3.2.1.1. Multi-Head Attention: Forwards 43
3.2.1.2. Mamba2: Forwards 43
3.2.1.3. MLP: Forwards 43
3.2.1.4. Backwards Pass: Approximate 43
3.2.1.5. Backwards Pass: More Precise 44
3.2.1.6. Total Model FLOPs 44
3.2.2. Training Time 45
3.2.3. Scaling Laws 45
3.2.3.1. Original Scaling Laws 46
3.2.3.2. Chinchilla Scaling Laws 47
4. Fine Tuning 48

4.1. Instruction Fine Tuning 48
4.1.1. Direct Preference Optimization 48
4.1.2. KTO: Preference Finetuning without Pairs 50
5. RL 51
5.1. Background 51
5.1.1. Terminology and Fundamentals 51
5.1.2. Policy Gradients 52
5.1.3. Specialization to LLMs 53
5.2. PPO 53
5.3. DeepSeek R1 and GRPO 54
5.4. DAPO 55
5.5. Skywork 55
6. Parallelism 56
6.1. Tensor Parallelism 56
6.1.1. MLP 56
6.1.2. Attention 57
6.1.3. Embedding and LM Head 59
6.1.4. LayerNorm and Dropout 59
6.1.5. Effects on Memory 60
6.2. Sequence Parallelism 60
6.3. Ring Attention 62
6.3.0.1. The Causal Mask 63
6.4. Pipeline Parallelism 65
6.4.1. Naive Schedule: The Worst 65
6.4.2. All Forward All Backward 66
6.4.3. 1F1B 67
6.4.4. Interleaved Schedules 68
6.4.5. Depth-First vs. Breadth-First Scheduling and Data Parallelism 68
6.4.6. The Zero-Bubble Strategies 69
6.4.7. DualPipe 70
7. Vision 71
7.1. Vision Transformers 71
7.2. CLIP 71
8. Mixture of Experts 72
8.1. Basics 72
8.2. Routing 73
8.2.1. Token Choice vs Expert Choice 73
8.3. MegaBlocks 74
8.4. MoE Variants 74
8.4.1. Shared Experts 74

8.5. DeepSeekV3 74
8.5.1. Multi-Head Latent Attention 74
9. Inference 75
9.1. Basics and Problems 75
9.1.1. Generation Strategies 75
9.1.1.1. Greedy 75
9.1.1.2. Simple Sampling: Temperature, Top-k, and Top-p 75
9.1.1.3. Beam Search 76
9.1.1.4. Speculative Decoding 76
9.1.2. The Bare Minimum and the kv-Cache 77
9.1.3. Basic Memory, FLOPs, Communication, and Latency 78
9.1.3.1. Prefill Timing 78
9.1.3.2. Decode Timing 78
9.1.3.3. Tensor Parallelism 79
9.2. Optimizations 79
9.2.1. PagedAttention 79
A Conventions and Notation 79
B Collective Communications 81
C Device Meshes 82
C.1 Basic Example 83
C.2 Slicing 83
D Hardware 84
D.1 NVIDIA GPU Architecture 84
D.2 CUDA Programming Model 85
D.3 NVIDIA GPU Stats 85
D.4 Compute-bound vs Memory-bound 86
D.4.1 Matrix-Multiplications vs. Element-wise Operations 86
D.4.2 Training vs. Inference 86
D.5 Intra- and Inter-Node Communication 87
E Batch Size, Compute, and Training Time 87
F Initialization, Learning Rates, μ-Transfer etc 90
F.1 Wide Models are Nearly Gaussian 90
F.2 muTransfer and Similar Ideas 91
F.2.1 A Toy Limit: Deep Linear Networks and SGD 91
F.2.1.1 Adam 95
F.2.1.2 Activations 97
G Cheat Sheet 97
G.1 Training 97
H Ops 98

H.1 Associative Scans 98
H.2 Convolutions 99
H.2.1 Options 100
H.2.2 Output Shape 100
H.2.3 Backwards 100
I Evaluation 101
I.1 RULER: Long Context Evaluation 103
Bibliography 105

1. Architecture

1.1. Decoder-Only Fundamentals

The Transformers architecture [1], which dominates Natural Language Processing (NLP) as of July 2023, is a relatively simple architecture. There are various flavors and variants of Tranformers, but focus here on the decoder-only versions which underlie the GPT models [2], [3], [4].

The full decoder-only architecture can be seen in Figure 1. See Appendix A for more on conventions.

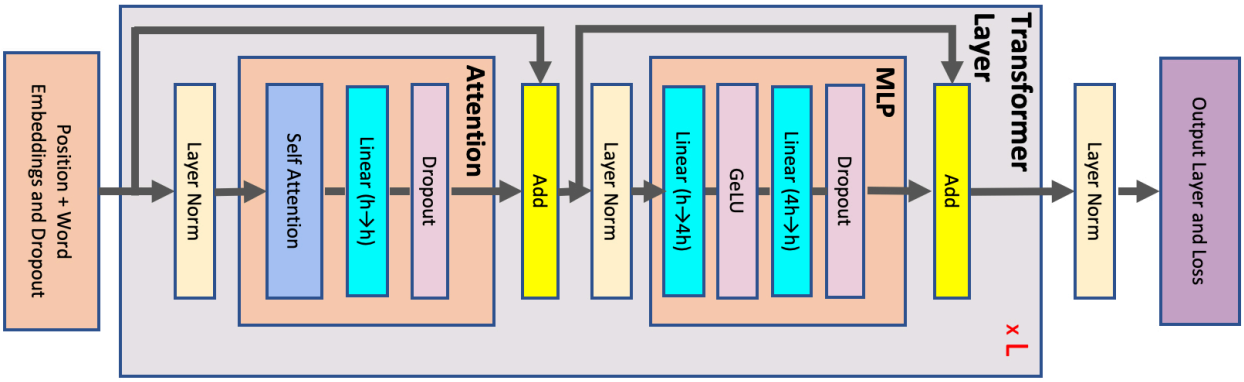


Figure 1: The full transformers architecture. Diagram taken from [5]

At a high level, decoder-only transformers take in an ordered series of word-like objects, called tokens, and are trained to predict the next token in the sequence. Given some initial text, transformers can be used to give a prediction for the likelihood of any possible continuation of that text. An outline of the mechanics[1]:

1. Raw text is **tokenized** and turned into a series of integers[2] whose values lie in `range(V)`, with V the vocabulary size.
2. The tokenized text is chunked and turned into `(B, S)`-shaped (batch size and sequence length, respectively) integer tensors, x_{bs}.

[1] This describes the vanilla architecture; almost every component is modified in the available variants.
[2] There are about 1.3 tokens per word, on average.

3. The **embedding layer** converts the integer tensors into continuous representations of shape `(B, S, D)`, z_{bsd}, with D the size of the hidden dimension. **Positional encodings** have also been added to the tensor at this stage to help the architecture understand the relative ordering of the text.

4. The z_{bsd} tensors pass through a series of transformer blocks, each of which has two primary components:

   a. In the **attention** sub-block, components of z_{bsd} at different positions (s-values) interact with each other, resulting in another `(B, S, D)`-shaped tensor, z'_{bsd}.

   b. In the **MLP** block, each position in z'_{bsd} is processed independently and in parallel by a two-layer feed-forward network, resulting once more in a -shaped tensor.

   Importantly, there are **residual connections** around each of these[3] (the arrows in Figure 1), meaning that the output of each block is added back to its original input.

5. Finally, we convert the `(B, S, D)`-shaped tensors to `(B, S, V)`-shaped ones, y_{bsv}. This is the role of the **language model head** (which is often just the embedding layer used in an inverse manner.)

6. The y_{bsv} predict what the next token will be, i.e. x_{bs+1}, having seen the **context** of the first s tokens in the sequence. Specifically, removing the batch index for simplicity, a `Softmax` of y_{sv} gives the conditional probability $p_{sv} = P(t_{s+1} \mid t_s ... t_0)$ for the indicated series of tokens. Because of the chain rule of probability, these individual probabilities can be combined to form the probability that any sequence of tokens follows a given initial seed[4].

Each batch (the b-index) is processed independently. We omitted `LayerNorm` and `Dropout` layers above, as well as the causal mask; these will be covered below as we step through the architecture in more detail.

1.1.1. Embedding Layer and Positional Encodings

The **embedding** layer is just a simple lookup table: each of the `range(V)` indices in the vocabulary is mapped to a D-dimensional vector via a large `(V, D)`-shaped table/matrix. This layer maps $x_{bs} \to z_{bsd}$. In , this is an `nn.Embedding(V, D)` instance.

To each item in a batch, we add identical **positional encodings** to the vectors above with the goal of adding fixed, position-dependent correlations in the sequence dimension which will hopefully make it easier for the architecture to pick up on the relative positions of the inputs[5] This layer maps $z_{bsd} \leftarrow z_{bsd} + p_{sd}$, with p_{sd} the positional encoding tensor.

---

[3] This gives rise to the concept of the **residual stream** which each transformer block reads from and writes back to repeatedly.

[4] In more detail, these probabilities are created by products: $P(t_{s+n} ... t_{s+1} \mid t_s ... t_0) = P(t_{s+n} \mid t_{s+n-1} ... t_s ... t_0) \times ... \times P(t_{s+1} \mid t_s ... t_0)$.

[5] Positional encodings and the causal mask are the only components in the vanilla transformers architecture which carry weights with a dimension of size S; i.e. they are the only parts that have explicit sequence-length dependence. A related though experiment: you can convince yourself that if the inputs z_{bsd} were just random

The above components require $(V+S)D \approx VD$ parameters per model.

1.1.2. Layer Norm

The original transformers paper [1] put `LayerNorm` instances after the **attention** and **MLP** blocks, but now it is common [6] to put them before these blocks[6].

The `LayerNorm` operations acts over the hidden dimension (since this is the dimension the subsequent `Linear` instances act on). Spelling it out, given the input tensor z_{bsd} whose mean and variance over the d-index are μ_{bs} and σ_{bs}, respectively, the `LayerNorm` output is

$$z_{bsd} \leftarrow \left(\frac{z_{bsd} - \mu_{bs}}{\sigma_{bs}}\right)\gamma_d + \beta_d \equiv \texttt{LayerNorm}_d\, z_{bsd} \tag{1}$$

where γ_d, β_d are the trainable scale and bias parameters. In `torch`, this is a `nn.LayerNorm(D)` instance. Since there are two `LayerNorm` instances in each transformer block, these components require $2D$ parameters per layer.

We will continue discussing `LayerNorm` instances in what follows in order to adhere to the usual construction and to discuss methods like sequence-parallelism in their original form (see Section 6.2), but note: the data-independent `LayerNorm` transformations due to γ_d, β_d are completely redundant when immediately followed by a `Linear` layer, since both act linearly on their inputs and `Linear` is already the most general data-independent linear transformation. Explicitly, the γ_d, β_d parameters can be absorbed into the `Linear` parameters:

$$(x_{bsd}\gamma_d + \beta_d)W_{dd'} + b_{d'} = x_{bsd}W'_{dd'} + b'_{d'}\,, \quad W'_{dd'} \equiv \gamma_d W_{dd'}\,, \quad b'_{d'} \equiv b_{d'} + \beta_d W_{dd'}\,, \tag{2}$$

for arbitrary x_{bsd}. That is, these transformations can be equivalently performed by the weight matrix and bias (if included) in the `Linear` layer[7].

1.1.3. Causal Attention

Causal attention is the most complex layer. It features A sets of weight matrices[8] $Q_{dea}, K_{dea}, V_{dea}$ where $a \in \{0, ..., A-1\}$ and $e \in \{0, ..., D/A\}$, where D is assumed perfectly divisible by A. From these, we form three different vectors:

$$q_{bsea} = z_{bsd}Q_{dea}\,, \quad k_{bsea} = z_{bsd}K_{dea}\,, \quad v_{bsea} = z_{bsd}V_{dea} \tag{3}$$

---

noise, the transformers architecture would not be able to predict the s-index of each such input in the absence of positional encodings.

[6] Which makes intuitive sense for the purposes of stabilizing the matrix multiplications in the blocks

[7] Note the importance of data-independence here: the data-dependent mean and standard deviation terms cannot be similarly absorbed. Also, because the usual training algorithms are not invariant under parameter redefinitions, the above unfortunately does not imply that removing the `Linear` learnable parameters (`elementwise_affine=False` in `torch`) will have no effect on training dynamics. γ_d, β_d can shoved into the `Linear` layer's parameters as a small inference-time optimization, though.

[8] There are also bias terms, but we will often neglect to write them explicitly or account for their (negligible) parameter count.

These are the **query, key, and value** tensors, respectively[9].

Using the above tensors, we will then build up an **attention map** $w_{bss'a}$ which corresponds to how much attention the token at position s pays to the token at position s'. Because we have the goal of predicting the next token in the sequence, we need these weights to be causal: the final prediction y_{bsv} should only have access to information propagated from positions $x_{bs'v}$ with $s' \leq s$. This corresponds to the condition that $w_{bss'a} = 0$ if $s' > s$. The entire causal Transformers architecture as a whole obeys this condition: the outputs $z_{bsd} = \texttt{CausalTransformer}\left(x_{bs'd'}\right)$ only depend on those inputs $x_{bs'd'}$ with $s' \leq s$.

These weights come from `Softmax`-ed attention scores, which are just a normalized dot-product over the hidden dimension:

$$w_{bss'da} = \texttt{Softmax}_{s'}\left(\frac{q_{bse}k_{bs'ea}}{\sqrt{D/A}} + m_{ss'}\right), \text{s.t.} \sum_{s'} w_{bdss'a} = 1 \tag{4}$$

The tensor $m_{ss'}$ is the causal mask which zeroes out the relevant attention map components above

$$m_{\{ss'\}} = \begin{cases} 0 & s \leq s' \\ -\infty = s > s' \end{cases} \tag{5}$$

forcing $w_{bss'da} = 0$ for $s > s'$. In other words, the causal mask ensures that a given tensor, say z_{bsd}, only has dependence on other tensors whose sequence index, say s', obeys $s' \leq s$. This is crucial for inference-time optimizations, in particular the use of the **kv-cache** in which key-value pairs do not need to be re-computed.

The $\sqrt{D/A}$ normalization is motivated by demanding that the variance of the `Softmax` argument be 1 at initialization, assuming that other components have been configured so that that the query and key components are i.i.d. from a Gaussian normal distribution[10].

The weights above are then passed through a dropout layer and used to re-weigh the **value** vectors and form the tensors

$$y_{bsea} = \texttt{Dropout}\left(w_{bdss'a}\right)v_{bs'ea} \tag{6}$$

and these `(B, S, D/A, A)`-shaped tensors are then concatenated along the e-direction to re-form a `(B, S, D)`-shaped tensor u_{bsd}

$$u_{bsd} = y_{bs(ea)} \tag{7}$$

in `einops`-like notation for concatenation. Finally, another weight matrix $O_{d'd}$ and dropout layer transform the output once again to get the final output

[9]There are of course many variants of the architecture and one variant which is popular in Summer 2023 is multi-query attention vectors and only the query changes across heads, as this greatly reduces

[10]The square root is removed in the normalization espou Appendix F.2

$$z_{bsd} = \texttt{Dropout}\ (u_{bsd'} O_{d'd}). \tag{8}$$

For completeness, the entire operation in condensed notation with indices left implicit is:

$$z \to \texttt{Dropout}\left(\texttt{Concat}\left(\texttt{Dropout}\left(\texttt{Softmax}\left(\frac{(z \cdot Q_a) \cdot (z \cdot K_a)}{\sqrt{\frac{D}{A}}}\right)\right) \cdot z \cdot V_a\right) \cdot O\right) \tag{9}$$

where all of the dot-products are over feature dimensions (those of size D or D/A).

Below is pedagogical[11] sample code for such a `CausalAttention` layer[12]:

```
class CausalAttention(nn.Module):
    def __init__(
        self,
        block_size=K,
        dropout=0.1,
        hidden_dim=D,
        num_attn_heads=A,
    ):
        super().__init__()
        self.block_size = block_size
        self.dropout = dropout
        self.hidden_dim = hidden_dim
        self.num_attn_heads = num_attn_heads

        self.head_dim, remainder = divmod(hidden_dim, num_attn_heads)
        assert not remainder, "num_attn_heads must divide hidden_dim evenly"

        self.Q = nn.ModuleList(
            [nn.Linear(hidden_dim, self.head_dim) for _ in
range(num_attn_heads)]
        )
        self.K = nn.ModuleList(
            [nn.Linear(hidden_dim, self.head_dim) for _ in
range(num_attn_heads)]
        )
        self.V = nn.ModuleList(
            [nn.Linear(hidden_dim, self.head_dim) for _ in
range(num_attn_heads)]
        )
        self.O = nn.Linear(hidden_dim, hidden_dim)

        self.attn_dropout = nn.Dropout(dropout)
```

[11]The code is written for clarity, not speed. An example optimization missing here: there is no need to form separate Q_a, K_a, V_a `Linear` layers, one large layer which is later chunked is more efficient

[12]When using sequence-parallelism, it will be more natural to separate out the final `Dropout` layer and combine it with the subsequent `LayerNorm`, as they are sharded together; see Section 6.2. The same is true for the `MLP` layer below.

```
        self.out_dropout = nn.Dropout(dropout)
        self.register_buffer(
            "causal_mask",
            torch.tril(torch.ones(block_size, block_size)[None]),
        )

    def get_qkv(self, inputs):
        queries = [q(inputs) for q in self.Q]
        keys = [k(inputs) for k in self.K]
        values = [v(inputs) for v in self.V]
        return queries, keys, values

    def get_attn_maps(self, queries, keys):
        S = queries[0].shape[1]
        norm = math.sqrt(self.head_dim)
        non_causal_attn_scores = [(q @ k.transpose(-2, -1)) / norm for q, k in
zip(queries, keys)]
        # Note: this mask shape is a bit of a hack to make generation from the
KV cache work without
        # specifying an extra boolean. When queries and keys have different
sequence lengths and the
        # queries are of seq_len == 1,p the query attends to all of the keys;
effectively there is
        # no mask at all.
        causal_attn_scores = [
            a.masked_fill(self.causal_mask[:, :S, :S] == 0, float("-inf"))
            for a in non_causal_attn_scores
        ]
        attn_maps = [a.softmax(dim=-1) for a in causal_attn_scores]
        return attn_maps

    def forward(self, inputs):
        queries, keys, values = self.get_qkv(inputs)
        attn_maps = self.get_attn_maps(queries, keys)
        weighted_values = torch.cat(
            [self.attn_dropout(a) @ v for a, v in zip(attn_maps, values)],
dim=-1
        )
        z = self.O(weighted_values)
        z = self.out_dropout(z)
        return z
```

The parameter count is dominated by the weight matrices which carry $4D^2$ total parameters per layer.

1.1.4. MLP

The feed-forward network is straightforward and corresponds to

$$z_{bsd} \to \texttt{Dropout}\left(\varphi\left(z_{bsd'}W^0_{d'e}\right)W^1_{ed}\right) \tag{10}$$

where W^0 and W^1 are `(B, S, D)`- and `(E*D, D)`-shaped matrices, respectively (see Appendix A for notation) and φ is a non-linearity[13]. In code, where we again separate out the last `Dropout` layer as we did in in Section 1.1.3:

```
class MLP(nn.Module):
    def __init__(
        self,
        hidden_dim=D,
        expansion_factor=E,
        dropout=0.1,
    ):
        super().__init__()
        self.hidden_dim = hidden_dim
        self.expansion_factor = expansion_factor
        self.dropout = dropout

        linear_1 = nn.Linear(hidden_dim, expansion_factor * hidden_dim)
        linear_2 = nn.Linear(expansion_factor * hidden_dim, hidden_dim)
        gelu = nn.GELU()
        self.layers = nn.Sequential(linear_1, gelu, linear_2)
        self.dropout = nn.Dropout(dropout)

    def forward(self, inputs):
        z = self.layers(inputs)
        z = self.dropout(z)
        return z
```

This bock requires $2ED^2$ parameters per layer, only counting the contribution from weights.

1.1.5. Language Model Head

The layer which converts the `(B, S, D)`-shaped outputs, z_{bsd}, to `(B, S, V)`-shaped predictions over the vocabulary, y_{bsv}, is the **Language Model Head**. It is a linear layer, whose weights are often tied to be exactly those of the initial embedding layer of Section 1.1.1.

1.1.6. All Together

It is then relatively straightforward to tie every thing together. In code, we can first create a transformer block like which corresponds to the schematic function

$$z \to z + \texttt{MLP}(\texttt{LayerNorm}(z + \texttt{CausalAttention}(\texttt{LayerNorm}(z)))) \tag{11}$$

indices suppressed.

[13] The `GeLU` non-linearity is common.

1.1.7. The Loss Function

The last necessary component is the loss function. The training loop data is the `(B, K)`-shaped[14] token inputs (x_{bs}) along with their shifted-by-one relatives y_{bs} where `x[:, s + 1] == y[:, x]`. The `(B, K, V)`-shaped outputs (z_{bsv}) of the `DecoderOnly` network are treated as the logits which predict the value of the next token, given the present context:

$$p\Big(x_{b(s+1)} = v|\ x_{bs}, x_{b(s-1)}, ..., x_{b0}\Big) = \texttt{Softmax}_v\, z_{bsv} \tag{12}$$

and so the model is trained using the usual cross-entropy/maximum-likelihood loss[15]

$$\begin{aligned}\mathcal{L} &= -\frac{1}{BK}\sum_{b,s} \ln p\Big(x_{b(s+1)} = y_{b(s+1)}|\ x_{bs}, x_{b(s-1)}, ..., x_{b0}\Big) \\ &= -\frac{1}{BK}\sum_{b,s} \texttt{Softmax}_v\, z_{bsv}|_{v=y_{b(s+1)}}\ .\end{aligned} \tag{13}$$

Note that the losses for all possible context lengths are included in the sum, equally weighted[16].

In `torch` code, the loss computation might look like the following (using fake data):

```
model = DecoderOnly(
    num_attn_heads=A,
    block_size=K,
    dropout=0.1,
    expansion_factor=E,
    hidden_dim=D,
    num_layers=L,
    vocab_size=V,
)
tokens = torch.randint(model.vocab_size, size=(B, model.block_size + 1))
inputs, targets = tokens[:, :-1], tokens[:, 1:]
outputs = model(inputs)
outputs_flat, targets_flat = outputs.reshape(-1, outputs.shape[-1]),
targets.reshape(-1)
loss = F.cross_entropy(outputs_flat, targets_flat)
```

[14] K is the block size, the maximum sequence-length for the model. See Appendix A.

[15] Here's an alternative derivation for why this loss is minimized when the learned distribution perfectly matches the actual one. Let $p(x)$ be the actual distribution and $q_\theta(x)$ be the model. Taking the continuous case, the expected loss is $\mathcal{L} = -\int \mathrm{d}x\, p(x) \ln q_\theta(x)$. We want to minimize this, subject to the condition that $\int \mathrm{d}x q_\theta(x) = 1$. So, we use the calculus of variations on the loss with a Lagrange multiplier: $\mathcal{L}' = \mathcal{L} + \lambda \int \mathrm{d}x\, q_\theta(x)$. Solving $\frac{\delta \mathcal{L}'}{\delta q_\theta(x)} = 0$ yields $q_\theta(x) = p(x)$. This seems more straightforward and general than the usual argument via the KL-divergence and Jensen's inequality.

[16] In Natural Language Processing (NLP), the perplexity is often reported instead of the loss, which is just the exponential of the loss, a geometric-mean over the gold-answer probabilities: perplexity $= e^{\mathcal{L}} = \Big(\prod_{b,s} p\Big(x_{b(s+1)} = |\ x_{bs}, x_{b(s-1)}, ..., x_{b0}\Big)\Big)^{-\frac{1}{BK}}$.

1.2. Architecture and Algorithm Variants

There are, of course, many variants on the basic architecture. Some particularly important ones are summarized here.

1.2.1. RMS Norm

RMS Norm [7] is a minimized layer norm (Section 1.1.2) which skips removing the mean, adding the bias, and only divides by the mean squared activation:

$$z_{bsd} \leftarrow \left(\frac{z_{bsd}}{\sqrt{\varepsilon + \frac{1}{D}\sum_d z_{bsd}^2}} \right) \gamma_d \equiv \texttt{RMSNorm}_d z_{bsd}. \tag{14}$$

Like in the layer norm case, the weight γ_d is again redundant if immediately followed by a linear layer.

1.2.2. GLU Variants

In [8], Shazeer advocated for replacing the usual linear-then-activation function pattern,

$$z_{d'} = \varphi(W_{d'd} x_d) \tag{15}$$

to

$$z_{d'} = V_{d'e} x_e \varphi(W_{d'd} x_d)\,. \tag{16}$$

So, just perform another linear operation on the original input and broadcast it against the usual activation function output. Biases for can also be included. This construction is typically called "φGLU" where φ is the name of the activation function: ReGLU, SwiGLU/SiGLU ($\varphi = x\sigma(x)$ used in the LLaMA models), etc.

1.2.3. Multi-Query Attention

In [9], the A different key and value matrices are replaced by a single matrix each, while A different query-heads remain. The mechanisms are otherwise unchanged: where there were previously distinct key and value tensors used across different heads, we just use the same tensors everywhere. This is **Multi-Query Attention** (MQA).

The primary reason for multi-query attention is that it vastly reduces the size of the kv-cache (see Section 9.1.2) during inference time, decreasing the memory-burden of the cache by a factor of A. This strategy also reduces activation memory during training, but that is more of a side-effect.

1.2.4. Grouped Attention

Grouped Query Attention (GQA) [10] is the natural extension of multi-query-attention to using $1 < G < A$ matrices for key and value generation. Each of the G different keys gets

matched up with A/G heads (nice divisibility assumed)[17].

1.2.5. Parallel `MLP` and `CausalAttention` Layers

Rather than first pass inputs into the `CausalAttention` layer of each block, and then pass those outputs on to `MLP` in series, GPT-J-6B instead processes the `LayerNorm` outputs in *parallel*. That is, instead of something like

$$z \leftarrow z + \texttt{MLP}(\texttt{LayerNorm}(z + \texttt{CausalAttention}(z))) \tag{17}$$

we instead have[18]

$$z \leftarrow z + \texttt{MLP}(z) + \texttt{CausalAttention}(z)\ . \tag{18}$$

Note that a `LayerNorm` instance is also removed.

1.2.6. RoPE Embeddings

A shortcoming of traditional embeddings $x_{bsd} \to x_{bsd} + p_{sd}$ is that they do not generalize very well: a model trained on such embeddings with a maximum sequence length K will do very poorly when evaluated on longer sequences. RoPE (Rotary Position Embedding) [13] and variants thereof can extend the viable context length by more clever mechanisms with stronger implicit biases.

RoPE and its variants can be motivated by a few natural conditions. Given the queries and keys for an input q_{sh}, k_{sh} (suppressing batch indices), the corresponding attention scores computation $a_{ss'}(q_s, k_{s'})$ should reasonably satisfy the below:

1. The attention score should only depend on the position indices s, s' through their difference $s - s'$, i.e., through their relative distance to each other.
2. The score computation should still be efficient, i.e., based on matrix-mulitiplies.
3. The operation should preserve the scale of the intermediate representations and attention scores, in order to avoid issues with standard normalization.

These conditions suggest a very natural family of solutions: just rotate the usual queries by some fixed element of $SO(H)$ using a generator proportional to the position index and rotate the keys by the conjugate element. That is, replace the q_{sh}, k_{sd} by

$$\begin{aligned} q'_{sh} &\equiv \left[e^{is\hat{n}\cdot T}\right]_{hh'} q_{sh'} \equiv R(s)_{hh'} q_{sh'} \\ k'_{sh} &\equiv \left[e^{-is\hat{n}\cdot T}\right]_{hh'} k_{sh'} \equiv R(s)^{\dagger}_{hh'} k_{sh'}, \end{aligned} \tag{19}$$

which makes their dot-product is $q'_{sh}k'_{s'h} = R(s-s')q_{sh}k_{sh'}$.

[17] Llama-2 [11] uses GQA with $G = 8$, seemingly chosen so that each group can be sharded and put on its own GPU within a standard 8-GPU node.

[18] This alternative layer was also used in PaLM [12] where it was claimed that this formulation is $\sim 15\%$ faster due to the ability to fuse the `MLP` and `CausalAttention`matrix multiplies together (though this is not done in the GPT-J-6B repo above).

Performing the above computation with a dense element of $SO(H)$ is infeasible, as it would require a new dense matrix-multiply by a unique $H \times H$ matrix at each sequence position[19] In the original RoPE paper, the rotation $\hat{n}$ was chosen such that the matrices are 2×2 block-diagonal with the entries of the form[20]

$$R(s)_{[h:h+2][h:h+2]} = \begin{pmatrix} \cos(s\theta_h) & -\sin(s\theta_h) \\ \sin(s\theta_h) & \cos(s\theta_h) \end{pmatrix} \tag{20}$$

where

$$\theta_h = 10^{-4h/H} \, . \tag{21}$$

The RoPE memory costs are thus $\mathcal{O}(KH)$[21]. The sparsity present in this constrained form of the RoPE matrices means that (19) can be computed in $\mathcal{O}(BSH)$ time, rather than $\mathcal{O}(BSH^2)$, as it would be for a general rotation matrix. See the paper for explicit expressions.

While not obvious, the 2x2 form of the RoPE rotations above is in fact completely general[22], after accounting for the aribtitrariness inherent in the key and query projections. A general $SO(N)$ rotation can be expressed in the form $R = C \cdot B \cdot C^T$ where $C \in SO(N)$ and B is block diagonal with 2x2 or 1x1 blocks. This can be demonstrated from the real Schur dedomposition specialized on $SO(N)$ elements or through the block diagonal decomposition after using specific properties of $SO(N)$ eigenvectors and eigenvalues[23]. From this decomposition, we are guaranteed that we can write $R(1) = C \cdot B \cdot B^T$, and hence $R(s) = R(1)^s = C \cdot B^s \cdot C^T$ satisfies all conditions we require, and the `C` factors can be absorbed into the key and query projectors without loss of generality.

1.2.6.1. RoPE Implementation Notes

There are two common implementation strategies which (annoyingly) differ from each other in important ways.

HuggingFace Style: In this implementation, cosine and sines of shape `(seqlen, head_dim)` are provided, $\cos_{sh}$ and $\sin_{sh}$. In this approach, rather than pairing up consecutive dimensions $h, h+1$ to perform the 2x2 rotations over, we pair up head dimension indices $h, h + \frac{H}{2}$. So, the rotated queries, say, are given by

[19] For one, the $\mathcal{O}(SH^2)$ memory cost to store the matrices would be significant. The FLOPs cost is only $2BSH^2$, the same as for other matrix multiplies, but because different matrices are needed at position (it's a batched matrix multiply), these FLOPs would be much more GPU memory-bandwidth intensive.

[20] If H isn't even, the vectors are padded by an extra zero.

[21] A single RoPE buffer can be shared amongst all attention layers and are broadcast across all heads, amortizing the memory costs.

[22] Thanks to Davis Wertheimer for teaching me these facts.

[23] Namely, that eigenvalues generically come in pairs of phases $\exp(\pm i\theta)$, the real and imaginary components of eigenvectors have the same norm, and the real and imaginary components of eigenvectors belonging to different eigenvalue pairs are all mutually orthogonal.

$$q_{sh}{}' = q_{sh}\cos_{sh} + \overline{q}_{sh}\sin_{sh},\ \overline{q}_{sh} \equiv \texttt{Concat}\Big(\Big[-q_{s[\frac{H}{2}:]}, q_{s[:\frac{H}{2}]}\Big]\Big)\ . \tag{22}$$

Complex Style: In this implementation, phases $p_{sh} = \exp(i\theta_{sh})$ are provided, where the d index is `head_dim/2` = $\frac{H}{2}$ dimensional. The query, say, is then reshaped to have a final two-dimensional axis, $q_{sh} \longrightarrow q_{sht}$, $t \in \{0, 1\}$ where the last two axes are interpreted as the real and imaginary components of a complex tensor, respectively: $q_{sht} \longrightarrow \overline{q}_{sh}$. The RoPE-rotated queries then come from taking the real and imaginary parts of $\overline{q}_{sh}\exp(i\theta_{sh})$ and reshaping into a `(seqlen, head_dim)` shaped tensor.

1.2.6.2. YaRN

YaRN [14] is a commonly-used RoPE extension variant based on the following observations. Let S be the original context length that the model was trained on and S' be the target length: $S' > S$.

First, assume that $\theta_h = b^{-\frac{\lfloor \frac{h}{2} \rfloor}{H}} \equiv \frac{2\pi}{\lambda_h}$ for some base[24], so that head h (and $h+1$) undergoes a full rotation when used to describe a relative distance $s = \lambda_h = 2\pi b^{\frac{\lfloor \frac{h}{2} \rfloor}{H}}$. Therefore, higher-indexed heads rotate over a longer token horizon, with the maximal horizon being $\mathcal{O}(b)$. For context lengths $1 \ll S \ll b$, low head-index pairs undergo many rotations, making the RoPE mapping from relative distances to angle degenerate, while high head-index pairs undergo less than a full rotation, leading to the intuition that low head-index pairs will be better at capturing local distance information, while high-index pairs can better capture longer-distance features.

When scaling up to lengths S', the most naive strategy is to rescale the overall angle: $\theta_h \longrightarrow \frac{S}{S'} \times \theta_h$. This is referred to as *Positional Interpolation*, or PI, and ensures that the same maximal rotations are achieved when processing sequences of length up to S' as were seen when originally only processing up to S. The naive PI strategy is empirically found to lead to degraded short-context performance. An alternative strategy is to instead rescale the base, $b \longrightarrow \kappa \times b$ for some $\kappa > 1$. The former strategy is a uniform dilation of the wavelength for all heads, while the latter[25] performs a greater dilation for higher head indices. A choice of κ that connects the two strategies is to choose this parameter such that the angle for the higher head index, θ_{H-2}, is the same in both cases. This is called the NTK-aware scheme.

YaRN aims for a middle ground, with the criteria of preserving the wavelengths of the short-wavelength heads and linearly scaling the long-wavelength heads so that they undergo the same maximal degree of rotation over horizon S' as they previously did over S. That is:

- If $\lambda_h \ll S$, do not adjust the wavelength.
- If $\lambda_h \geq S$, use linear interpolation, as in PI.

Let $r_h \equiv \frac{S}{\lambda_h}$ be the number of rotations that head-pair idx h undergoes over the original context length. We specify two parameters[26], $\alpha < \beta$, that demarcate the boundaries of the above regions

[24] Typical default is $b = 10^4$, as above.

[25] Called ABF for adjusted base frequency [15].

[26] Typical: $\alpha, \beta = 1, 32$.

and a linear interpolation function $0 \leq \gamma_h \leq 1$ such that the YaRN wavelengths are a weighted sum of the PI and original wavelengths:

$$\lambda'_h = (1 - \gamma_h) \times \frac{S'}{S} \times \lambda_h + \gamma_h \lambda_h \tag{23}$$

where high head-index/long-wavelength pairs ($\gamma_h \longrightarrow 0$) get the full linear interpolation and the low head-index pairs ($\gamma_h \longrightarrow 1$) have unchanged wavelengths. The specific form of γ_h is:

$$\gamma_h(r_h) = \begin{cases} 0 & \text{if } r_h < \alpha \\ \frac{r_h - \alpha}{\beta - \alpha} & \text{if } \alpha \leq r_h \leq \beta \\ 1 & \text{if } r_h > \beta \end{cases} \tag{24}$$

or as a function of the head index h:

$$\gamma_h = \begin{cases} 1 & \text{if } \lfloor \frac{h}{2} \rfloor < H \ln\left(\frac{S}{2\pi\beta}\right) \\ \frac{r_h - \alpha}{\beta - \alpha} & \text{if } H \ln\left(\frac{S}{2\pi\beta}\right) \leq \lfloor \frac{h}{2} \rfloor \leq H \ln\left(\frac{S}{2\pi\alpha}\right) \\ 0 & \text{if } \lfloor \frac{h}{2} \rfloor > H \ln\left(\frac{S}{2\pi\alpha}\right) \end{cases} . \tag{25}$$

Finally, YaRN suggests scaling[27] the temperature by $T \longrightarrow \frac{T}{\left(0.1 \ln\left(\frac{S'}{S}\right)+1\right)^2}$, found by a phenomenological fit.

1.2.7. Flash Attention

Flash Attention [16], [17] optimizes the self attention computation by never materializing the $\mathcal{O}(S^2)$ attention scores in off-chip memory. This increases the arithmetic intensity of the computation and reduces the activation memory required, at the expense of needing recomputation in the backwards pass.

The central idea is to decompose the attention computation in the following way. Dropping the batch index, let q_{sd}, k_{sd}, v_{sd} be the queries, keys, and values, and z_{sd} be the final output. Splitting into attention heads as in $q_{sd} = q_{s(ah)} \rightarrow q_{sah}$ and similar, the computation is[28]

$$z_{sah} = \texttt{Softmax}_{s'}(q_{sah'} k_{s'ah'}) v_{s'ah} \tag{26}$$

which is then concatenated as $z_{s(ah)} \rightarrow z_{sd}$ to get the result. We are omitting the (very important) causal mask for clarity of presentation. Because each attention head computation is identical, we also omit the a-index going forward in this section.

The issue is that a naive computation would compute all $\mathcal{O}(S^2)$ components of the attention scores $q_{sh'} k_{s'h'}$ for each attention head and their exponential all at once, which incurs a penalty of shuttling back and forth $\mathcal{O}(S^2)$ elements to and from on-chip memory multiple times in order

[27] Equivalent to scaling the keys and queries by $0.1 \ln\left(\frac{S'}{S}\right) + 1$.

[28] We omit the usual $\sqrt{D/A}$ normalization factor inside the Softmax to de-clutter the presentation. Really, this normalization should just be enforced at the level of the matrices which are used to generate the queries, keys, and values, anyway.

to get the final z_{sh} outputs (in addition to being potentially memory expensive). Flash Attention functions by instead computing the exponentials in stages with fewer memory transfers and never populating the attention scores or exponentials on off-chip memory.

This works by first chunking all of the inputs along their sequence dimensions as in:

- $q_{sh} = q_{(ir)h} \to q_{irh}$ where $i \in \{0, ..., I-1\}$ and $r \in \{0, ..., R-1\}$ with $S = RI$
- $k_{sh} = k_{(jc)h} \to k_{jch}, v_{sh} = v_{(jc)h} \to v_{jch}$ where $j \in \{0, ..., J-1\}$ and $c \in \{0, ..., C-1\}$ with $S = JC$

The chunk sizes are determined by memory constraints, as discussed below. Then, the per-attention-head computation is equivalently written as

$$\begin{aligned} z_{irh} &= \texttt{Softmax}_{jc}\big(q_{irh'}k_{jch'}\big)v_{jch} \\ &= \frac{\exp\big(q_{irh'}k_{jch'}\big)}{\sum_{jc}\exp\big(q_{irh''}k_{jch''}\big)}v_{jch} \\ &\equiv \frac{\sum_j Z_{irjh}}{\sum_{j'c}\exp\big(q_{irh''}k_{j'ch''}\big)} \\ &\equiv \frac{\sum_j Z_{irjh}}{\sum_{j'} L_{ij'r}} \\ &\equiv \frac{Z_{irh}}{L_{ir}} \end{aligned} \tag{27}$$

where we introduced the notation which will be used in the algorithm below. The algorithm proceeds similarly to how it's outlined above: we compute in chunks, looping over i and an inner j loop which is used to compute the numerator and denominator simultaneously.

Ignoring the important causal mask and not tracking the maximum logits (which we should do for numerical stability), the basic version which captures the essentials of the algorithm is below. Additional recomputation is needed for the backwards pass.

For $i \in ...$ `# Computing outputs z[i, r, h] for all r, h`

  Initialize off-chip tensor z_{irh} to zeros

  Move q_{irh} on-chip, instantiate temp Z_{irh} to zeros on-chip.

  For $j \in ...$ `# On-chip compute. r, c indices processed in parallel.`

    Move k_{jch}, v_{jch} on-chip $Z_{irh} \leftarrow Z_{irh} + \exp\big(q_{irh'}k_{jch'}\big)v_{jch}$

    Update numerator $L_{ir} \leftarrow L_{ir} + \sum_c \exp\big(q_{irh'}k_{jch'}\big)$

  Update denominator $z_{irh} \leftarrow \frac{Z_{irh}}{L_{ir}}$ `# Write result off-chip`

Algorithm 1: Flash Attention (Naive - Missing causal mask/max tracking.)

We now analyze the memory transfer costs. As a baseline, vanilla attention requires $\mathcal{O}(S^2 + DS)$ memory transfers per attention head, where the two factors come from the attention scores and q, k, v, respectively. For flash attention, we no longer shuttle the attention scores off-chip, but k, v are repeatedly moved back and forth. These transfers form most of the memory operations in the inner loop above, which access $\mathcal{O}(IJCH) \sim \mathcal{O}\left(\frac{HS^2}{R}\right)$ elements over the lifetime of the algorithm (per attention head). The factor H/R determines the memory-access advantage, and this number is bound by the on-chip memory size. The on-chip bytes from the queries, keys, and vectors take $\mathcal{O}(CH + RH)$ memory and the temporaries from attention scores and exponentials require $\mathcal{O}(RC)$. If we have M bytes of on-chip memory, then we have the constraint $CH + RH + RC \lesssim M$, and assuming assuming the chunks were chosen to maximize on-chip memory usage, $\frac{H}{R} \sim \frac{H^2}{M}$. Since $M \sim 10^5$ bytes on 2023 GPUs, this is a small factor for the typical head dimensions $H \sim 64$, as desired.

Flash attention is also a big win for activation memory: a naive algorithm has a $\mathcal{O}(ABS^2)$ per-layer contribution to activation memory due to needing to save the attention weights, but these are discarded and re-computed for flash attention. The only additional memory cost comes from the $\mathcal{O}(ABS)$ elements in the ℓ_{abs} statistics, which are dominated by the $\mathcal{O}(BSD)$ costs from needing to save inputs, and hence negligible.

1.2.7.1. The Details

Here we give more detailed descriptions of the flash-attention forwards and backwards passes.

For the forwards pass, we add in maximum-logits tracking for more numerically stable exponential computation and the causal mask. The causal mask $C_{ss'} = C_{(ir)(jc)}$ is zero if $s \geq s'$ and $-\infty$ otherwise. The algorithm is as below.

For $i \in$ ... `#Computing outputs  z[i, r, h] for all r, h`

  Initialize off-chip tensors z_{irh}, ℓ_{ir} to zeros

  Move q_{irh} on-chip, instantiate temp Z_{irh} to zeros and $M_{ir}^{\text{new}}, M_{ir}^{\text{old}}$ to $-\infty$ on-chip

  For $j \in$ .. `# On-chip compute. r, c indices processed in parallel`

    Move k_{jch}, v_{jch} on-chip

    $S_{irjc} \leftarrow q_{irh'} k_{jch'} + C_{ijrc}$ `# logits + causal mask`

    $M_{ir}^{\text{new}} \leftarrow \max\left(M_{ir}^{\text{old}}, \max_c S_{irjc}\right)$

    $Z_{irh} \leftarrow Z_{irh} + \exp\left(S_{ijrc} - M_{ir}^{\text{new}}\right) v_{jch}$ `# Update numerator`

    $L_{ir} \leftarrow e^{M_{ir}^{\text{old}} - M_{ir}^{\text{new}}} L_{ir} + \sum_c \exp\left(S_{ijrc} - M_{ir}^{\text{new}}\right)$ `# Update denominator`

    $M_{ir}^{\text{old}} \leftarrow M_{ir}^{\text{new}}$

  $z_{irh} \leftarrow \frac{Z_{irh}}{L_{ir}}, \ell_{ir} \leftarrow M_{ir}^{\text{old}} + \ln L_{ir}$ `# Write off-chip. l[i, r] saved for bwd`

Algorithm 2: Flash Attention Forward Pass

For the backwards pass, the main complication comes from computing derivatives with respect to the attention scores. Recalling the `Softmax` derivative (96).

given gradients $\frac{\partial \mathcal{L}}{\partial z_{irj}} \equiv g_{irj}$ we have the building blocks[29]

$$\begin{aligned}
\frac{\partial P_{irjc}}{\partial S_{irj'c'}} &= P_{irjc}\delta_{jj'}\delta_{cc'} - P_{ijrc}P_{irj'c'} \\
\frac{\partial \mathcal{L}}{\partial P_{irjc}} &= g_{irh}v_{jch} \\
\frac{\partial \mathcal{L}}{\partial S_{irjc}} &= g_{irh}\frac{\partial P_{irjc}}{\partial S_{irj'c'}} \\
&= g_{irh}\left(P_{irjc}v_{jch} - P_{irjc}P_{irj'c'}v_{j'c'h}\right) \\
&= g_{irh}\left(P_{irjc}v_{jch} - P_{irjc}z_{irh}\right) \\
&= P_{irjc}\left(\frac{\partial \mathcal{L}}{\partial P_{irjc}} - g_{irh}z_{irh}\right)
\end{aligned} \tag{28}$$

from which we compute

$$\begin{aligned}
\frac{\partial \mathcal{L}}{\partial v_{jch}} &= g_{irh}P_{irjc} \\
\frac{\partial \mathcal{L}}{\partial q_{irh}} &= \frac{\partial \mathcal{L}}{\partial S_{irjc}}k_{jch} \\
\frac{\partial \mathcal{L}}{\partial k_{jch}} &= \frac{\partial \mathcal{L}}{\partial S_{irjc}}q_{irh}
\end{aligned} \tag{29}$$

Above we let $P_{ijrc} \equiv \texttt{Softmax}_{jc}(S_{irjc})$ where $S_{ijrc} \equiv q_{irh'}k_{jch'} + C_{ijrc}$, keeping notation similar to the above algorithm. All of this suggests a very similar algorithm to the above. Using the unfortunate, but common, notation, $\mathrm{d}X = \frac{\partial \mathcal{L}}{\partial X}$ the algorithm is[30]:

[29]The fact that we can replace the j' sum with the cached attention outputs in the final derivative below is crucial.

[30]In the FA2 paper, they actually pre-compute the $g_{irh}z_{irh}$ sum prior to the main loop and store it in a tensor they call D. And in the official `triton` example, $\mathrm{d}q$ is computed in a separate loop. So, take the below as more of a guideline than a strict recipe.

For $i \in \ldots$
  Initialize off-chip tensors $\mathrm{d}q_{irh}, \mathrm{d}k_{jch}, \mathrm{d}v_{jch}$ to zeros
  Move $z_{irh}, q_{irh}, g_{irh}$ and the cached ℓ_{ir} + on-chip.
  For $j \in \ldots$ `# On-chip compute. r, c processed in parallel`
    Instantiate $P_{irjc}, \mathrm{d}P_{irjc}, \mathrm{d}S_{irjc}$ to zeros.
    Move k_{jch}, v_{jch} on-chip
    $P_{irjc} \leftarrow \exp\left(q_{irh'}k_{jch'} + C_{ijrc} + \ell_{ir}\right)$ `# Get probabilities`
    $\mathrm{d}P_{irjc} \leftarrow g_{irh}v_{jch}$ `# Get derivatives w.r.t. P`
    $\mathrm{d}S_{irjc} \leftarrow P_{ijrc}\left(\mathrm{d}P_{ijrc} - g_{irh}z_{irh}\right)$ `# Get derivatives w.r.t. S`
    $\mathrm{d}k_{jch} \leftarrow \mathrm{d}S_{irjc}q_{irh}$ `# Get derivatives w.r.t. q, k, v`
    $\mathrm{d}v_{jch} \leftarrow g_{irh}P_{irjc}$ `# Write k, v derivatives to off-chip`
    $\mathrm{d}q_{irh} \leftarrow \mathrm{d}q_{irh} + \mathrm{d}S_{irjc}k_{jch}$
  Write $\mathrm{d}q_{irh}$ derivative off-chip

Algorithm 3: Flash Attention Backwards Pass

1.2.8. Implementation Notes

In addition to the final outputs, z_{sah}, typical flash attention implementations will also return the logarithm of the sum of exponentials (minus the max value) which provide the normalization factor. Usually called `lse` or `losgumexp`:

$$\begin{aligned}\ell_{as} &= \log\left(\sum_{s'}\exp\Big(q_{sah}k_{s'ah} - \max_{s'}(q_{sah}k_{s'ah})\Big)\right)\\ z_{sah} &= \exp\Big(q_{sah'}k_{s'ah'} - \max_{s'}(q_{sah}k_{s'ah}) - \ell_{as}\Big)v_{s'ah}\end{aligned} \tag{30}$$

This quantity is important and frequently used, e.g. in distributed attention implementations.

2. Linear Attention

2.1. Intro

The all-to-all attention mechanism of transformers is a pain: $\mathcal{O}(S^2)$ compute at training time and $\mathcal{O}(S)$ next-token generation. Linear attention models return, more or less, to the old LSTM type strategy of encoding the conditional history into finite-sized state. The dream is faster generation and better memory efficiency:

- Parallelizable[31] $\mathcal{O}(S)$ training.
- Constant $\mathcal{O}(1)$ generation.

[31] Better parallelization support is what differentiated S4 models from their RNN/LSTM predecessors; see Section 2.4.5.

- Sequence-length-independent state, reducing the inference-time memory bandwidth burden compared to the kv-cache; Section 9.1.2.

2.2. General Structure

We follow the unified discussion of [18] on linear attention models here, to get a general orientation. Later, we discuss specific linear attention variants.

2.2.1. Simple Linear Attention

The simplest linear attention variant comes from just removing the `Softmax` operation. The linear attention outputs are of the form[32]:

$$o_{sd} = q_{sd'} k_{s'd'} v_{s'd} m_{ss'} \ , \tag{31}$$

with the crucial point being that the d' and s' sums can be done in any order, whereas for `Softmax` -attention the d' sum must be done first[33]. The outputs can be therefore written as

$$o_{sd} = q_{sd'} S_{sd'd} \ , \tag{32}$$

where the state $S_{sd'd}$ is recursively built up as[34]

$$S_{sd'd} = S_{(s-1)d'd} + k_{sd'} v_{sd} \ , \ S_{-1d'd} \equiv 0 \ . \tag{33}$$

This property enables $\mathcal{O}(1)$ inference, since only the most recent state needs to be cached, rather that the full $\mathcal{O}(S)$ KV-cache.

This simplest version of linear attention is not very performant, especially at long sequence lengths, and a zoo of variants exists. Intuitively [19], the lack of performance is likely connected to following facts:

- At any given time step, we are compressing all information into a fixed-size $\mathcal{O}(D^2)$ and S-independent state, unlike in `Softmax`-attention where the state (KV-cache) is $\mathcal{O}(S)$
- Information is continually added to the state. There is no explicit mechanism to replace old information or avoid adding present information.

Gating and the delta-rule are two architecture alterations which attempt to alleviate these pain points, as discussed below.

2.2.2. $S_{sd'd}$ as a Look-up Table

A common mental model is to think of $S_{sd'd} = \sum_{s'=0}^{s} k_{s'd'} v_{s'd}$ as a noisy dictionary we are extracting information from. That is, in the limit where the k_{sd} each have unit norm and have small inner products for $s \neq s'$, then we can extract out a given value at s' up to an error term

[32] We neglect the causal mask for simplicity.

[33] Here $m_{ss'}$ is the causal mask.

[34] No sum over s in the RHS of either of these expressions now. Other linear attention architectures may not have this specific key-and-value structure for building up state, but they will have this general additive, recursive structure.

as in[35]

$$k_{s'd'}S_{sd'd} = v_{s'd} + \sum_{s''\neq s'} v_{s''d}k_{s''d'}k_{s'd'} \equiv v_{s'd} + \varepsilon \ . \tag{34}$$

From this viewpoint, the limitation with constant-space state and long sequence is clear: there can be at most D orthogonal keys, whereas we may be storing $S \gg D$ steps worth of information. Songling Yang has a nice writeup here.

2.2.3. Gating

The ability to remove old information can be implemented by *gating*: we add a (data-dependent) prefactor to the recursion relation

$$S_{sd'd} = \alpha_s S_{(s-1)d'd} + k_{sd'}v_{sd} \ , \tag{35}$$

where usually $0 \leq \alpha_s \leq 1$. In general, α_s may have a more complex tensor structure, but we keep it simple here. For sufficiently expressive α_s, gating can completely erase old state when needed, e.g. when the subject of a conversation completely changes.[36] The mamba models are one example which using gating.

2.2.4. Delta Rule

Rather than completely erasing old state, it may be desirable to selectively overwrite old state, which is what the *delta rule* strives for [19]. The form of the update is

$$S_{sd'd} = (\delta_{d'd''} - \beta_s k_{sd'}k_{sd''})S_{(s-1)d''d''} + \beta_s k_{sd'}v_{sd} \tag{36}$$

where β_s is again data-dependent[37].

Taking the viewpoint in Sec. Section 2.2.2 that we query the state $S_{sd'd}$ as in $k \cdot S \sim v$, the above modification makes the architecture capable of removing entries from the dictionary. For instance, in the limit that $\beta_s = \frac{1}{k_s \cdot k_s}$ the matrix $\mathbf{1} - \beta_s k_{sd''}k_{sd'}$ is a projector onto the space orthogonal[38] to k_{sd} and so this removes the corresponding entry from the old dictionary at step $s-1$ and overwrites it with the new $\sim v \otimes k$ entry. Often the k and v tensors are explicitly normalized prior to use, as well.

2.3. Efficient Implementations

Efficient kernel implementations use tricks for linear attention. We start with vanilla linear attention.

[35] Of couse, there is no reason to assume the k_{sd} are nearly orthonormal in general, and they cannot be if $S > D$, but this is nevertheless a sometimes-useful perspective.

[36] For instance, we could ask our architecture to process two independent sequences concatenated together with a special separator token in the middle. The hidden state should be reset at the separator token and a gated linear attention architecture would be (in-principle) capable of this.

[37] The value range of β_s varies by model, but usually it's between zero and 1 or −1 and 1.

[38] That is, $S \cdot (1 - k \otimes k) \cdot k = 0$, schematically.

2.3.1. Flash Linear Attention

Two natural, but naive, approaches to linear attention are suboptimal on modern hardware:

- Fully parallelized: for $o_{sd} = q_{sd'} k_{s'd'} v_{s'd} m_{ss'}$, compute $q \cdot k$ first, apply the mask, and then dot into the values. The positive is that this uses GPU-friendly matmuls, but the total computational complexity is too high[39]: $\mathcal{O}(S^2 D + D^2 S)$, rather than being purely linear in S.
- Fully recurrent: use recursion to compute $o_{sd} = q_{sd'} S_{sd'd}$ as in (33), iteratively building up $S_{sd'd}$ at total cost $\mathcal{O}(SD^2)$. This is cheaper in FLOPs, realizing linear-in-sequence scaling $\mathcal{O}(D^2 S)$, but all of the operations have low arithmetic intensity.

A middle-ground chunked strategy can perform better than either of the above. We chunk the sequence index $s \to (tr) \to tr$ into chunks of size R with $t \in \left\{0, ..., \frac{S}{R} - 1\right\}$ and $t \in \{0, ..., R - 1\}$. We then compute:

$$o_{trd} = q_{trd'} S_{trd'd} \tag{37}$$

where the recursion relation becomes:

$$\begin{aligned} S_{trd'd} &= S_{(t-1)0d'd} + \sum_{r'=0}^{r} k_{tr'd'} v_{tr'd} \\ &\equiv \mathcal{S}_{(t-1)d'd} + \sum_{r'=0}^{r} k_{tr'd'} v_{tr'd}, \end{aligned} \tag{38}$$

introducing notation for the state at the start of each chunk boundary. Getting only the states at chunk boundaries is easy and GPU friendly, since the sum extends over the entire r-index (i.e. it is a matmul):

$$\mathcal{S}_{td'd} = \mathcal{S}_{(t-1)d'd} + k_{trd'} v_{trd} \ , \tag{39}$$

all such tensors computable in $\mathcal{O}(D^2 S)$ time. And so the entire output can be written as:

$$o_{rtd} = q_{trd'} \mathcal{S}_{(t-1)d'd} + q_{trd'} \sum_{r'=0}^{r} k_{tr'd'} v_{tr'd} \ . \tag{40}$$

The first term takes $\mathcal{O}(D^2 S)$ time, again[40], and the second can be computed in the fully-parallelized manner described above:

$$q_{trd'} \sum_{r'=0}^{r} k_{tr'd'} v_{tr'd} = q_{trd'} k_{tr'd'} v_{tr'd} m_{rr'} \ , \tag{41}$$

[39] Overloading S to be both sequence length and state here.

[40] Note, however, that these are more efficient flops than those of the $\mathcal{O}(D^2 S)$ flops of the recurrent implementation. Here, this scaling arises from a true matmul, which may have high arithmetic intensity, while in the recurrent case the scaling comes from performing D separate vector outer products, $\sim k \otimes k$, which are all necessarily low arithmetic intensity.

where $m_{rr'}$ is the inter-chunk mask. The time complexity of the above is $\mathcal{O}(SRD)$. Note that the $R \longrightarrow 1$ and $R \longrightarrow S$ limits reduce to the fully recurrent and fully parallel limits, respectively.

So, for $1 \ll R \ll S$, then this is both:
1. Parametrically cheaper than the fully parallelized implementation.
2. Hardware friendly[41]

There are further important implementation details, such as whether all the $\mathcal{S}$ are realized in HBM or not as this affects the degree of parallelism, but the above is the core strategy that everything builds on.

2.3.2. Gated

Let's add a gate in the following form:

$$S_{sd'd} = \alpha_{sd'} S_{(s-1)d'd} + k_{sd'} v_{sd} \ . \tag{42}$$

The corresponding parallelized form of the outputs is

$$o_{sd} = \prod_{s''=1}^{s-1} \alpha_{s''d'} q_{sd'} k_{s'd'} v_{s'd} m_{ss'} \ . \tag{43}$$

If we let $b_{sd} = \prod_{s'=1}^{s-1} \alpha_{s'd}$ with $b_{-1} \equiv 1$, then we could write the above as

$$o_{sd} = (q_{sd'} b_{sd'}) \left(\frac{k_{d's'}}{b_{s'd'}} \right) v_{s'd} m_{ss'} \ , \tag{44}$$

and use the linear attention strategy, since the above is of exactly the same form with modifies queries and keys. But this form is unacceptably numerically unstable (since the division is likely to fail and `nan`). So we instead need to do something like:

$$o_{sd} = q_{sd'} k_{d's'} \exp(\ln b_{sd'} - \ln b_{s'd'}) v_{s'd} m_{ss'} \ , \tag{45}$$

Chunking again[42], we have:

$$S_{trd'd} = \prod_{r'=0}^{r} \alpha_{tr'd'} \mathcal{S}_{(t-1)d'd} + \sum_{r'=0}^{r} \prod_{r''=r'+1}^{r-1} \alpha_{tr''d'} k_{tr'd'} v_{tr'd} \ , \tag{46}$$

and the boundary value recursion simplifies to

$$\mathcal{S}_{td'd} = \prod_{r} \alpha_{crd'} \mathcal{S}_{(t-1)d'd} + \sum_{r'=0} \prod_{r''=r'+1}^{R-1} \alpha_{tr''d'} k_{tr'd'} v_{tr'd} \tag{47}$$

---

[41] Specifically, taking the chunk size R to be a multiple of 16, allows for the use of the 16x faster tensor cores on modern GPUS, which can make up the difference in scaling. Roughly, the recurrent flops are $\mathcal{O}(SD^2)$ but they are all at least 16x slower than the chunk implementation flops, so the recurrent to chunk time ratio is something like $\frac{16SD^2}{SD^2+SRD} = \frac{16}{1+\frac{R}{D}}$, meaning we are faster as long $R \lesssim 10D$.

[42] Almost certainly have some off-by one or worse errors in this section.

$$\equiv \mathcal{B}_{td'}\mathcal{S}_{(t-1)d'd} + \sum_{r'=0}\prod_{r''=r'+1}^{R-1}\alpha_{tr''d'}k_{tr'd'}v_{tr'd} \tag{47}$$

The last term can be computed in a GPU friendly way by, say, first computing $\prod_{t''=t'+1}^{R-1}\alpha_{tr''d'}k_{tr'd'}$ and finally computing the dot product on r'.

And so the entire output can be written as:

$$o_{trd} = q_{trd'}\prod_{r'=0}^{r}\alpha_{tr'd'}\mathcal{S}_{(t-1)d'd} + q_{trd'}\sum_{r'=0}^{r}\prod_{r''=r'+1}^{r-1}\alpha_{tr''d'}k_{tr'd'}v_{tr'd}\ . \tag{48}$$

The first term we compute in $\mathcal{O}(D^2S)$, first solving the recursion relation for the $\mathcal{S}_{tdd'}$ computing $q_{trd'}\prod_{r'=0}^{r}\alpha_{tr'd'}$ and dotting the result against $\mathcal{S}_{(t-1)d'd}$, all relatively GPU friendly operations which are themselves constructed in $\mathcal{O}(D^2S)$ via recursion. The final term is more challenging; finish later.

2.3.3. Delta

An efficient chunking strategy for the delta rule brings in some new elements. We chunk (36) as above and partially unroll:

$$\begin{aligned}
S_{trd'd} &= (\delta_{d'd''} - \beta_{tr}k_{trd'}k_{trd''})S_{t(r-1)d''d''} + \beta_{tr}k_{trd'}v_{trd} \\
&= \prod_{r'=0}^{r}(\delta_{d'd''} - \beta_{tr'}k_{tr'd'}k_{tr''d''})\mathcal{S}_{td''d} \\
&\quad + \sum_{r''=0}^{r}\prod_{r'=r''+1}^{r}(\delta_{d'd''} - \beta_{tr'}k_{tr'd'}k_{tr'd''})\beta_{tr''}k_{tr''d'}v_{tr''d} \\
&\equiv \left(\delta_{d'd''} - \sum_{r'=0}^{r}k_{tr'd'}w_{tr'd''}\right)\mathcal{S}_{td''d} + \sum_{r'=0}^{r}k_{tr'd'}u_{tr'd}
\end{aligned} \tag{49}$$

defining the w_{trd} and u_{trd} tensors, following [20]. The total output of the layer can be written as

$$\begin{aligned}
o_{trd} &= q_{trd'}S_{trd'd} \\
&= q_{trd'}\left(\delta_{d'd''} - \sum_{r'=0}^{r}k_{tr'd'}w_{tr'd''}\right)\mathcal{S}_{td''d} + q_{trd'}\sum_{r'=0}^{r}k_{tr'd'}u_{tr'd} \\
&= q_{trd'}\mathcal{S}_{td''d} + q_{trd'}\sum_{r'=0}^{r}k_{tr'd'}(u_{tr'd} - w_{tr'd''}\mathcal{S}_{td''d}) \\
&= q_{trd'}\mathcal{S}_{td''d} + q_{trd'}k_{tr'd'}m_{rr'}(u_{tr'd} - w_{tr'd''}\mathcal{S}_{td''d})
\end{aligned} \tag{50}$$

where we inserted the inter-chunk causal mask $m_{rr'}$ in the final line so that we can compute the explicit sum via a matmul.

The final expression above is hardware friendly, but requires knowing the u_{trd} and w_{trd} tensors, which were only implicitly defined above. Recursive expressions for these quantities follow from the expressions above, but they suffer from the now-familiar affliction of being hardware unfriendly if actually computed via direct recurrence[43]. Fortunately, there is a neat trick for computing these in an alternate way.

Start with w_{trd} and let us suppress the d index in what follows for simplicity; dot product dimensions are hopefully clear from context. Then, from its definition, w_{tr} is given by

$$w_{tr} = \beta_{tr} k_{tr} \cdot \left(\mathbf{1} - \beta_{t(r-1)} k_{t(r-1)} \otimes k_{t(r-1)}\right) \cdot \ldots \cdot \left(\mathbf{1} - \beta_{t0} k_{t0} \otimes k_{t0}\right) \ . \tag{51}$$

Now if we let

$$A_{trr'} = \begin{cases} -\beta_{tr} k_{tr} \cdot k_{t(r-1)} & r < r' \\ 0 & r \geq r' \end{cases} \tag{52}$$

then the above can be written as:

$$\begin{aligned} w_{tr} &= \beta_{tr} k_{tr} + A_{rr'} \beta_{tr'} k_{tr'} + A_{rr'} A_{r'r''} \beta_{tr''} k_{tr''} + \ldots \\ &\equiv \left(\mathbf{1} + A + A^2 + \ldots + A^{r-1}\right)_{rr'} \beta_{tr'} k_{tr'} \\ &= \left(\mathbf{1} - A\right)^{-1}_{rr'} \beta_{tr'} k_{tr'} \end{aligned} \tag{53}$$

where the final equality relies on the fact that the $A_{rr'}$ matrix above obeys the special quality that $A^n = 0$ for $n \geq r$ (ultimately due to causality). The above can be proven either from the usual Taylor matrix expansion (assuming convergence) which terminates early due to the preceding property, or just from direct computation:

$$\begin{aligned} \left(\mathbf{1} - A\right) \cdot \left(\mathbf{1} + A + A^2 + \ldots + A^{r-1}\right) &= \left(\mathbf{1} + A + A^2 + \ldots + A^{r-1}\right) \\ &\quad - A \cdot \left(\mathbf{1} + A + A^2 + \ldots A^{r-1}\right) \\ &= \left(\mathbf{1} + A + A^2 + \ldots A^{r-1}\right) \\ &\quad - \left(A + A^2 + \ldots A^{r-1}\right) \\ &= \mathbf{1} \ . \end{aligned} \tag{54}$$

Because of the triangular nature of $A_{rr'}$, the inverse can be computed relatively cheaply and then all of the remaining operations are matmuls. In exactly the same way, we can also derive a similar equation for u_{tr}:

$$u_{tr} = \left(\mathbf{1} - A\right)^{-1}_{rr'} \beta_{tr'} v_{tr'} \ . \tag{55}$$

[43] The explicit expressions are in [20], and are of the form $w_{trd} = B_{trd} + A_{trr'} w_{tr'd}$ for some tensors B, A with $A_{rr'} = 0$ if $r' \geq r$ (making the relation recursive), from which it follows that $w_{trd} = \left(\mathbf{1} - A_t\right)^{-1}_{rr'} B_{tr'd}$; thanks to this Eleuther AI video for this explanation. Below we go through a more explicit form of this argument.

This trick is related to the UT transformation [21] (though it's presented fairly differently), and the recursive representation of w and u (that we didn't write) is called the WY representation [22].

The final, hardware friendly representation of the delta model outputs is:

$$\begin{aligned} o_{trd} = & q_{trd'}\mathcal{S}_{td''d} \\ & + q_{trd'}k_{tr'd'}m_{rr'}(\mathbf{1}-A)^{-1}_{r'r''}\beta_{tr''}v_{tr''d} \\ & - q_{trd'}k_{tr'd'}m_{rr'}(\mathbf{1}-A)^{-1}_{r'r''}\beta_{tr''}k_{tr''d''}\mathcal{S}_{td''d} \ . \end{aligned} \tag{56}$$

Computational complexity:
1. Compute the first line in $\mathcal{O}(SD^2)$
2. Compute $q_{trd'}k_{tr'd'}m_{rr'}$ in $\mathcal{O}(SRD)$.
3. Compute $(\mathbf{1}-A)^{-1}_{r'r''}$ in $\mathcal{O}(R^2)$.
4. Compute all remaining matmuls in $\mathcal{O}(SR^2 + SRD + SD^2)$

2.4. Mamba2

2.4.1. Architecture Overview

Mamba2 is a gated, linear attention variant. It is somewhat complex, so we sketch an overview here:
1. The inputs x_s are projected to create several different intermediates y_s
2. A short-filter, depthwise, causal convolution is performed over many of the intermediates, with some having an activation function applied subsequently.
3. A linear, gated recursion relation along the lines of Sec. Section 2.2.3 is applied to build up internal state, with the inputs and gating factors built from the preceding intermediates.
4. The recursion outputs are multiplied by another elementwise gate, and the final output comes from norming and projecting.

2.4.2. Architecture Details

Now the specifics. The inputs x_{sd} create the initial intermediates through projections, activation functions, and convolutions:

$$\begin{aligned} h_{se} &= K^h_{ess'} * \texttt{silu}\big(W^h_{ed}x_{sd}\big) \\ \Delta_{sa} &= \texttt{Softplus}\big(W^\Delta_{ae}x_{se}\big) \\ G_{se} &= \texttt{silu}\big(W^G_{ed}x_{sd}\big) \\ B_{sgn} &= K^B_{ess'} * W^B_{gne}x_{se} \\ C_{sgn} &= K^C_{ess'} * W^C_{gne}x_{se} \end{aligned} \tag{57}$$

which can be implemented through fused matmuls and convolutions. The meaning of the various new indices is discussed below. A combination of Δ, h, and B form the hidden state which gets

added at every step in the recursion relation[44], Δ controls the gating ($\alpha_s \sim e^{\Delta W^A}$ in (35)), G_{se} controls the final gate, C is used as an intermediate projection.

Given the above, a particular scan (discussed in detail below) is performed to create another intermediate,

$$y_{se} = \texttt{scan}\big(h_{se}, \Delta_{sa}, B_{sgn}, C_{sgn}, W_a^D, W_a^A\big) \tag{58}$$

and the final output of the model is a projection of the normed, gated scan outputs:

$$z_{se} = W_{de}^O \ \texttt{Norm}_e(y_{se} \otimes G_{se}) \ . \tag{59}$$

Circling back to notation, we have:

- $d \in (0, ..., D-1)$: the input's hidden dimensions, `d_model`.
- $s \in (0, ..., S-1)$: sequence length.
- $a \in (0, ..., A-1)$: an attention-head-like index.
- $h \in \big(0, ..., \frac{D}{H} - 1\big)$: a head-dim-like index.
- $e \in (0, ..., E \times D - 1)$: expanded internal hidden dimension[45].
- $n \in (0, ..., N-1)$: another internal hidden dimension, controlling the size of the internal memory; `d_state`[46].
- `d_conv` (not yet mentioned) is the convolution kernel size[47].
- Learnable parameters are the W's (projections) and K's (convolutional kernels). W_{en}^A is notable in that it is strictly non-positive[48].

The scan operation first divides the intermediate h_{se} into heads, $h_{se} \to h_{s(ah)}$, solves the recursive relation[49]

$$w_{sgahn} = \exp\big(\Delta_{sa} W_a^A\big) w_{(s-1)gahn} + \Delta_{sa} B_{sgn} h_{sah} \tag{60}$$

and returns the final result[50]

$$y_{sah} = C_{sgn} w_{sgahn} + W_a^D x_{sah} \tag{61}$$

after re-concatenating the heads: $y_{s(ah)} \longrightarrow y_{se}$. We can, of course, also write out the full solution as

[44] The analogue of the $v_s \cdot k_s$ term in Equation 33.

[45] Default: $E = 2$.

[46] Defaults: $N = 16$.

[47] Default: 4.

[48] Implemented as $W^A = -\exp(A)$ where A is the actual learnable weight in `mamba-ssm`.

[49] This is the Mamba2 analogue of the $S_s = \alpha_s S_{s-1} + v_s \cdot k_s$ form in Section 2.2.3.

[50] Note that each recursion step requires $\mathcal{O}(AHGN) = \mathcal{O}(DGN)$ operations, and so we should be able to optimally solve the entire recursion in $\mathcal{O}(SDGN)$ time.

$$y_{sah} = C_{sgn}\left(\sum_{s'=0}^{s} e^{\Delta_{sa}W_a^A} \times \ldots \times e^{\Delta_{(s'+1)a}W_a^A}\Delta_{s'a}B_{s'gn}x_{s'ah}\right) + W_a^D x_{sah}$$
$$= C_{sgn}\left(\sum_{s'=0}^{s} e^{\sum_{s''=s'+1}^{s}\Delta_{s''a}W_a^A}\Delta_{s'a}B_{s'gn}h_{s'ah}\right) + W_a^D x_{sah} \tag{62}$$

2.4.3. Mamba2 Duality with Attention

There are only two steps in which tokens at different temporal positions interact in the Mamba2 model:

1. In the short 1D convolution.
2. In the recurrence relation, where we create the intermediate

$$y_{sah} = C_{sgn}\left(\sum_{s'=0}^{s} e^{\Delta_{sa}W_a^A} \times \ldots \times e^{\Delta_{(s'+1)a}W_a^A}\Delta_{s'a}B_{s'gn}w_{s'ah}\right) \equiv M_{ass'}w_{s'ah} \tag{63}$$

   which is the most complicated step of the model.

ws noted above, the second case is ultimately just a matrix-multiply on the input tensors w_{sah} with the tensor $M_{ass'}$, where operations across attention head are all independent. The $M_{ass'}$ tensor has $\mathcal{O}(AS^2)$ elements, which we clearly do not want to concurrently materalize. All of this should sound familiar: the above is highly analogous to the structure and problems of flash attention, Section 1.2.7, albeit with important differences in the details of the operator $M_{ass'}$. This is the "duality" discussed in [23] and the broad strokes of the efficient algorithm implementation for Mamba2 echos that of flash attention: partition the computation over the sequence dimensions and compute y_{sah} in chunks over the s-dimension, so as to avoid realizing any $\mathcal{O}(S^2)$ tensors.

2.4.4. Details: Cumsums, Chunking, and the Mamba2 Scan

Some more details about how to compute the recursion solution in (60). Similarly to the previous section, let

$$\begin{aligned} y_{sah} &= C_{sgn}\left(\sum_{s'=0}^{s} e^{\Delta_{sa}W_a^A} \times \ldots \times e^{\Delta_{(s'+1)a}W_a^A}\Delta_{s'a}B_{s'gn}w_{s'ah}\right) \\ &\equiv C_{sgn}\mathcal{A}_{ss'a}\Delta_{s'a}B_{s'gn}w_{s'ah} \\ &\equiv C_{sgn}\mathcal{A}_{ss'a}\mathcal{B}_{s'ganh} \end{aligned} \tag{64}$$

The above is the most complex part of the Mamba2 model and the official `mamba_ssm` repo takes a multi-step approach to its computation. A few important points:

1. The matrix $\mathcal{A}_{ss'a}$ vanishes for $s' > s$ (causality).
2. As in flash attention, we wish to chunk over the sequence dimension to avoid every realizing the full $\mathcal{A}_{ss'a}$ tensor.

3. Unlike flash attention, there must be a smarter way to perform the sums over s, s': the naive computation above would be $\mathcal{O}(S^2)$, while the express purpose of state space models is to reduce the attention-like operation to $\mathcal{O}(S)$.

The chunked version is then

$$z_{clah} = C_{clgn}\mathcal{A}_{cc'll'a}\mathcal{B}_{c'l'ganh} \,, \tag{65}$$

where we have chunked the a-index into the c, l pair with L the size of each chunk ,$l \in \{0, ..., L-1\}$ $c \in \left\{0, ..., \frac{S}{L} - 1\right\}$ indexing the chunk and). The chunked computation breaks down into two further cases, based on the values of the c, c' indices[51]:

- $c = c'$: these cases are effectively smaller versions of the entire, unchunked computation, and hence shares in its sparsity in that $\mathcal{A}_{cc'll'a}$ vanishes for $l' > l$.
- $c > c'$: there is no sparsity here, as $\mathcal{A}_{cc'll'a}$ will be generically non-zero for all l, l'.

2.4.4.1. The $c = c'$ Cases

Logically, we compute the scan using cumulative sums and matrix-multiplies. Let $\Delta_{sa}W_a^A = A_{sa} = A_{(cl)a} = A_{cla}$ so that

$$\begin{aligned}\mathcal{A}_{ss'a} &= \begin{cases} e^{A_{sa}} \times ... \times e^{A_{(s'+1)a}} = \exp\left(\sum_{s''=s'+1}^{s} A_{s''a}\right) & s \geq s' \\ 0 & s < s' \end{cases} \\ &\equiv e^{A_{ss'a}}. \end{aligned} \tag{66}$$

Sharding and taking only the diagonal $c' = c$ terms, the above turns into (no sum over the repeated c-index):

$$\begin{aligned}\mathcal{A}_{ccll'a} &= \begin{cases} e^{A_{cla}} \times ... \times e^{A_{c(l'+1)a}} = \exp\left(\sum_{l''=l'+1}^{l} A_{cl''a}\right) & l \geq l' \\ 0 & l < l' \end{cases} \\ &\equiv e^{A_{ccll'a}}. \end{aligned} \tag{67}$$

The argument $A_{ccll'a}$ can be constructed in various ways[52]:

$$\begin{aligned} A_{ccll'a} &= \texttt{segsum}_{ll'}(A_{cla}) + M_{ll'} \\ &\equiv \texttt{cumsum}_l A_{cla} - \texttt{cumsum}_{l'} A_{cl'a} + M_{ll'} \\ &= \texttt{cumsum}_l(A_{cla}Z_{ll'}) + I(ll') \\ Z_{ll'} &\equiv \begin{cases} 0 & l \leq l' \\ 1 & l > l'' \end{cases} \\ I_{ll'} &\equiv \begin{cases} -\infty & l < l' \\ 0 & l \geq l'' \end{cases} \end{aligned} \tag{68}$$

[51] The $c' > c$ cases are trivial as $\mathcal{A}_{cc'll'a}$ vanishes.

[52] $\texttt{cumsum}_s X_s \equiv \sum_{s'=0}^{s} X_{s'}$ and segsum stands for "segment sum": $\texttt{segsum}_{ss'}(X_s) = \sum_{s''=s'+1}^{s} X_{s''}$.

where the final form with the additional mask $Z_{ll'}$ is better behaved numerically, as it does not rely on cancellations between sums. Careful attention should be paid to the inequality symbols in the masks. The remainder of these diagonal computations is straightforward: just compute $y_{clah} = C_{clgn} e^{A_{cc'll'a}} \mathcal{B}_{cl'gah}$, which takes $\mathcal{O}(LSDGN)$ time, a factor of the chunk size L larger that the optimal compute time the chunk index.

2.4.4.2. The $c > c'$ Cases

Now we compute the remaining off-diagonal terms. Compute one (c, c') chunk at a time, i.e. we compute

$$y_{clah} = C_{clgn} \mathcal{A}_{cc'll'a} \mathcal{B}_{c'l'gah} \ , \tag{69}$$

by iterating over the c' sum, similarly to flash attention. As written, this would be $\mathcal{O}(S^2 DGN)$ computation which is an entire factor of S larger than optimal due to the sums over c', l', but we'll see that it's possible to improve upon this naive counting.

$\mathcal{A}_{cc'll'a}$ is made up of $\sim e^{A_{sa}}$ factors which each serve to propagate $\mathcal{B}_{(s-1)gah}$ forward one step in time. Specifically, $\mathcal{A}_{cc'll'a}$ contains all the factors needed to get from the times specified by the (c', l') indices up to the (c, l) indices:

$$\mathcal{A}_{cc'll'a} = \exp\left(\sum_{s=c'L+l'+1}^{cL+l} A_{sa} \right), \quad \text{for } c > c'. \tag{70}$$

We will break the above into three factors[53]:

1. A right-factor which propagates the $\mathcal{B}_{c'l'gah}$ from their disparate times l' within a chunk all up to the final time in a chunk: $l = L - 1$.
2. A center-factor which propagates the previous element together for a period.
3. A left-factor which finally propagates these elements up to their final time slices (c, l) specified by C_{clgn}.

The specific form of the factors comes from the following (non-unique) decomposition of the preceding expression:

$$\begin{aligned} \mathcal{A}_{cc'll'a}|_{c>c'} &= \exp\left(\sum_{s=c'L+l'+1}^{cL+l} A_{sa} \right) \\ &= \exp\left(\sum_{l''=0}^{l} A_{cl''a} + \sum_{c''=c'+1}^{c-1} \sum_{l=0}^{L-1} A_{c''la} + \sum_{l''=l'+1}^{L-1} A_{c'l''a} \right) \end{aligned} \tag{71}$$

[53] In the nomenclature of [23], whose authors also refer to these as the B, A, and C blocks, respectively (though we actually differ slightly in detail from what the paper and `mamba-ssm` do).

$$= \exp\left(\sum_{l''=0}^{l} A_{cl''a}\right) \exp\left(\sum_{c''=c'+1}^{c-1} \sum_{l=0}^{L-1} A_{c''la}\right) \exp\left(\sum_{l''=l'+1}^{L-1} A_{c'l''a}\right)$$
$$\equiv U_{cla} \boldsymbol{A}_{cc'a} T_{c'l'a}, \tag{71}$$

such that the contribution from the $c' < c$ terms reads

$$M_{cc'} C_{clgn} U_{cla} \boldsymbol{A}_{cc'a} T_{c'l'a} \mathcal{B}_{c'l'ganh} = M_{cc'} \boldsymbol{C}_{clgna} \boldsymbol{A}_{cc'a} \boldsymbol{B}_{c'ganh},$$
$$M_{cc'} = \begin{cases} 1 & c > c' \\ 0 & c \leq c' \end{cases}. \tag{72}$$

which (I believe) are the C, A, and B blocks from [23]. These factors can be conveniently, and succinctly, vectorized as in[54]:

$$\begin{aligned} T_{c'l'a} &= \exp(A_{c'a} - \texttt{cumsum}_{l'} A_{c'l'a}) \quad \text{where} \quad A_{ca} \equiv \texttt{sum}_l A_{cla} = (\texttt{cumsum}_l A_{cla})|_{l=-1} \\ \boldsymbol{A}_{cc'a} &= \exp(\texttt{segsum}_{cc'} A_{ca} - A_{ca}) \\ U_{cla} &= \exp(\texttt{cumsum}_l(A_{cla})). \end{aligned} \tag{73}$$

Several crucial computational points:

- $\mathcal{A}_{cc'll'a} = U_{cla} \boldsymbol{A}_{cc'e} T_{c'l'a}$ is factorizable (low-rank)

(71).

- The middle factor $\boldsymbol{A}_{cc'a}$ can be factorized further:

$$\begin{aligned} \boldsymbol{A}_{cc'a} &= \exp(\texttt{segsum}_{cc'} A_{ca} - A_{ca}) \\ &= \exp(\texttt{cumsum}_c A_{ca} - A_{ca}) \times \exp(-\texttt{cumsum}_{c'} A_{c'a}) \\ &= L_{ca} R_{c'a}. \end{aligned} \tag{74}$$

  which encodes the special property that SSM attention is optimally $\mathcal{O}(S)$, rather than $\mathcal{O}(S^2)$.

The entire c' sum also involves a causal mask,

$$M_{cc'} \boldsymbol{A}_{cc'a} \boldsymbol{B}_{c'ganh}, \tag{75}$$

which makes the whole combination $M_{cc'} \boldsymbol{A}_{cc'a}$ non low-rank[55]. However, we have the manipulations:

$$M_{cc'} \boldsymbol{A}_{cc'a} \boldsymbol{B}_{c'ganh} = M_{cc'} L_{ca} R_{c'a} \boldsymbol{B}_{c'ganh} \tag{76}$$

[54]These can also be written in a form similar to Equation 68 where we use masks instead of relying in numerically unstable cancellations. $T_{c'l'a} = \exp(\texttt{sum}_l Z_{ll'} A_{c'la})$, $U_{cla} = \exp(\texttt{sum}_{l'}(1 - Z_{l'l}) A_{c'l'a})$ with $Z_{ll'}$ the mask in Equation 68;.

[55]Were it not for the mask, we could simply compute the sum by breaking the low-rank matrix into its natural factors as $\boldsymbol{A}_{cc'a} T_{c'l'a} = L_{ca} R_{c'a} T_{c'l'a}$, compute the c' sum in $\mathcal{O}(S)$ and multiply the scalar sum by L_c in $\mathcal{O}(S)$ again.

$$
\begin{aligned}
&= L_{ca} \sum_{c'=0}^{c-1} R_{c'a} \boldsymbol{B}_{c'ganh} \\
&= L_{ca} \times \left(\texttt{cumsum}_c \left(R_{ca} \boldsymbol{B}_{cganh} \right) - R_{ca} \boldsymbol{B}_{cganh} \right) \\
&= L_{ca} \times Z_{cl'a} \qquad (76)
\end{aligned}
$$

Thus, the sums over c, c' in (77) can be performed in $\mathcal{O}(S)$ time, rather than the naive $\mathcal{O}(S^2)$ time.

Therefore, one way we could compute (72) in stages as in:
1. $y_{c'ganh} \leftarrow T_{c'l'a} \mathcal{B}_{c'l'ganh}$ in $\mathcal{O}(SDGN)$
2. Use (76) to compute $y_{ganh} \leftarrow M_{cc'} \boldsymbol{A}_{cc'} y_{c'ganh}$ in $\mathcal{O}(\frac{SDGN}{L})$
3. $y_{clganh} \leftarrow C_{clgn} U_{cla} z_{cganh}$ in $\mathcal{O}(SDGN)$

The parenthesized terms can all be computed in $\mathcal{O}(S)$ and the final elementwise product also takes $\mathcal{O}(S)$ time, as claimed[56]. `mamba-ssm` appears to use something like this strategy.

2.4.4.3. Chunked Solution

The full solution decomposed in this way is then:

$$
z_{clah} = C_{clgn} e^{A_{ccll'a}} \mathcal{B}_{cl'ganh} + M_{cc'} C_{clgn} U_{cla} \boldsymbol{A}_{cc'a} T_{c'l'a} \mathcal{B}_{cl'ganh} \qquad (77)
$$

2.4.4.4. The Non-Chunked Solution

For completeness, we can also write out the full, explicitly $\mathcal{O}(S)$ solution to (64) computed without chunking and written in vectorized notation. We have:

$$
\begin{aligned}
z_{sah} &= C_{sgn} \sum_{s'=0}^{s} \exp \left(\sum_{s=s'+1}^{s} A_{s'a} \right) \mathcal{B}_{s'ganh} \\
&= C_{sgn} \sum_{s'=0}^{s} \exp(\texttt{cumsum}_s A_{sa} - \texttt{cumsum}_{s'} A_{s'a}) \mathcal{B}_{s'ganh} \\
&= C_{sgn} \exp(\texttt{cumsum}_s A_{sa}) \times \texttt{cumsum}_s \left(\exp(-\texttt{cumsum}_s A_{sa}) \mathcal{B}_{sganh} \right) \qquad (78)
\end{aligned}
$$

Assuming an efficient $\mathcal{O}(S)$ `cumsum` implementation (up to constant factors), the above realizes the optimal asymptotic $\mathcal{O}(SDGN)$ time (and memory) efficiency of

2.4.4.5. Notes On the `mamba-ssm` Implementation

The `mamba-ssm` implementation uses other names for quantities above.

Focusing on the off-diagonal part of the scan, the first step computes:

$$
\texttt{states}_{cganh} = \exp \left(\sum_{l''=l'+1}^{L-1} A_{cl''a} \right) \mathcal{B}_{cl'ganh}, \qquad (79)
$$

[56] Unfortunately, this is also a very numerically unstable way to perform the computation, relying on cancellations in products of exponentials.

which are the "states" at upper time limit of each chunk. The `_chunk_state_fwd` kernel handles this.

The second step takes $\texttt{states}_{cganh}$ and an $\texttt{initial_states}_{ganh}$ tensor, which defaults to zeros, and computes the tuple of `final_state` and updates `states` defined as in[57] (making sums explicit)

$$
\begin{aligned}
A_{ca} &\equiv \sum_{l=0}^{L-1} A_{c''la} \\
\texttt{out_states}_{cganh} &= \sum_{c'=0}^{c-1} \exp\left(\sum_{c''=c'+1}^{c} A_{c''a}\right) \texttt{states}_{c'ganh} \\
&\quad + \exp\left(\sum_{c'=0}^{c-1} A_{c'a}\right) \texttt{initial_states}_{ganh}, \quad \left(c < \frac{S}{L} - 1\right) \\
\texttt{final_states}_{ganh} &= \sum_{c'=0}^{\frac{S}{L}-1} \exp\left(\sum_{c''=c'+1}^{\frac{S}{L}-1} A_{c''a}\right) \texttt{states}_{c'ganh} + \exp\left(\sum_{c'=0}^{\frac{S}{L}-1} A_{c'a}\right) \texttt{initial_states}_{ganh} \\
\texttt{states}_{cganh} &= \texttt{Concat}_c\left(\texttt{out_states}_{[:-1]ganh}, \texttt{states}_{0ganh}\right)
\end{aligned} \tag{80}
$$

via a loop over chunks, with `initial_states` setting initial conditions in the loop[58]. This is the role of the `_state_passing_fwd` kernel. The updated `states` is used in the remainder of the off-diagonal computation, as well as the diagonal part. `final_state` can optionally be returned, but not used again (it can be used for inference).

The final step uses the updated `states` and computes

$$
\texttt{out}_{clah} = C_{clgn} \exp\left(\sum_{l'=0}^{l} A_{cl'a}\right) \texttt{states}_{cganh} \tag{81}
$$

This is performed by `_chunk_scan_fwd`, which also performs the diagonal computation.

Inference and `initial_states`: to motivate the introduction of `initial_states`, note that we can break up the `out_states` computation with no `initial_states` as in

$$
\texttt{out_states}_{cganh} = \sum_{c'=0}^{c} \exp\left(\sum_{c''=c'+1}^{c} A_{c''a}\right) \texttt{states}_{c'ganh} \tag{82}
$$

[57] I believe. Note the upper limit of the c'' sum.

[58] The indexing can be confusing and off-by-one errors are easy to make. The extreme components take on values $\texttt{out_states}_{0\ldots} = \texttt{initial_states}_{\ldots}$, $\texttt{out_states}_{c\ldots} = \texttt{states}_{c\ldots} + e^{A_{ca}} \texttt{states}_{c-1\ldots}$.

$$
\begin{aligned}
&= \sum_{c'=\texttt{stop}+1}^{c} \exp\left(\sum_{c''=c'+1}^{c} A_{c''a}\right) \texttt{states}_{c'ganh} \\
&\quad + \exp\left(\sum_{c''=\texttt{stop}+1}^{c} A_{c''a}\right) \sum_{c'=0}^{\texttt{stop}} \exp\left(\sum_{c''=c'+1}^{\texttt{stop}} A_{c''a}\right) \texttt{states}_{c'ganh} \\
&\equiv \sum_{c'=\texttt{stop}+1}^{c} \exp\left(\sum_{c''=c'+1}^{c} A_{c''a}\right) \texttt{later_states}_{c'ganh} \\
&\quad + \exp\left(\sum_{c''=\texttt{stop}+1}^{c} A_{c''a}\right) \texttt{initial_states}_{ganh} \qquad (82)
\end{aligned}
$$

That is, we can break down the computation into a first chunk which propagates the state up to some arbitrary chunk boundary `stop`, and then use this info to propagate further from `stop` up to c. Clearly, this is useful for caching and/or piecewise computations.

Importantly, additional state for the depthwise convolution must also be passed to perform this computation properly, as the convolution affects the values of `later_states` above.

2.4.4.6. Context Parallel Mamba2 Scan

Sharding the mamba2 scan over devices. Shard the sequence dimension as $s \to (rt)$ with $r \in \{0, ..., R-1\}$ indexing the rank and $t \in \left\{0, ..., \frac{S}{R} - 1\right\}$. We want to use the mamba2 chunking strategy (and kernels) locally on each node, so we further shard $t \to (cl)$ with $l \in \{0, ..., L-1\}$ and $c \in \left\{0, ..., \frac{S}{LR} - 1\right\}$. Organize the computation of the outputs z_{rclah} similarly to the full computation, into diagonal terms which can be completely computed locally to a rank and off-diagonal terms which require shards across ranks.

The calculations are the essentially the same as before. The diagonal computation is embarassingly-parallel across ranks

$$
z^{\text{diag}}_{rclah} = C_{rclgn} e^{\sum_{l''=l'+1}^{l} A_{rcl'a}} \mathcal{B}_{rcl'ganh}. \qquad (83)
$$

The first step of the off-diagonal computation is to create the states, which is also embarassingly-parallel across ranks:

$$
\texttt{states}_{rcganh} = \exp\left(\sum_{l'=l+1}^{L-1} A_{rcl'a}\right) \mathcal{B}_{rclganh} \qquad (84)
$$

Properly updating the states in the middle step requires computing

$$
\texttt{out_states}_{rcganh} \leftarrow \exp\left(\sum_{c''=c'+1}^{c} A_{rc''a}\right) \texttt{states}_{rc'ganh} \qquad (85)
$$

$$
\begin{aligned}
&+\exp\left(\sum_{c'=0}^{c} A_{rc'a}\right)\sum_{r'=0}^{r-1}\exp\left(\sum_{r''=r'}^{r-1}\sum_{c''=c'+1}^{\frac{S}{LR}-1} A_{r''c''a}\right)\texttt{states}_{r'c'ganh}\\
&=\exp\left(\sum_{c''=c'+1}^{c} A_{rc''a}\right)\texttt{states}_{rc'ganh}\\
&\quad+\exp\left(\sum_{c'=0}^{c} A_{rc'a}\right)\sum_{r'=0}^{r-1}\exp\left(\sum_{r''=r'+1}^{r-1} A_{r''a}\right)\texttt{final_states}_{r'ganh} \qquad (85)
\end{aligned}
$$

and the sums over different ranks necessarily involve communication.

One way to compute the above would be serial:
1. Rank 0 computes its $\texttt{states}_{0ganh}$ to rank 1.
2. Rank 1 uses $\texttt{states}_{0ganh}$ as the `initial_states` for its computation of $\texttt{states}_{1ganh}$
3. Rank 1 passes $\texttt{states}_{1ganh}$ to rank 2.
4. ...

This idles GPUs, but the state passing step also has a computational complexity which is smaller than those of other steps by a factor of L, the chunk size, and so the serial nature is likely acceptable for `world_size` $\lesssim L$.

A different strategy is the following:
1. Every rank computes (GG: finish)

2.4.5. Aren't These Just RNNs?

Yes, but very special ones with the important computational difference that the recursion relations are *linear* in the hidden state h. This crucial difference improves the ability to parallelize the computations. Compare to what typical RNN recursion relations would look like:

$$
\begin{aligned}
h_{bs} &= \varphi\big(A_{bb'}h_{b'(s-1)} + B_{ba}x_{as}\big)\\
y_{cs} &= \varphi(C_{cb}h_{bs} + D_{ca}x_{as}). \qquad (86)
\end{aligned}
$$

for some non-linearity φ. The recursion relations would solve to an expression with nested φ factors which would make the computation of h_{bs} non-associative. But in the linear $\varphi(x) = x$ limit, the operations are *associative* which makes them *parallelizable*, via known scan algorithms [24].

3. Pretraining

3.1. Memory

In this section we summarize the train-time memory costs of Transformers under various training strategies[59].

[59] A nice related blog post is here.

The memory cost is much more than simply the cost of the model parameters. Significant factors include:

- Optimizer states, like those of
- Mixed precision training costs, due to keeping multiple model copies.
- Gradients
- Activation memory[60] , needed for backpropagation.

Because the activation counting is a little more involved, it is in its own section.

Essentials Memory costs count the elements of all tensors in some fashion, both from model parameters and intermediate representations. The gradient and optimizer state costs scale with the former quantity: $\mathcal{O}(N_{\text{params}})\mathcal{O}(LD^2)$, only counting the dominant contributions from weight matrices. Activation memory scales with the latter, which for a -shaped input gives $\mathcal{O}(BDLS)$ contributions from tensors which preserve the input shape, as well as $\mathcal{O}(ABLS^2)$ factors from attention matrices.

3.1.1. No Sharding

Start with the simplest case where there is no sharding of the model states. Handling the different parallelism strategies later will be relatively straightforward, as it involves inserting just a few factors here and there.

3.1.1.1. Parameters, Gradients, Optimizer States, and Mixed Precision

Memory from the bare parameter cost, gradients, and optimizer states are fixed costs independent of batch size and sequence-length (unlike activation memory), so we discuss them all together here. The parameter and optimizer costs are also sensitive to whether or not mixed-precision is used, hence we also address that topic, briefly. We will assume the use of[61] throughout, for simplicity and concreteness. It will some times be useful below to let p to denote the precision in bytes that any given element is stored in, so corresponds to $p = 4$, for instance. Ultimately, we primarily consider vanilla training in $p = 4$ precision and / ($p = 4$/ $p = 2$) mixed-precision, other, increasingly popular variants to exist, so we keep the precision variable where we can.

Without mixed precision, the total cost of the ($p = 4$ bytes) model and optimizer states in bytes is then

$$M_{\text{model}} = 4N_{\text{params}}, \quad M_{\text{optim}} = 8N_{\text{params}} \quad (\text{no} \quad \text{mixed} \quad \text{precision},)p = 4) \tag{87}$$

[60] Activations refers to any intermediate value which needs to be cached in order to compute backpropagation. We will be conservative and assume that the inputs of all operations need to be stored, though in practice gradient checkpointing and recomputation allow one to trade caching for redundant compute. In particular, flash attention [16] makes use of this strategy.

[61] Which stores two different running averages per-model parameter.

where, from the previous section, the pure parameter-count of the decoder-only Transformers architecture is

$$N_{\text{params}} \approx (4+2E)LD^2 \times \left(1 + \mathcal{O}\left(\frac{V}{DL}\right) + \mathcal{O}\left(\frac{1}{D}\right)\right). \tag{88}$$

where the first term comes from the weight matrices[62], the first omitted subleading correction term is the embedding matrix, and the last comes from biases, instances, and other negligible factors. The optimizer states cost double the model itself.

The situation is more complicated when mixed-precision is used [25]. The pertinent components of mixed-precision[63]:

- A half-precision ($p = 2$ bytes) copy of the model is used to perform the forwards and backwards passes
- A second, "master copy" of the model is also kept with weights in full $p = 4$ precision
- The internal states are kept in full-precision

Confusingly, the master copy weights are usually accounted for as part of the optimizer state, in which case the above is altered to

$$M_{\text{model}} = 2N_{\text{params}}, \quad M_{\text{optim}} = 12N_{\text{params}} \quad (\text{mixed precision}). \tag{89}$$

The optimizer state is now six times the cost of the actual model used to process data and the costs of (89) are more than those of (87);. However, as we will see, the reduced cost of activation memory can offset these increased costs, and we get the added benefit of increased speed due to specialized hardware. The above also demonstrates why training is so much more expensive than inference.

3.1.1.2. Gradients

Gradients are pretty simple and always cost the same regardless of whether or not mixed-precision is used:

$$M_{\text{grad}} = 4N_{\text{params}}. \tag{90}$$

[62] So, in the usual $E = 4$ case, the layers are twice as costly as the layers.

[63] A note on the implementation of mixed-precision in : usually mixed-precision occurs by wrapping the forward pass in a context manager, . The default behavior is to then create copies of some tensors in lower-precision and do the forward pass with those. For instance, this is done with matrix-multiplies whose arguments and outputs will be in , but for sums the inputs and outputs will all be , for vanilla mixed-precision usage. Consequently, any such versions of tensor will often persist effectively as contributors to activation memory, since the backwards pass will need those same tensors. This can be verified by inspecting the saved tensors: if is the output of a matrix-multiply in such an autocast context, will be a copy of the weights used to perform the matrix-multiply. In effect, the cost of the model weights which are used for the actual forward pass are only materialized within the lifetime of the context manager.

In mixed precision, even though the gradients are initially computed in $p = 2$, they have to be converted to $p = 4$ to be applied to the master weights of the same precision.

3.1.1.3. Activations

Activations will require a more extended analysis [5]. Unlike the above results, the activation memory will depend on both the batch size and input sequence length, B and S, scaling linearly with both.

3.1.1.3.1. Attention Activations

We will count the number of input elements which need to be cached. Our -shaped inputs to the attention layer with BDS elements are first converted to $3BDS$ total query, key, value elements, and the query-key dot products produce ABS^2 more, which are softmaxed into ABS^2 normalized scores. The re-weighted inputs to the final linear layer also have BDS elements, bringing the running sum to $BS(5D + 2AS)$

Finally, there are also the dropout layers applied to the normalized attention scores and the final output whose masks must be cached in order to backpropagate. In torch, the mask is a tensor, but surprisingly these use one *byte* of memory per element, rather than one bit[64]. Given this, the total memory cost from activations is

$$M_{\text{act}}^{\text{Attention}} = BLS((5p + 1)D + (2p + 1)AS). \tag{91}$$

3.1.1.3.2. MLP Activations

First we pass the -shaped inputs into the first MLP layer. These turn into the inputs of the non-linearity, whose same-shaped outputs are then passed into the last layer, summing to $(2E + 1)BDS$ total elements thus far. Adding in the dropout mask, the total memory requirement across all layers is:

$$M_{\text{act}}^{\texttt{MLP}} = (2Ep + p + 1)BDLS. \tag{92}$$

3.1.1.3.3. LayerNorm, Residual Connections, and Other Contributions

Then the last remaining components. The instances each have BDS inputs and there are two per transformer block, so $M_{\text{act}}^{\texttt{LayerNorm}} = 2pBDLS$, and there is an additional instance at the end of the architecture[65]. There are two residual connections per block, but their inputs do not require caching (since their derivatives are independent of inputs). Then, there are additional contributions from pushing the last layer's outputs through the language-model head and computing the loss function, but these do not scale with L and are ultimately $\mathcal{O}\left(\frac{V}{DL}\right)$ suppressed, so we neglect them.

3.1.1.3.4. Total Activation Memory

Summing up the contributions above, the total activation memory cost per-layer is

[64] As you can verify via

[65] Following [5] we will neglect this in the below sum, an $\mathcal{O}\left(\frac{1}{L}\right)$ error

$$M_{\text{act}}^{\text{total}} \approx 2BDLS\left(p(E+4)+1+\mathcal{O}\left(\frac{V}{DL}\right)\right)+ABLS^2(2p+1). \tag{93}$$

Evaluating in common limits, we have:

$$\begin{aligned} M_{\text{act}}^{\text{total}} \mid_{E=4,p=4} &= BLS(66D+15AS) \\ M_{\text{act}}^{\text{total}} \mid_{E=4,p=2} &= BLS(34D+5AS) \end{aligned} \tag{94}$$

3.1.1.4. When does mixed-precision reduce memory?

(Answer: usually.) We saw in Section 3.1.1.1 that mixed precision *increases* the fixed costs of non-activation memory, but from the above we also see that it also *reduces* the activation memory and the saving increase with larger batch sizes and sequence lengths. It is straightforward to find where the tipping point is. Specializing to the case $E = 4$, vanilla mixed-precision case with no parallelism[66], the minimum batch size which leads to memory savings is

$$B_{\text{min}} = \frac{6D^2}{8DS+AS^2}. \tag{95}$$

Plugging in numbers for the typical $\mathcal{O}(40\text{ GiB})$ model in the Summer of 2023 gives $B_{\text{min}}\mathcal{O}(1)$, so mixed-precision is indeed an overall savings at such typical scales.

Side Note: Optimizations

The above analysis is conservative and accounts for more tensors than are actually saved in practice.

For instance, both the input and outputs of all non-linearities were counted, but there are many activations whose derivatives can be reconstructed from its outputs alone: $\varphi'(z) = F(\varphi(z))$ for some F. Examples:

- `ReLU`: since $\varphi(z) = z\theta(z)$, then (defining the derivative at zero to be zero) $\varphi'(z) = \theta(z) = \theta(\varphi(z))$. Correspondingly, torch only uses the outputs to compute the derivative (there is no self arg in the line).
- tanh: since $\tanh'(z) = 1 - \tanh(z)^2$.

Other cases do not have this nice property, in which case both the inputs and outputs need to be stored:

- `GeLU` [26]: $\varphi(z) = z\Phi(z)$ here and the derivative $\varphi'(z) = \Phi(z) + \frac{ze^{-z^2/2}}{\sqrt{2\pi}}$, both the inputs and outputs must be used in the backwards pass..

  The explicit CUDA kernel is here.

[66] With both tensor- and sequence-parallelism, the parallelism degree T actually drops out in the comparison (since both form of memory are decrease by $1/T$, so this restriction can be lifted.

If the inputs in each of these cases are not needed for any other part of the backwards pass, they are garbage collected in soon after creation.

`Softmax` is another instance where this occurs, since

$$\partial_i \texttt{Softmax}(x_j) = \delta_{ij} \texttt{Softmax}(x_j) - \texttt{Softmax}(x_i) \texttt{Softmax}(x_j) \tag{96}$$

Because of this, the actual amount of activation memory due to the attention layer after the forwards pass is (91) with $2p \to p$ in the $\mathcal{O}(S^2)$ term, though the above expression better reflects the necessary peak memory.

3.2. Training FLOPs

The total number of floating point operations (FLOPs)[67] needed to process a given batch of data is effectively determined by the number of matrix multiplies needed.

Recall that a dot-product of the form $v \cdot M$ with $v \in \mathbb{R}^m$ and $M \in \mathbb{R}^{m,n}$ requires $(2m-1) \times n \approx 2mn$ FLOPs . For large language models, $m, n \mathcal{O}(10^3)$, meaning that even expensive element-wise operations like acting on the same vector v pale in comparison by FLOPs count[68]. It is then a straightforward exercise in counting to estimate the FLOPs for a given architecture. The input tensor is of shape throughout.

Essentials The number of FLOPs to push a batch of B of sequence-length S examples through the forwards-pass of a decoder-only transformer is approximately $2BSN_{\text{params}}$ where the number of parameters accounts for any reductions due to tensor- and sequence-parallelism[69]. The backwards-pass costs about twice as much as the forwards-pass. This is true as long as $S \lesssim D$).

[67] The notation surrounding floating-point operations is very confusing because another quantity of interest is the number of floating-point operations a given implementation can use *per-second*. Some times, people use FLOPS or FLOP/s to indicate the rate, rather than the gross-count which has the lower case "s", FLOPs, but there's little consistency in general. We will use FLOPs and FLOP/s.

[68] Since their FLOPs counts only scales as $\mathcal{O}(n)$ where the omitted constant may be relatively large, but still negligible when all dimensions are big.

[69] A quick argument: a computation of the form $T_{a_0 \dots a_n j} = V_{a_0 \dots a_A i} M_{ij}$ requires $2A_0 \dots A_n IJ$ FLOPs where the capital letters represent the size of their similarly-index dimensions. Thus, the FLOPs essentially count the size of the matrix M (that is, IJ), up to a factor of 2 times all of the dimensions in V which weren't summed over. Therefore, passing a -shaped tensor through the Transformer architecture would give $\sim 2BS \times$(sum of sizes of all weight-matrices) FLOPs, and that this last factor is also approximately the number of parameters in the model (since that count is dominated by weights). Thus, FLOPs $\approx 2BSN_{\text{params}}$. This is the correct as long as the self-attention FLOPs with $\mathcal{O}(S^2)$-dependence which we didn't account for here are actually negligible (true for $S \lesssim 10D$).

3.2.1. No Recomputation

Start with the case where there is no recomputation activations. These are the **model FLOPs** of [5], as compared to the **hardware FLOPs** which account for gradient checkpointing.

3.2.1.1. Multi-Head Attention: Forwards

The FLOPs costs:

- Generating the query, key, and value vectors: $6BSD^2$ for vanilla MHA, or $(2 + \texttt{gqa}) \times BSD^2 \approx 2BSD^2$ if using GQA (Sec. Section 1.2.4) with $\texttt{gqa} = \frac{N_{\frac{k}{v}}}{N_q}$.
- Attention scores: $2BDS^2$
- Re-weighting values: $2BDS^2$
- Final projection: $2BSD^2$

3.2.1.2. Mamba2: Forwards

The forward-pass flops for a Mamba2 layers are as follows:

1. Matmul for creating various intermediates: $4ESD^2 \times \left(1 + \mathcal{O}\left(\frac{\texttt{n_groups}\ N}{D}, \frac{H}{D}\right)\right)$
2. Convolutions: $\mathcal{O}(SDC)$
3. Scan: $\mathcal{O}(SD)$
4. Out projections: $2SED^2$

Collecting the dominant $\mathcal{O}(SD^2)$ terms and assuming sequence lengths are short enough such that the $\mathcal{O}(S^2)$ MHA terms are negligible, we see that Mamba2 is $\frac{6E}{4+\texttt{gqa}}$ times more expensive in terms of FLOPS. For the typical $E = 2, \texttt{gqa} \ll 1$ values, this is a $\approx 3\times$ difference.

3.2.1.3. MLP: Forwards

Passing a through the layer, the FLOPs due to the first and second matrix-multiplies are equal, with total matrix-multiply FLOPs $4BSED^2$.

3.2.1.4. Backwards Pass: Approximate

The usual rule of thumb is to estimate the backwards pass as costing twice the flops as the forwards pass. This estimate comes from just counting the number of $\mathcal{O}(n^2)$ matrix-multiply-like operations and seeing that for every one matrix multiplication that was needed in the forward pass, we have roughly twice as many similar operations in the backwards pass.

The argument: consider a typical sub-computation in a neural network which is of the form $z' = \varphi(W \cdot z)$ where z', a are intermediate representations z, z', φ is some non-linearity, and where the matrix multiply inside the activation function dominates the forwards-pass FLOPs count, as above. Then, in the backwards pass for this sub-computation, imagine we are handed the upstream derivative $\partial_{z'}\mathcal{L}$. In order to complete backpropagation, we need both to compute $\partial_W \mathcal{L}$ to update W and also $\partial_z \mathcal{L}$ to continue backpropagation to the next layer down. Each of these operations will cost about as many FLOPs as the forwards-pass, hence the estimated factor of two (but, as we will see, this is a very rough estimate).

Being more precise, let z be -shaped and let W be -shaped such that it acts on the last index of z, making z' -shaped. Denoting $D = \prod_i D_i$ be the number of elements along the D_i directions for brevity, the forward-FLOPs cost of the sub-computation is therefore $2DIJ$.

Adding indices, the two derivatives we need are

$$\begin{aligned}\frac{\partial \mathcal{L}}{\partial W_{ij}} &= \frac{\partial \mathcal{L}}{\partial z'_{d_0 \dots d_n i}} \varphi'\Big((W \cdot z)_{d_0 \dots d_n i}\Big) z_{d_0 \dots d_n j} \\ \frac{\partial \mathcal{L}}{\partial z_{d_0 \dots d_n j}} &= \frac{\partial \mathcal{L}}{\partial z'_{d_0 \dots d_n i}} \varphi'\Big((W \cdot z)_{d_0 \dots d_n i}\Big) W_{ij},\end{aligned} \tag{97}$$

which have shapes and , respectively. On the right side, z and $W \cdot z$ are cached and the element-wise computation of $\varphi'(W \cdot z)$ has negligible FLOPs count, as discussed above: its contribution is $\mathcal{O}\left(\frac{1}{I}\right)$ suppressed relative to the matrix-multiplies. The FLOPs count is instead dominated by the broadcast-multiplies, sums, and matrix-products.

The two derivatives in (97) each have the same first two factors in common, and it takes DI FLOPs to multiply out these two -shaped tensors into another result with the same shape. This contribution is again $\mathcal{O}\left(\frac{1}{I}\right)$ suppressed and hence negligible. Multiplying this factor with either $z_{d_0 \dots d_n i}$ or W_{ij} and summing over the appropriate indices requires $2DIJ$ FLOPs for either operation, bringing the total FLOPs to $4DIJ$, which is double the FLOPs for this same sub-computation in the forward-direction, hence the rough rule of thumb[70].

3.2.1.5. Backwards Pass: More Precise

TODO

3.2.1.6. Total Model FLOPs

The grand sum is then[71]:

$$C^{\text{model}} \approx 12BDLS(S + (2 + E)D). \tag{98}$$

We can also phrase the FLOPs in terms of the number of parameters (123) as

$$C^{\text{model}} \,|_{T=1} = 6BSN_{\text{params}} \times \left(1 + \mathcal{O}\left(\frac{S}{D}\right)\right) \tag{99}$$

where we took the $T = 1, D \gg S$ limit for simplicity and we note that BS is the number of total tokens in the processed batches.

[70] Note also that the very first layer does not need to perform the second term in Equation 97;, since we do not need to backpropagate to the inputs, so the total backwards flops is more precisely $4DIJ(L-1) + 2DIJ$.

[71] With a large vocabulary, the cost of the final language model head matrix multiply can also be significant, but we have omitted its L-independent, $2BDSV$ contribution here.

3.2.2. Training Time

Training is generally compute bound (see Appendix D.4) and based on the results of Section 3.2 the quickest one could possibly push a batch of data through the model is

$$t_{\text{min}} = \frac{C^{\text{model}}}{\lambda_{\text{FLOP/s}}}. \tag{100}$$

Expanding to the entire training run, then with perfect utilization training will take a time

$$t_{\text{total}} \approx \frac{6 N_{\text{params}} N_{\text{tokens}}}{\lambda_{\text{FLOP/s}}}. \tag{101}$$

Adjust $\lambda_{\text{FLOP/s}}$ to the actual achievable FLOP/s in your setup to get a realistic estimate.

How many tokens should a model of size N_{params}? Scaling laws (Section 3.2.3) provide the best known answer, and the Summer 2023 best-guess is that we optimally have $N_{\text{tokens}} \approx 20 N_{\text{params}}$. So that the above is

$$t_{\text{total}} \approx \frac{120 N_{\text{params}}^2}{\lambda_{\text{FLOP/s}}}, \tag{102}$$

leading to quadratic growth in training time.

Note that the above is only correct if we are actually only spending C^{model} compute per iteration. This is not correct if we use gradient checkpointing and recomputation, in which case we alternatively spend true compute $C^{\text{hardware}} > C^{\text{model}}$, a distinction between **hardware FLOPs** and **model FLOPs**. Two corresponding efficiency measures are **model FLOPs utilization** (MFU) and **hardware FLOPs utilization** (HFU). If our iterations take actual time t_{iter}, then these are given by

$$\text{MFU} = \frac{t_{\text{iter}}}{t_{\text{min}}^{\text{model}}}, \quad \text{HFU} = \frac{t_{\text{iter}}}{t_{\text{min}}^{\text{hardware}}}, \tag{103}$$

where $t_{\text{min}}^{\text{model}}$ is (100) and $t_{\text{min}}^{\text{hardware}}$ is similar but using C^{hardware}.

3.2.3. Scaling Laws

Empirically-discovered scaling laws have driven the race towards larger and larger models.

> **Essentials** Decoder-only model performance improves predictably as a function of the model size, dataset size, and the total amount of compute. As of Summer 2023, there is little sign of hitting any kind of wall with respect to such scaling improvements.

The central parameters are:

- The number of non-embedding model parameters, as excising embedding params was found to generate cleaner scaling laws. Because our N_{params} has already been typically neglecting these

parameters, we will just use this symbol in scaling laws and keep the above understanding implicit.[72] [27].

- C: total compute, often in units like PFLOP/s-days $\sim 10^{20}$ FLOPs
- N_{tokens}: dataset-size in tokens
- $\mathcal{L}$: cross-entropy loss in nats

The specific form of any given scaling law should also be understood to apply to a pretty narrowly defined training procedure, in which choices like the optimizer, learning-rate scheduler, hyperparameter search budget, vocabulary size, tokenization, etc. are often rigidly set. Changing different components of the training procedure is liable to create different scaling laws (though nice laws of some form are still expected to exist).

3.2.3.1. Original Scaling Laws

The first scaling-laws were reported in [27]. Their simplest form relates the value of the cross-entropy loss *at convergence* (and in nats), $\mathcal{L}$, to the number of non-embedding parameter, dataset size in token, and the amount of compute, *in the limit* where only one of this factors is bottle-necking the model[73]. The laws (in our notation):

- $\mathcal{L}(N_{\text{params}}) \approx \left(\frac{N^{\star}_{\text{params}}}{N_{\text{params}}}\right)^{\alpha_N}$, with $\alpha_N \approx 0.076$ and $N^{\star}_{\text{params}} \approx 8.8 \times 10^{13}$
- $\mathcal{L}(N_{\text{tokens}}) \approx (N^{\alpha_T}_{\frac{\text{tokens})^{\star}}{N_{\text{tokens}}}}$, with $\alpha_T \approx 0.095$ and $N^{\star}_{\text{tokens}} \approx 5.4 \times 10^{13}$
- $\mathcal{L}(C) \approx \left(\frac{C^{\star}}{C}\right)^{\alpha_C}$, with $\alpha_C \approx 0.050$ and $C^{*} \approx 3.1 \times 10^{8}$ PFLOP/s-days, where the batch size was assumed to be chosen to be compute optimal per the criteria they outline

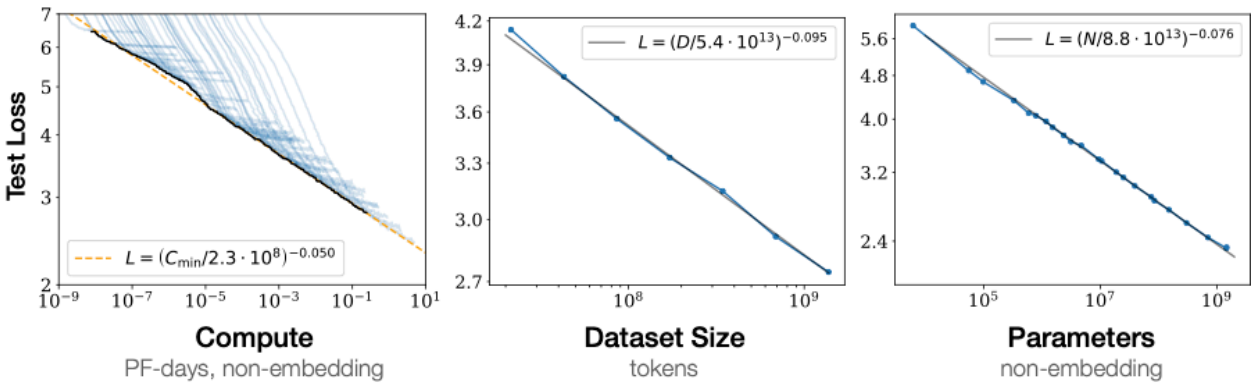


Figure 2: Original scaling laws from [27].

[72] Presumably, the scaling laws are cleaner with these neglected because these params do not contribute directly to FLOPs, unlike most other parameters.

[73] Unclear to me how you know when this is the case?

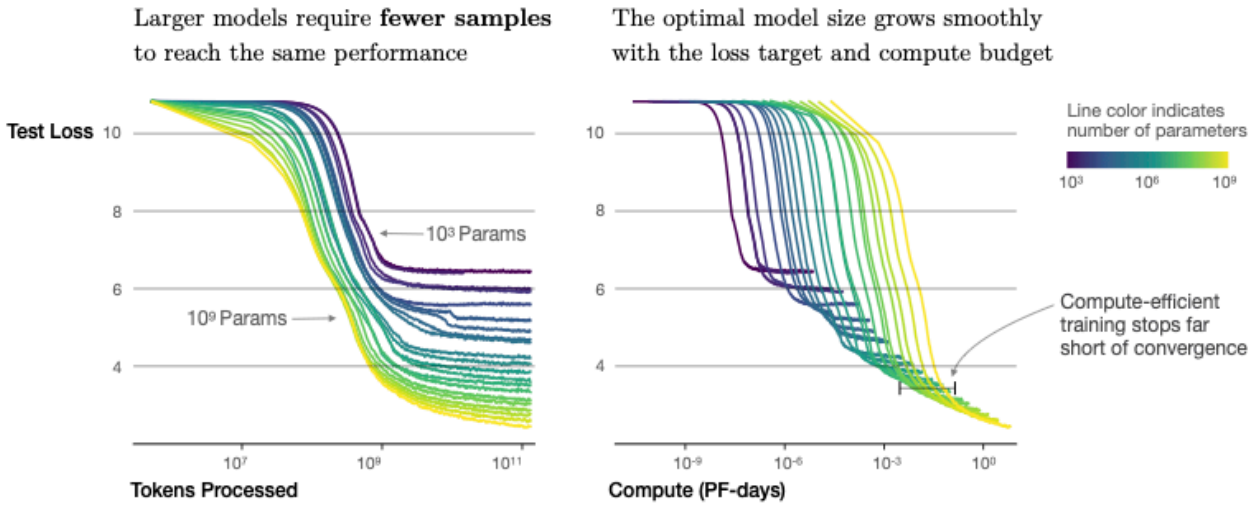


Figure 3: From [27]. Larger models are much more sample-efficient (faster).

3.2.3.2. Chinchilla Scaling Laws

As of Summer 2023, the Chinchilla scaling laws in [28] are the de facto best scaling laws for guiding training. The central difference between [28] and [27] is that in the former they adjust their cosine learning-rate schedule to reflect the amount of planned training, while in the latter they do not[74].

Several different analyses are performed which all give very similar results. The outputs are the optimal values of $N_{\text{params}}, N_{\text{tokens}}$ given a compute budget C.

- They fix various buckets of model sizes and train for varying lengths. In their resulting loss-vs-FLOPs plot, they determine the model size which led to the best loss at each given FLOPs value, thereby generating and optimal model size vs compute relation.
- They fix various buckets of FLOPs budget and train models of different sizes with that budget, finding the optimal model size in each case. A line can then be fit to the optimal settings across FLOPs budgets in both the parameter-compute and tokens-compute planes.
- They perform a parametric fit to the loss[75]:

$$\mathcal{L}\left(N_{\text{params}}, N_{\text{tokens}}\right) = E + \frac{A}{N_{\text{params}}^{\alpha}} + \frac{B}{N_{\text{tokens}}^{\beta}}, \tag{104}$$

  fit over a large range of parameter and token choices. The best-fit values are:

[74]The learning-rate schedule consist of a linear warm-up stage from a very small η up to the largest value $\eta_{\max}$, after which the cosine bit kicks in: $\eta(s) = \eta_{\min} + (\eta_{\max} - \eta_{\min}) \times \cos\left(\frac{\pi s}{2 s_{\max}}\right)$ with s the step number. In Fig. A1 of [28] they demonstrate that having the planned $s_{\max}$ duration of the scheduler be longer than the actual number of training steps is detrimental to training (they do not study the opposite regime), which is effectively what was done in [27]. Probably the more important general point is again that the precise form of these scaling laws depend on details of fairly arbitrary training procedure choices, such as the choice of learning-rate scheduler.

[75]In [28] they model the scaling of the test loss, while in [27] they use the training loss.

$$E = 1.69\,, \quad A = 406.4\,, \quad B = 410.7\,, \quad \alpha = 0.34\,, \quad \beta = 0.28\,. \tag{105}$$

Using $C \approx 6N_{(\text{params})}\, N_{\text{tokens}}$, the above can be minimized at fixed compute either for number of parameter or the size of the dataset.

In all cases, the findings are that at optimality $N_{\text{params}} N_{\text{tokens}} C^{.5}$: both the parameter and tokens budget should be scaled in equal measure.

4. Fine Tuning

4.1. Instruction Fine Tuning

Generally, instruction fine-tuning is a follow-on step after model pre-training[76]. The pre-training, pure next-token prediction task is altered to optimize an objective which now incorporates other data, typically information regarding human preferences[77].

4.1.1. Direct Preference Optimization

Direct Preference Optimization (DPO) [29] is a vast simplification of previous reinforcement-learning based methods (namely PPO-based ones [30]).

DPO aims to solve the RLHF optimization problem defined over a dataset $\mathcal{D}(x, y_l, y_w)$ corresponding to prefixes (x) and pairs of preferred and dispreferred completions[78] (y_l, y_w). The relevant components are:

1. A baseline language model: $\pi_{\text{ref}}(y|x)$, usually a supervised fine-tuned model trained on

high-quality data.

1. The to-be-trained model: $\pi_\theta(y \mid x)$, usually initialized to $\pi_{\text{ref}}(y|x)$. This is

the *policy* in the literature.

1. A reward model which produces $p(y_w \succ y_l \mid x)$, the probability[79] y_w is favored over y_l. The reward function $r(x, y)$

reflects how well y completes the prefix x, in this context, and we assume the probability can be expressed in terms of the reward function $p(y_w \succ y_l \mid x) = p(r(x, y_w), r(x, y_l))$. The reward model is commonly an LLM with a scalar output head attached.

[76] A terminology note: pre-training is standard next-token training on an enormous, general dataset, supervised fine-tuning typically indicates additional, subsequent training on a higher-quality, maybe domain-specific dataset, and instruction fine-tuning follows.

[77] One failure mode this corrects for: next-token training would do best by replicating common mistakes in grammar or statements of fact which can be corrected for using these methods.

[78] I guess the l, w subscripts are for "lose" and "win"?

[79] Whether one completion is preferred over another is a probabalistic question since, e.g., not everyone in the population will agree.

First, a quick review of RLHF, which proceeds in stages. First, $\mathcal{D}$ is used to train a reward model informed by the dataset $\mathcal{D}$. The optimal reward model r_* minimizes the binary cross-entropy loss over $\mathcal{D}$, which is just

$$\mathcal{L}_r = -E_{x,y_l,y_w\mathcal{D}} \ln p(y_w \succ y_l|\ x). \tag{106}$$

The reward model embodies human preferences and we want to transfer this knowledge to the language model π_θ. This can be done by optimizing π_θ to generate completions of inputs that lead to large rewards, reflecting human-preferred generations. In order to also keep the model from straying too far from its reference base, a tunable KL-divergence penalty is also added[80]:

$$\mathcal{L}_{\text{RLHF}} = E_{x\mathcal{D},y\pi_\theta(y|x)}(-r_\star(x,y) + \beta D_{\text{KL}}\Big(\pi_\theta(y|x)\|\ \pi_{\text{ref}))(\text{y}|\text{x}}\Big). \tag{107}$$

Reinforcement-learning methods are typically used to optimize the π_θ model and the generation step is particularly costly. In particular, the usual gradient-based optimization methods cannot be used because the loss depends on generated tokens which are discontinuous (non-differentiable) functions of the model's parameters.

DPO improves upon RLHF by skipping any generation step, removing the explicit reward function, and making the optimization problem amenable to gradient based methods by choosing a specific functional relation between the reward function $r(x,y)$ and the preference probability $p(y_w \succ y_l \mid x)$. Whereas RLHF minimizes the loss $\mathcal{L}_{\text{rlhf}}$ (107) subject to a fixed, optimal reward function found by first minimizing the reward loss $\mathcal{L}_r$ (106);, DPO is essentially derived in the opposite direction: first, find the functional form of π_θ which minimizes the RLHF loss for an arbitrary reward function, and then use this form when minimizing of the cross-entropy defining the reward function[81].

The π_θ which minimizes the RLHF loss (107) for an arbitrary reward function $r(x,y)$ is given by[82]

$$\pi_\theta(y|x) = \frac{\pi_{\text{ref}}(y|x)e^{\frac{r(x,y)}{\beta}}}{Z(x)}, \tag{108}$$

where $Z(x) = \int \text{d}y\, \pi_{\text{ref}(y|x)} e^{\frac{r(x,y)}{\beta}}$ is a intractable normalization (partition function) factor. However, if $p(y_w \succ y_l \mid x)$ only depends on $r(x,y_w)$ and $r(x,y_l)$ through their difference[83], these

[80] We've written the above as a loss so that we're minimizing everywhere.

[81] This is analogous to minimizing the regular function $f(x,y)$ subject to also minimizing $g(x)$. This can either be done by solving the second for x_* and minimizing $f(x_*,y)$ (the RLHF strategy), or first solving $\frac{\partial f}{\partial y} = 0$ to find $x_*(y)$ and then minimizing $g(x_*(y))$ (the DPO strategy).

[82] This is easy to show using the calculus of variations, though it's not the route taken in the paper. The explicit RLHF loss is $\mathcal{L}_{\text{RLHF}} = \int \text{d}x\ \text{d}y\, p(x)\pi_\theta(y|x)\Big(-r(x,y) + \beta \ln \pi_\theta \frac{y|x}{\pi_{\text{ref}(y|x)}}\Big)$ and we want to minimize this subject to the constraint that $\pi_\theta(y \mid x)$ is properly normalized. So, we use a Lagrange multiplier and extremize $\mathcal{L}' = \mathcal{L}_{\text{RLHF}} + \int \text{d}x\ \text{d}y\, \lambda(x)\pi_\theta(y|x)$. Solving $\frac{\delta \mathcal{L}'}{\delta \pi_\theta(y|x)} = 0$ yields Equation 108;.

[83] In [29], the DPO symmetry $r(x,y) \to r(x,y) + f(x)$, for arbitrary $f(x)$, is said to induce an equivalence class relation between different reward functions.

factors cancel out. Letting $p(y_w \succ y_l \mid x) = \sigma(r(x, y_w) - r(x, y_l))$, for some[84] σ, and eliminating the reward function in the cross-entropy loss via (108) reduces $\mathcal{L}_r$ to

$$\mathcal{L}_{\text{DPO}} = -E_{x,y_l,y_w\mathcal{D}} \ln \sigma \left(\beta \left(\ln \frac{\pi_\theta(y_w|x)}{\pi_{\text{ref}(y_w|x)}} - \ln \frac{\pi_\theta(y_l|x)}{\pi_{\text{ref}(y_l|x)}} \right) \right), \tag{109}$$

which we've now renamed the DPO loss. The loss (109) can now be minimized by standard, gradient based methods without any generation step.

4.1.2. KTO: Preference Finetuning without Pairs

DPO requires a dataset of triplets: a prefix, one preferred completion, and one dispreferred completion. KTO alignment [31] attempts to reduce the inputs a prefix, a completion, and a binary signal indicating whether the output is desirable or not, since such datasets are easier to construct.

The method is based on the ideas of Kahneman and Tversky and the central ingredient is a value function which monotonically maps outcomes to perceived values $v : \mathcal{Z} \to \mathbb{R}$, with $\mathcal{Z}$ the space of outcomes. Some normalization point z_0 defines the boundary between positive and negative outcomes, the value function[85] is taken to be a function of $z - z_0$, and human value functions are known to be convex for $z > z_0$ (diminishing returns) and exhibit loss aversion[86].

KTO applies this framework to the usual text-prediction problem as in the following. The space of outcomes $\mathcal{Z}$ is the reward function value taken to be

$$r_\theta(x, y) \equiv \ln \frac{\pi_\theta(y|x)}{\pi_{\text{ref}}}(y|x), \tag{110}$$

the difference in reference and model surprisal, as inspired by DPO. The reference point is just the expected value of the reward function over prefixes and trainable-model-generated completions, i.e., the KL divergence averaged over prefixes:

$$z_0 \equiv E_{y\pi_\theta(y|x),xD} r_\theta(x, y) = E_{xD} D_{\text{KL}\left(\pi_\theta(y|x) \| \, \pi_{\text{ref}(y|x)}\right)}. \tag{111}$$

Splitting the space of completions into desirable and undesirable ones, $\mathcal{Y} = \mathcal{Y}_D \cup \mathcal{Y}_U$, the KTO loss[87] is taken to be:

$$\mathcal{L}_{\text{KTO}} = -E_{x,yD} v(r_\theta(x, y) - z_0) \tag{112}$$

[84] In the specific case where σ is the sigmoid function, this is known as the Bradley-Terry model.

[85] Which can be taken to satisfy $v(0) = 0$.

[86] Which I suppose means that $v(z - z_0) + v(z_0 - z) \leq 0$ for $z > 0$.

[87] They also add a constant term to the loss for normalization purposes which we have omitted. The KTO loss falls into the broader category of Human Aware Loss Objectives (HALOs) which are a general class of objectives that roughly fit into the Kahneman-Tversky form. See the paper for a further discussion and comparison of HALO vs non-HALO methods.

$$v(r_\theta(x,y) - z_0) \equiv \begin{cases} \lambda_D \sigma(\beta(r_\theta(x,y) - z_0)) & y \in \mathcal{Y}_D \\ \lambda_U \sigma(\beta(-r_\theta(x,y) + z_0)) & y \in \mathcal{Y}_U \end{cases} \tag{112}$$

for hyperparameters[88] $\beta, \lambda_D, \lambda_U \in \mathbb{R}^+$ and where σ is the sigmoid function. So, $v(r_\theta(x,y) - z_0)$ is maximized by sending $r_\theta \to \infty$ for desirable results and to $-\infty$ for undesirable ones, while the normalization point z_0 concentrates updates on examples whose rewards do not stray wildly from the average reward, which implicitly carries information about the reference model.

The reference point z_0 (111) is a problem, because it requires generation which is both expensive and not differentiable (the problem DPO solves). So, the authors perform a rough estimate of the scale and do not backpropagate through z_0, (which is a bit questionable).

5. RL

5.1. Background

See Jared Kaplan's notes for a concise review of RL.

5.1.1. Terminology and Fundamentals

In Reinforcement Learning (RL), a *policy* is a map from states to actions: $\pi : S \longrightarrow A$. From this perspective, a LLM is a policy whose actions are predictions and whose state is all previously seen tokens.

Other terminology:

- **On-Policy vs Off-Policy**: On-policy RL only learns based on the current policy, whereas off-policy incorporates information from other policies. RL with a larger teacher model is an example of off-policy, as is performing multiple policy-gradient optimizer updates (see below) with the reference model held fixed, as opposed to updating the reference model after each step.
- **Reward Function**: the reward for the given action taken at time t. RL attempts to maximize some function of rewards. If s_t is the state at time t and a_t the action taken, then $R_t = R_t(a_t, s_t, s_{t+1})$ in general, though for LLM RL often $R_t = R_t(a_t)$ alone.
- **Advantage Function**: just the difference between the reward function and some baseline, $A_t = A_t(a_t, s_t, s_{t+1})$. Sometimes more useful than a reward function directly.

Starting from some initial state described by distribution $\rho(s_0)$, the probability distribution for a specific state-action pair *trajectory*, often denoted by τ, is given by[89]

$$P(s_T, a_{T-1}, ..., a_0, s_0, \pi) \equiv P(\tau, \pi) = \rho(s_0) \prod_{t=0}^{T-1} p(s_{t+1} |\ a_t, s_t) \pi(a_t | s_t) \tag{113}$$

[88] Risk aversion would seem to require $\lambda_U > \lambda_D$, but the KTO paper empirically finds that the opposite regime performs better.

[89] Note that this is not a completely general formula: we have some Markov-like assumptions here.

where the policy $\pi(a_t|s_t)$ describes the probability that action a_t is taken given the current state s_t, and where $p(s_{t+1}|\ a_t, s_t)$ is a distribution describing the probability of transitioning to state s_{t+1} given that action a_t was taken.

5.1.2. Policy Gradients

In *policy gradient* (PG) RL, the goal is to learn an improved policy which maximizes some reward function which grades the policy's chosen actions[90]. In general, the goal is to maximize some quantity R defined over the state-action path taken by the system. A common historical choice is to use a weighted sum[91] of rewards (or advantages): $R(s_T, a_{T-1}, ..., s_0) = \sum_t \gamma^t R_t(a_t, s_t, s_{t+1})$. The expected value is then

$$\begin{aligned} J(\pi) &= \left(\prod_{t=0}^{T-1} \int da_t ds_t\right) \rho(s_0) \prod_{t=0}^{T-1} p(s_{t+1}|\ a_t, s_t)\pi(a_t, s_t) R(s_T, a_{T-1}, ..., s_0) \\ &= E[R|\pi] \equiv \langle R \rangle_\tau \end{aligned} \tag{114}$$

$J(\pi)$ is a functional of the policy and is called the **Policy Value** or **Expected Return**. Other related quantities are defined by integrating over fewer variables:

- The **Value Function** $V(\pi, s_t) = E[R|\pi, s_t]$, $J(\pi) = \int ds_t V(\pi, s_t)$. Useful for comparing the relative merits of different states.
- The **Q Function** $Q(\pi, a_t, s_t) = E[R|\pi, a_t, s_t]$, $J(\pi) = \int ds_t da_t Q(\pi, a_t, s_t)$. Given that you're in state s_t, the Q function can be used to determine your optimal action, by maximizing over a_t.

If the policy depend on parameters θ, $\pi = \pi_\theta$, then we could attempt to maximize the above through gradient-based optimization strategies, as usual. Taking a gradient and manipulating terms, we have $J(\pi) = J(\theta)$ and

$$\begin{aligned} \nabla_\theta J(\theta) &= \left(\prod_{t=0}^{T-1} \int da_t ds_t\right) \rho(s_0) \prod_{t=0}^{T-1} p(s_{t+1}|\ a_t, s_t)\pi_\theta(a_t, s_t) R(s_T, a_{T-1}, ..., s_0) \\ &\quad \times \sum_{t'=0}^{T-1} \nabla_\theta \ln \pi_\theta(a_{t'}, s_{t'}) \\ &= \left(\prod_{t=0}^{T-1} \int da_t ds_t\right) P(s_T, a_{T-1}, ..., a_0, s_0, \pi) R(s_T, a_{T-1}, ..., s_0) \times \sum_{t'=0}^{T-1} \nabla_\theta \ln \pi_\theta(a_{t'}, s_{t'}) \\ &\equiv \langle R \sum_\tau \nabla_\theta \ln \pi_\theta \rangle_\tau. \end{aligned} \tag{115}$$

Some practical notes:

- The transition probabilities are generally not known and not being modeled, rather they are just treated as an environment[92], and so we estimate the average by generating multiple trajectories and averaging over them: $S(\theta) = \langle R \sum_\tau \ln \pi_\theta \rangle_{\text{empirical}}$

[90] An alternative RL method is Q-learning, which we skip for now.

[91] This can help with training variance and stability.

- A subtlety is that this *surrogate* function $S(\theta)$ is not the same as the expectation value above: $S(\theta) \neq J(\theta)$. Only their first derivatives match in general[93]. $S(\theta)$ itself is therefore not an inherently meaningful quantity and so you may not want to plot the surrogate loss! Decreasing surrogate loss is not necessarily informative of anything.

5.1.3. Specialization to LLMs

For generative language models, the state s_t is just the current prefix and $\pi_\theta(a_t|s_t)$ is the probability the LLM assigns for a_t to be the next token. The initial state s_0 is often some initial prompt, usually a question we'd like the model to be able to answer. Unlike in canonical RL cases, the transition probabilities here are trivial: $p(s_{t+1}|a_t, s_t) = \delta(s_{t+1} - \texttt{Concat}(a_t, s_t))$, and $\rho(s_0)$ is similarly a delta function.

5.2. PPO

Proximal policy optimization [30] (PPO) is the starting point which most LLM RL pipelines build upon. Essentially, it just adds clipping to the surrogate objective.

Let $\pi_\theta(a_t|s_t)$ be the current model, $\pi_{\theta_{\text{old}}}(a_t|s_t)$ be the same model with parameters θ_{old}, and $r_t(\theta) = \frac{\pi_\theta(y_t|x_{t-1})}{\pi_{\theta_{\text{old}}}(y_t|x_{t-1})}$. The PPO empirical surrogate objective is:

$$S(\theta) = \langle \sum_t \min(r_t(\theta)A_t, \text{clip}(r_t(\theta), 1-\varepsilon, 1+\varepsilon)A_t) \rangle_{\text{empirical}} \ . \tag{116}$$

Here $\varepsilon > 0$ is a hyperparameter[94], A_t is the advantage, and $\text{clip}(x, l, h)$ is

$$\text{clip}(x, l, h) := \begin{cases} l \text{ if } x < l \\ x \text{ if } l \leq x \leq h \\ h \text{ if } h < x \end{cases} \tag{117}$$

In practice, the min and clip operations are applied for every token prediction and averaged over the sequence length, rather than computed once at the sequence level, but we leave the notation schematic above and will be more precise in later sections. Note that if all the RL steps are strictly on-policy, then $\theta = \theta_{\text{old}}$ and the clipping is completely trivial[95].

The PPO surrogate creates the same derivatives as the surrogate in Section 5.1.2 in the limit where (using the same advantage function) we take the derivative with respect to θ and set $\theta_{\text{old}} \longrightarrow \theta$ post-derivative. So, a typical workflow could be:

1. Start from a given checkpoint and perform some batch of PPO RL. The gradients for this step will be the same as vanilla PG, since the clipping does nothing.

[92] Think of training a model $\pi_\theta(a_t|s_t)$ to learn a video game with non-trivial random elements in it, as described by $p(s_{t+1}|a_t, s_t)$.

[93] Due to the gradient only hitting the ln term in the empirical average, while they also hit the distribution in $J(\theta)$. For example: $\nabla_i \nabla_j J(\theta) = \langle R(\nabla_i \ln \pi_\theta \nabla_j \ln \pi_\theta + \nabla_i \nabla_j \ln \pi_\theta) \rangle$ while $\nabla_i \nabla_j S(\theta) = \langle R \nabla_i \nabla_j \ln \pi_\theta \rangle_{\text{empirical}}$, schematically.

[94] $\varepsilon \sim \mathcal{O}(1e-1)$ is common

[95] In this case, we take the derivative before setting $\theta \longrightarrow \theta_{\text{old}}$, which still trivializes the derivative.

2. Perform more PPO RL steps with $\pi_{\theta_{\text{old}}}$ kept as the original checkpoint. The clipping is non-trivial for these steps.
3. Update $\pi_{\theta_{\text{old}}}$ at some point[96].

The clipping is punitive:

- If for a given term in the empirical average, A_t is positive and $r_t > 1 + \varepsilon$, then the contribution to $S(\theta)$ is $\sim (1 + \varepsilon)A_t$ which is θ-independent and which therefore provides no gradient to the learning algorithm.
- For $A_t < 0$ terms, non-confident $r_t < 1 - \varepsilon$ cases will similarly produce no gradient.

So, very confident moves which lead to worse outcomes still provide gradients, while very confident moves towards better outcomes get their gradients discarded.

In order for the predictions of the updated model to avoid straying too far from the initial checkpoint, a $-\beta D_{\text{KL}}\left(\pi_{\theta_{\text{old}}} \| \pi_\theta\right)$ penalty term is sometimes added.

5.3. DeepSeek R1 and GRPO

We left the PPO advantage function unspecified in Section 5.2, but historically it involves specifying an independent critic model and having multiple in-flight models is a headache.

In DeepSeek's work on Group-Relative Policy Optimization (GRPO) [32] as applied to their R1 model [33], they dispense with the need for a secondary model and define their advantage function as follows. The reference model $\pi_{\theta_{\text{old}}}$ is the current version of the model.

1. For initial state s_0 (that is, prefix question), sample G total responses (trajectories) from $\pi_{\theta_{\text{old}}}$: a_g, $g \in \{0, ..., G-1\}$.
2. A reward is computed for each such response: r_g. Programmatically verifiable rewards are especially attractive for this step, where r_g takes on some pair of binary values depending on whether the answer is correct or not[97].
3. The advantage of each response is just the z-score:

$$A_{t,g} = \frac{r_g - \text{mean}_{g'}\left(r_{g'}\right)}{\text{std}_{g'}\left(r_{g'}\right)} \tag{118}$$

If we always take the current model as the $\pi_{\theta_{\text{old}}}$, then we use (116) with $\theta_{\text{old}} \longrightarrow \theta$ post-derivative, in which case the min and clip operations are trivial. DeepSeek's GRPO also uses the KL-divergence penalty, but uses a different unbiased estimator than the naive one.

An important point is that $A_{t,g}$ is independent of t: we are assigning the same reward to every step in the trajectory. So, writing $A_{t,g} = A_g$ the policy gradients factorize in the form:

[96] A confusing point: as of Summer 2025, many PPO libraries and variants update $\pi_{\theta_{\text{old}}}$ after every optimizer step (i.e. they are "on-policy"), in which case the min and clipping are completely irrelevant. The appendix of [32] has a good discussion of this and comparison of different training methods.

[97] Such simple scoring is also claimed to be robust against "reward hacking", where the model optimizes RL rewards through unintended means, e.g. through undesirably short or long answer, style-over-substance generation, etc.

$$\nabla S(\theta) = \langle \sum_{t,g} A_{t,g} \nabla_\theta C(r_\theta) \rangle_\tau = \langle \sum_g A_g \sum_t \nabla_\theta C(r_\theta) \rangle_\tau \tag{119}$$

where $C(r_\theta)$ is the clipping function above, or some similar variant thereof. Thus there is no token-specific feedback with these advantages, as there might be in some more sophisticated scheme.

5.4. DAPO

Dynamic sAmpling Policy Optimization (DAPO) [34] proposes several improvements over GRPO:

1. Dropping the KL divergence penalty.
2. Weighting each token in each response equally, rather than weighting each total response equally. Specifically, DAPO does this by dividing the total surrogate by the sum of all tokens.
3. Introducing independent high and low clipping parameters: $\varepsilon \longrightarrow \varepsilon_{\text{low}}, \varepsilon_{\text{high}}$ with $\varepsilon_{\text{high}} > \varepsilon_{\text{low}}$.
4. The GRPO style advantage (118) is still used, but because this advantage leads to zero gradient when all responses are either correct or incorrect, DAPO continually samples responses and rejects batches which have such trivial gradients, continuing until the desired DAPO global batch size is reached.

Reasoning and justification:

1. Unclear, maybe overly restrictive to generating CoT and reasoning behavior effectively.
2. Penalizes overly-long and poor CoT responses, which are a common failure mode.
3. Decoupling the clipping with $\varepsilon_{\text{high}} > \varepsilon_{\text{low}}$ encourages more exploration from positive-advantage samples, , relative to the $\varepsilon_{\text{high}} = \varepsilon_{\text{low}}$ baseline, as grads now contribute for $A_t > 0$ whenever $r_t(\theta) < 1 + \varepsilon_{\text{high}}$.
4. As the model improves, the number of responses groups which are all correct tends to increase, so this strategy naturally injects more signal by discarding such cases.

More from DAPO:

- They penalize responses which exceed some pre-defined threshold, which they say greatly helps stability.
- They track the following quantities during training:
  - Mean response length.
  - Mean reward score.
  - Entropy (of what, exactly? mean entropy over the vobcab averaged over response length?)
  - Generation probabilities (also unclear how this is defined)

5.5. Skywork

Skywork [35] uses some similar tactics to DAPO (Sec. Section 5.4):

1. Filtering out prompts that yield all-correct or all-wrong answers within each iteration, and removing all-correct prompts entirely from future iterations.
2. Multi-stage training, with progressively increasing sequence lengths.

3. An entropy-per-token term is added to the surrogate, with its coefficient dynamically adjusted to maintain a targeted entropy level.
4. Also does not average over tokens within a sequence, but unlike DAPO they remove all token-counting division factors entirely, so the gradient scales with the number of tokens in a batch
5. No KL loss term.

6. Parallelism

The simplicity of the Transformers architecture lends itself to a deep variety of parallelism strategies. We review some of them below.

6.1. Tensor Parallelism

Side Note: I wrote a blog post on this here.

In **Tensor Parallelism**, some times also called **Model Parallelism**, individual weight matrices are split across devices [36]. We consider the and layers in turn. Assume T-way parallelism such that we split some hidden dimension into T-equal parts across T workers[98]

> **Essentials:** The cost of large weights can be amortized by first sharding its output dimension, resulting in differing activations across group members. Later, the activations are brought back in sync via a . Weights which act on the sharded-activations can also be sharded in their input dimension. In the backwards pass, another `AllReduce` is required.

6.1.1. MLP

It is straightforward to find the reasonable ways in which the weights can be partitioned. We suppress all indices apart from those of the hidden dimension for clarity.

The first matrix multiply $z_d W^0_{de}$ is naturally partitioned across the output index, which spans the expanded hidden dimension $e \in (0, ..., ED-1)$. This functions by splitting the weight matrix across its output indices across T devices: $W^0_{de} = W^0_{d(ft)} \equiv \bar{W}^0_{df\bar{t}}$ (again in -like notation, with bars denoting that the tensor and particular indices are sharded; see Appendix A), where in the split weights $\bar{t} \in (0, ..., T-1)$, and $f \in \left(0, ..., \frac{ED}{T} - 1\right)$. Each of the T workers compute one shard of $z_d \bar{W}^0_{df\bar{t}}$, i.e. each has a different value of $\bar{t}$.

Let the partial outputs from the previous step be $\bar{z}_{f\bar{t}}$ (batch-index suppressed), which are -shaped, with the final dimension sharded across workers. The non-linearity φ acts element wise, and using the updated $\bar{z}_{f\bar{t}}$ to compute the second matrix multiply requires a splitting the weights as in $W^1_{ed'} = W^1_{(ft)d'} \equiv \bar{W}^1_{f\bar{t}d'}$ (dividing up the incoming e dimension), such that the desired output is computed as in $\bar{z}_{f\bar{t}} \cdot \bar{W}^1_{f\bar{t}d'}$, sum over $\bar{t}$ implied. Each device has only $\bar{t}$ component in the sum

[98] All T workers work on processing the same batch collectively. With $N > T$ workers, with N perfectly divisible by T, there are N/T different data parallel groups. Critical-path TP communications occur within each data parallel group and gradients are synced across groups. Ideally, all the workers in a group reside on the same node, hence the usual $T = 8$.

(a -shaped tensor) and an is used to give all workers the final result. This is the only forward-pass collective communication[99].

One-line summary of the parallel decomposition:

$$z_{sd'} \leftarrow \varphi\big(z_d W^0_{de}\big) W^1_{ed'} = \varphi\Big(z_d \bar{W}^0_{df\bar{t}}\Big) \bar{W}^1_{f\bar{t}d'} \,. \tag{120}$$

The progression of tensor shapes held by any single worker is

1. `(B, S, D)`
2. `(B, S, E*D/T)`
3. `(B, S, D)`

In the backwards pass, another `AllReduce` (see Appendix B) is needed for proper gradient computations with respect to the first layer's outputs. This is true whenever an operation producing a sharded output involved non-sharded tensors: if an operation $\bar{y}_{\bar{r}} = F(x, ...)$ produces a sharded output from an unsharded in put x (all other indices suppressed), the derivative with respect to x requires a sum over ranks, $\frac{\partial \mathcal{L}}{\partial x} = \frac{\partial \mathcal{L}}{\partial \bar{y}_{\bar{r}}} \frac{\partial \bar{y}_{\bar{r}}}{\partial x}$. Note that each worker will have to store all components of the input z for the backward pass.

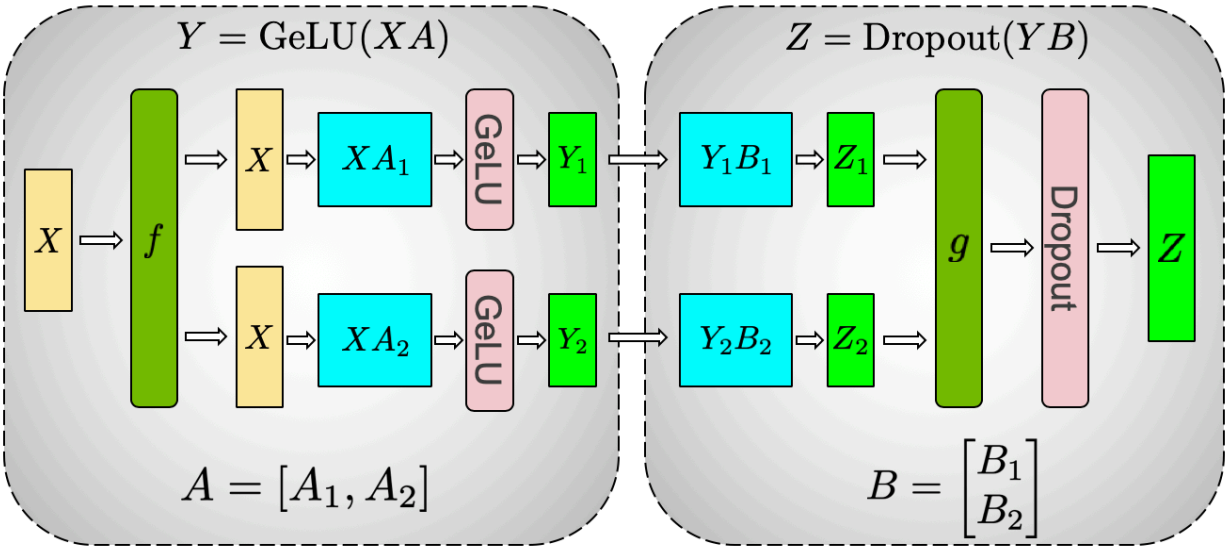


Figure 4: Tensor parallelism for the layers. Graphic from [36]. The f/g operations are the collective identity/ operations in the forwards pass and the /identity operations in the backwards pass.

6.1.2. Attention

Because the individual attention head computations are independent, they can be partitioned across T workers without collectively communications. An is needed for the final projection, however, which results in the various re-weighted values y_{bsea} (6);.

To review, the attention outputs z'_{sd} generated from inputs z_{sd} can be expressed as

[99]The amount of communicated data is $\mathcal{O}(BSD)$.

$$z'_{sea} = \texttt{MHA}(q_{sea}, k_{sea}, v_{sea})O_{ead} \tag{121}$$

where:

- We have split the d-index as in $z_{sd} \to z_{s(ea)}$ with e and a the head-dimension and head-index
- $q_{sea}, k_{sea}, v_{sea}$ are the query, keys and values derived from the inputs
- `MHA` is the multi-head attention function, whose outputs are the same shape as its value inputs
- The dual sum over head-dimension index (e) and attention-head-index (a) is the sum-and-concatenate step from the more explicit description in Section 1.1.3
- and biases were ignored for simplicity

In order to parallelize the above T-ways, we simply shard across the dimension a which indexes the different attention heads. The `MHA` computations all process in embarassingly-parallel fashion, and an all-reduce is needed to complete the sum over the a-index across devices.

The collective communications story is essentially equivalent to that of the layers[100]: one is needed in the forwards pass and one in the backwards-pass.

The progression of tensor shapes held by any single worker is

1. `(B, S, D)`
2. `(B, S, D/A, A/T)`
3. `(B, S, D)`

It is worth comparing the communications and FLOPs costs of these sharded layers. Each layer costs $\mathcal{O}\left(BS(4+2E)\frac{D^2}{T}\right)$ FLOPs and communicates $\mathcal{O}(BSD)$ bytes and so the communication-to-compute-time ratio is

$$\frac{t_{\text{compute}}}{t_{\text{comms}}}\frac{(4+2E)D}{T} \times \frac{\lambda_{\text{comms}}}{\lambda_{\text{FLOP/s}}}. \tag{122}$$

Since[101] $\frac{\lambda_{\text{comms}}}{\lambda_{\text{FLOP/s}}} 10^{-3}$FLOPs/B, communication and compute take similar times when $D\mathcal{O}(10^3)$ for typical setups with $T\mathcal{O}(10)$ and so tensor-parallelism requires $D \gtrsim 10^4$ to reach similar efficiency to the non-tensor-parallel implementations.

[100]The amount of communicated data is again $\mathcal{O}(BSD)$.

[101]Assuming $\lambda_{\text{FLOP/s}}$100 TFLOP/s and λ_{comms} 100 GiB/s.

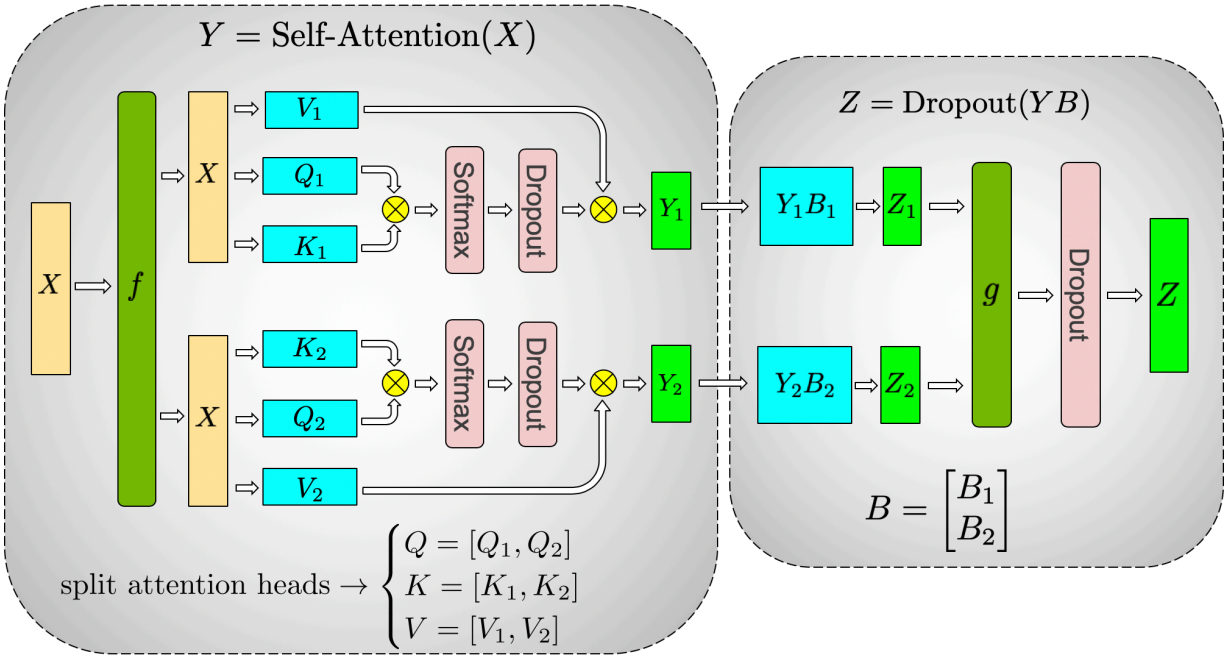


Figure 5: Tensor parallelism for the layers. Graphic from [36]. The f/g operators play the same role as in Figure 4.

6.1.3. Embedding and LM Head

Last, we can apply tensor parallelism to the language model head, which will also necessitate sharding the embedding layer, if the two share weights, as typical.

For the LM head, we shard the output dimension as should be now familiar, ending up with T different -shaped tensors, one per group member. Rather than communicating these large tensors around and then computing the cross-entropy loss, it is more efficient to have each worker compute their own loss where possible and then communicate the scalar losses around[102].

For a weight-tied embedding layer, the former construction requires in order for every worker to get the full continuous representation of the input.

6.1.4. LayerNorm and Dropout

instances are not sharded in pure tensor parallelism both because there is less gain in sharding them parameter-wise, but also sharding in particular would require additional cross-worker communication, which we wish to reduce as much as possible. layers are also not sharded in where possible in pure tensor parallelism, but sharding the post-attention layer is unavoidable. It is the goal of sequence parallelism is to shard these layers efficiently; see Section 6.2.

[102] In more detail, given the gold-answers y_{bs} for the next-token-targets, a given worker can compute their contribution to the loss whenever their -shaped output $z_{bsv'}$ contains the vocabulary dimension v_* specified by y_{bs}, otherwise those tensor components are ignored.

6.1.5. Effects on Memory

The per-worker memory savings come from the sharding of the weights and the reduced activation memory from sharded intermediate representations.

The gradient and optimizer state memory cost is proportional to the number of parameters local to each worker (later we will also consider sharding these components to reduce redundantly-held information). The number of parameters per worker is reduced to

$$N_{\text{params}} \approx (4 + 2E)\frac{LD^2}{T}, \tag{123}$$

counting only the dominant contribution from weights which scale with L, since every weight is sharded. Since all non-activation contributions to training memory scale with N_{params}, this is a pure $1/T$ improvement.

The per-layer activation memory costs (91) and (92) are altered to:

$$\begin{aligned} M_{\text{act}}^{\text{Attention}} &= BS\left(\left(p + \frac{4p}{T} + 1\right)D + \left(\frac{2p+1}{T}\right)AS\right) \\ M_{\text{act}}^{\texttt{MLP}} &= \left(\frac{2Ep}{T} + p + 1\right)BDS. \end{aligned} \tag{124}$$

The derivation is similar to before. Adding in the (unchanged) contributions from instances, the total, leading order activation memory sums to

$$M_{\text{act}}^{\text{total}} \approx 2BDLS\left(p\left(2 + \frac{E+2}{T}\right) + 1\right) + ABLS^2\left(\frac{2p+1}{T}\right). \tag{125}$$

Again, the terms which did not receive the $1/T$ enhancement correspond to activations from unsharded and instances and the $1/T$'s improvements can be enacted by layering sequence parallelism on top (Section 6.2).

6.2. Sequence Parallelism

In (125) not every factor is reduced by T. **Sequence Parallelism** fixes that by noting that the remaining contributions, which essentially come from and[103], can be parallelized in the sequence dimension (as can the residual connections).

The collective communications change a bit. If we shard the tensors across the sequence dimension before the first , then we want the following:

1. The sequence dimension must be restored for the layer
2. The sequence should be re-split along the sequence dimension for the next instance
3. The sequence dimension should be restored for the layer[104]

[103] Recall, though, from Section 1.1.2 that the parameters in are completely redundant and can simply be removed without having any effect on the expressive capabilities of the architecture.

The easiest way to achieve the above is the following.

1. If the tensor parallelization degree is T, we also use sequence parallelization degree T.
2. The outputs of the first are -ed to form the full-dimension inputs to the layer
3. The tensor-parallel layer functions much like in Figure 5 *except* that we do not re-form the outputs to full-dimensionality. Instead, before the layer, we them from being hidden-sharded to sequence-sharded and pass them through the subsequent / combination, similar to the first step
4. The now-sequence-sharded tensors are reformed with another to be the full-dimensionality inputs to the layer whose final outputs are similarly -ed to be sequence-sharded and are recombined with the residual stream

The above allows the mask and weights to be sharded T-ways, but if we save the full inputs to the and layers for the backwards pass, their contributions to the activation memory are not reduced (the p-dependent terms in (124);). In [5], they solve this by only saving a $1/T$ shard of these inputs on each device during the forward pass and then performing an extra when needed during the backwards pass. Schematics can be s e n in Figure 6 and Figure 7 below. The activation memory is then reduced to:

$$M_{\text{act}}^{\text{total}} = (2BDLS\frac{p(E+4)+1}{T} + \frac{ABLS^2(2p+1)}{T} + \mathcal{O}(BSV). \tag{126}$$

In more detail:

- The norms are just linear operations on the z_{sd}, $z'_{sd} = \text{NORM}\ (z_{sd})$, and so we split and shard cross the sequence dimension $z_{sd} \to z_{(tr)d} \equiv \bar{z}_{\bar{t}rd}$ with the TP-index t sharded across devices.
- The residual stream is also sharded across the sequence dimension.
- The sharded outputs $\bar{z}_{\bar{t}rd}$ must be re-formed to create the attention and MLP inputs via an . There is an optimization choice here: either the re-formed tensors can be saved for the backward pass (negating the $1/T$ memory savings) or they can be re-formed via an , at the cost of extra communication.
- Both the MLP and attention layers need to produce final sums of the form $\bar{y}_{s\bar{y}e}\bar{O}_{\bar{t}ed}$ for some intermediate $\bar{y}$ and weight $\bar{O}$ sharded across the TP-dimension $\bar{t}$. The outputs are added to the sequence-sharded residual stream, and so sum is optimally computed through an with final shape $\bar{z}_{\bar{t}'rd} = z_{(t'r)d} = z_{sd} = \bar{y}_{s\bar{t}e}\bar{O}_{\bar{t}ed}$. This (along with the mentioned above) replace the s from the tensor-parallel case and have the same overall communication cost.

[104] This doesn't seem like a hard-requirement, but it's what is done in [5].

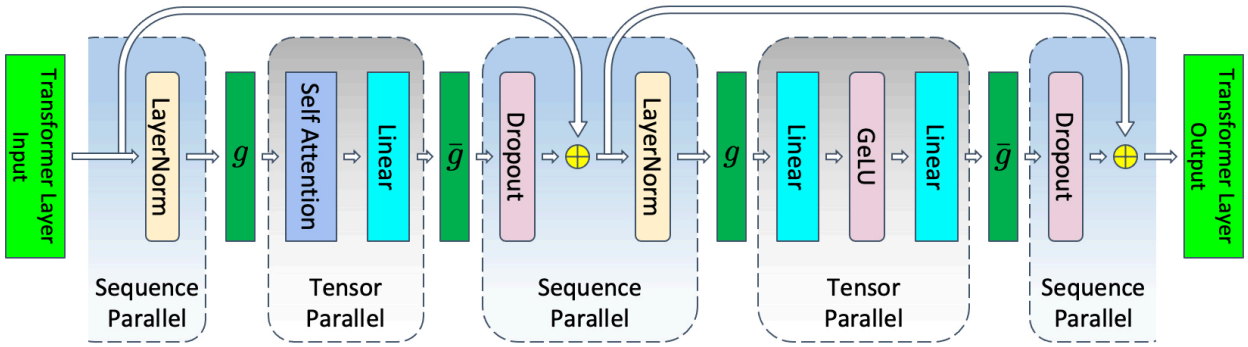


Figure 6: Interleaved sequence and tensor parallel sections. g and $\bar{g}$ are and in the forward pass, respectively, and swap roles in the backwards pass. Graphic from [36].

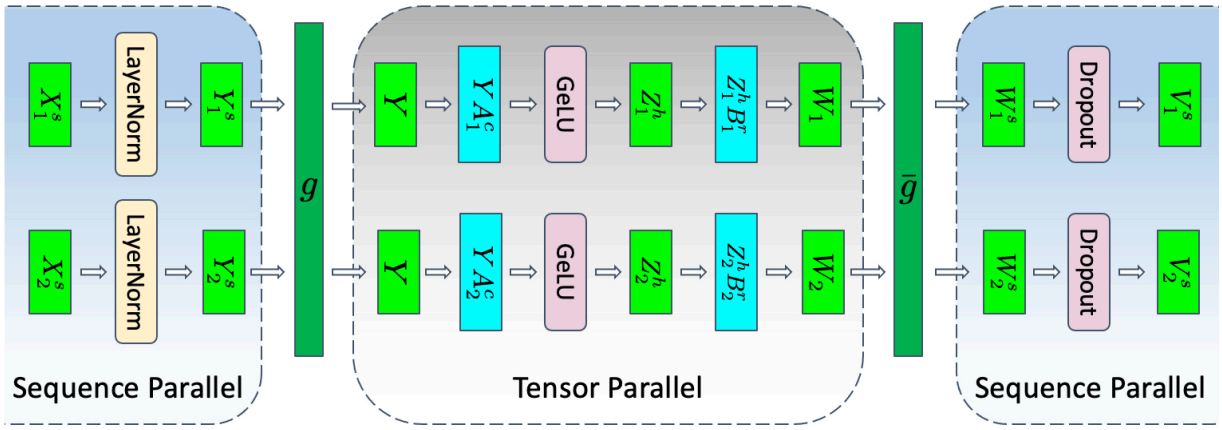


Figure 7: Detail of the sequence-tensor parallel transition for the . Graphic from [36].

6.3. Ring Attention

Ring Attention [37] is roughly a distributed version of Flash Attention Section 1.2.7: it enables extremely long sequence-length processing by never realizing the entire $\mathcal{O}(S^2)$ attention scores at once.

It works by sharding over the sequence dimension. Let z_{sd} is the (batch-dim suppressed) residual stream of a non-sharded Transformer[105]:

$$z_{sd} = \texttt{Softmax}_{s'}(q_{sd'} k_{s'd'}) v_{s'd}, \tag{127}$$

suppressing the causal mask for simplicity of presentation.

Then in Ring Attention, we shard over R devices via $z_{sd} \to z_{\bar{r}td}$, and similar for other tensors, to compute the sharded outputs

$$z_{\bar{r}td} = \texttt{Softmax}_{\bar{w}x}(q_{\bar{r}td'} k_{\bar{w}xd'}) v_{\bar{w}xd} \tag{128}$$

[105] Like in Sec. Section 1.2.7, we omit any normalization factor inside the `Softmax`.

$$
\begin{aligned}
&= \frac{\exp(q_{\bar{r}td'} k_{\bar{w}xd'})}{\sum_{\bar{w}'x'} \exp(q_{\bar{r}td''} k_{\bar{w}'x'd''})} v_{\bar{w}xd} \\
&\equiv \frac{Z_{\bar{r}td}}{\sum_{\bar{w}'x'} \exp(q_{\bar{r}td''} k_{\bar{w}'x'd''})} \\
&\equiv \frac{Z_{\bar{r}td}}{L_{\bar{r}t}} \qquad (128)
\end{aligned}
$$

where we introduced some notation which will be useful blow. Ring Attention is essentially an algorithm for computing the sharded sums over barred indices via communication. Since the MLP layers act on every sequence position identically, only the Attention layers (and the loss computation) require special handling.

The algorithm performs the $\bar{w}$ sum as a loop. We present the simplified case without a causal mask or maximum attention score tracking. These are important omissions[106].

Initialize $Z_{\bar{r}td}$, $L_{\bar{r}t}$ to zeros
Populate the shards $q_{\bar{r}td}, k_{\bar{w}xd'}, v_{\bar{w}xd'}$ with $\bar{r} = \bar{w} = r$ on rank r
For $\bar{w} \in \{r, ..., R-1, 0, ..., r-1\}$ `# Compute z[r, t, d] for all t, d`
  If $\bar{w} \neq (r-1) \bmod R$: prefetch shards $k_{(\bar{w}+1)xd}, v_{(\bar{w}+1)xd}$ while computing below
  $Z_{\bar{r}td} \leftarrow Z_{\bar{r}td} + \exp(q_{\bar{r}td'} k_{\bar{w}xd'}) v_{\bar{w}xd}$ `# Can use flash attention kernels here`
  $L_{\bar{r}t} \leftarrow L_{\bar{r}t} + \sum_x \exp(q_{\bar{r}td'} k_{\bar{w}xd'})$ `# Can use flash attention kernels here`
Return $z_{\bar{r}td} \leftarrow \frac{Z_{\bar{r}td}}{L_{\bar{r}t}}$

Algorithm 4: Ring Attention (Naive - Missing causal mask/max tracking.) Attention Backwards Pass

At every step in the loop in the algorithm we are computing the sums $\exp(q_{\bar{r}td'} k_{\bar{w}xd'}) v_{\bar{w}xd}$ and $\sum_x \exp(q_{\bar{r}td'} k_{\bar{w}xd'})$ for fixed values of $\bar{r}, \bar{w}$ and all values of the other indices. These are precisely the ingredients that go into the usual attention computation and for this reason it's possible to use flash attention kernels for every individual step. implementations of Ring Attention which leverage flash attention kernels can be found here and here.

The full forms of the forwards and backwards passes are again similar to those of flash attention; see Sec. Section 1.2.7.1.

6.3.0.1. The Causal Mask

A naive, row-major sharding of the queries, keys, and vectors is highly suboptimal for causal attention because it leads to idling GPUs. Sharding the queries and keys as in $q_s = q_{(\bar{r}t)}$ and $k_{s'} = k_{(t'\bar{r}')}$ in row-major order[107], causality means that the entire chunked attention computation will

[106] See [38] for causal mask efficiency considerations.
[107] That is, $s = \bar{r}T + t$ for $\bar{r} \in (0, ..., R-1)$ and $t \in (0, ...T-1)$ with $S = RT$.

be trivial for any iteration in which $r' > r$. This is the case for $R-1$ iterations for the $r=0$ GPU, for instance.

So, we shouldn't shard this way for ring attention. In [38] they demonstrate the speed-up achieved by just reversing the sharding pattern to column-major: $q_s = q_{(t\bar{r})}$ and $k_{s'} = k_{(t'\bar{r}')}$ which guarantees non-trivial work for every GPU on every iteration, which they call striped ring attention. In the `ring-flash-attention` repo, they come up with yet another sharding strategy ("zig-zag" attention; see this github issue) which increases efficiency even more. Their strategy can't be naturally writtein in `einops` notation, but it is easy enough to explain: they split the sequence length into $2R$ sequential chunks and give zero-indexed chunks r and $2R-r-1$ to GPU r, which ends up optimally distributing the work.

We analyze the efficiency of each strategy now. Let q_s be sharded to $q_{\bar{r}t}$ according to the strategy's specifics and similar for k_s. On the first iteration every rank has keys and queries with identical sequence positions, meaning $\frac{\frac{S}{R}\left(\frac{S}{R}+1\right)}{2}$ positions will be attended to on every rank. The difference comes about in the subsequent iterations:

1. For naive ring attention, it's all or nothing. $q_{\bar{r}t}$ can attend to all of $k_{\bar{w}x}$ if $\bar{r} \geq \bar{w}$. This means that at least one rank needs to perform S^2/R^2 operations every iteration (after the first one), bottlenecking processes which have no work.

2. For striped attention ring attention, rank $r = R-1$ will have queries corresponding to positions $(R-1, 2R-1, ..., S-1)$ and it will always be able to attend to $\frac{\frac{S}{R}\left(\frac{S}{R}+1\right)}{2}$ positions at every iteration, just like on the first iteration. This rank (and others which perform the same number of operations for some iterations) is the bottleneck, since rank $r=0$, which owns sequence positions $(0, R, ..., S-R-1)$ is only ever able to attend to $\frac{\frac{S}{R}\left(\frac{S}{R}-1\right)}{2}$ positions, since its zero-position component can never attend to any thing after the first iteration. This mismatch between ranks is suboptimal.

3. For zig-zag attention, there are two scenarios. Each produces the same amount of work, which makes this strategy optimal. When $\bar{r} < \bar{w}$ the two sets of position indices covered by t on $q_{\bar{r}t}$ fall between those[108] covered by the x index on $k_{\bar{w}x}$. That is, every index in the query can attend to exactly half of the indices on the keys: $\frac{S^2}{2R^2}$ operations. In the opposite[109] $\bar{w} < \bar{r}$ case, the query positions sandwich those of the keys and so the upper half of the query positions can attend to all of the key's positions, again $\frac{S^2}{2R^2}$ operations. So work is perfectly distributed and no rank serves as a bottleneck.

[108] Example: four ranks with sequence length $S=8$, the rank-zero queries cover positions $(0, 7)$ while the rank-one keys cover $(2, 6)$, so the former sandwich the latter.

[109] The $\bar{w} = \bar{r}$ case was already treated in the first iteration.

6.4. Pipeline Parallelism

Pipeline Parallelism splits the model along the layer dimension, putting different layers on different GPUs in the simplest cases. Model and optimizer state memory is thereby saved by sharding the costs across GPUs.

While this also shards activation memory, efficiency demands that multiple batches be concurrently in flight with all GPUs processing data at all times to avoid any bubbles. This means that efficient pipeline parallelism does not parametrically decrease activation memory usage, relative to ZeRO-3/FSDP.

All of the complication is in the pipeline schedule. Some terminology and notation:

- Gradient aggregation is used in pipeline parallelism, with M concurrent "minibatches" in flight per-pipeline. Each minibatch may contain more than one example batched together, but different elements in the minibatch may be at different stages of the pipeline.
- The pipeline parallel degree P is the number of GPUs used in the scheme.
- t_f and t_b are the times for it would take for the *entire* model, respectively, perform a forward and backwards pass[110] on a single minibatch on a single GPU (assuming it had the memory available to do so).
- We define the bubble efficiency ratio $0 \leq \varepsilon_b \leq 1$ as the ratio of the ideal time for the entire workload to the actual time $\varepsilon_b = t_{\text{ideal}}/t_{\text{actual}}$, where the ideal (and unachievable) compute time for P GPUs is

$$t_{\text{ideal}} = \frac{M}{P} \times \left(t_f + t_b\right). \tag{129}$$

- We assume that the work per pipeline stage is constant.

Having $M \geq P$ is typical and is a strict necessity for efficient usage of simple pipeline strategies, but the interleaved schedules of Section 6.4.4 help alleviate this requirement and we do not assume this relation holds. We largely ignore communication costs in the simplified analysis below.

6.4.1. Naive Schedule: The Worst

The most naive, and worst, schedule is to do the computation maximally sequentially. See Figure 8. However, it is a simple case to analyze and lets us introduce useful notation. Any number of minibatches M can be in flight with this strategy, up to memory constraints.

In this strategy, the pipeline stages (indexed by $p \in \{0, \ldots, P-1\}$) proceed in order, with $p = 0$ performing the forward on all M minibatches at once, then $p = 1$ taking over starting from the communicated activations, etc. The backwards is just in the reversed order.

For the present case, every GPU computes is idle for $P - 1$ out of P stages and each stage performs a total of t_{ideal} compute, so the total actual time is $t_{\text{actual}} = P \times t_{\text{ideal}}$ giving:

[110] $t_b \approx 2t_f$ is generally a good approximation, which we will sometimes use.

$$\varepsilon_b^{\text{naive}} = \frac{1}{P}, \tag{130}$$

which reduces to $e_b = 1$ as $P \longrightarrow 1$, as it should.

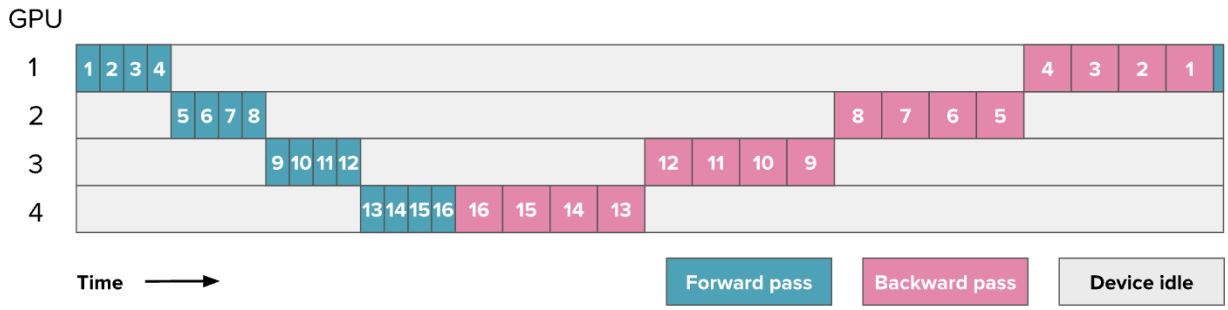


Figure 8: Diagram of naive pipelining, from [39].

6.4.2. All Forward All Backward

The easiest improvement over the naive schedule of Section 6.4.1 is to separate the minibatches. The previous approach processed all M batches in stage $p = 0$ before passing any activations along. Better is to process each minibatch item individually and pass on activations immediately upon availability. See Figure 9. Any number of minibatches M can be in flight with this strategy, up to memory constraints. This is the GPipe algorithm [40].

With this strategy the final rank does not need to wait as long to start computing: after $\frac{P-1}{P} \times t_f$ it receives its first data, a reduction by a factor of M. The last rank performs all of its compute in time $\frac{M}{P} \times \left(t_f + t_p\right) = t_{\text{ideal}}$, and it takes another $\frac{P-1}{P} \times t_b$ for the previous ranks to finish the backwards pass on the last minibatch. The total time is then

$$\begin{aligned} t_{\text{actual}}^{\text{AFAB}} &= \frac{P-1}{P} \times \left(t_f + t_b\right) + t_{\text{ideal}} \\ &= \frac{P-1}{M} \times t_{\text{ideal}} + t_{\text{ideal}} \end{aligned} \tag{131}$$

yielding

$$\varepsilon_b^{\text{AFAB}} = \frac{1}{1 + \frac{P-1}{M}}, \tag{132}$$

which reduces to (130) when $M = 1$ and goes to unity as $M \longrightarrow \infty$, as it should.

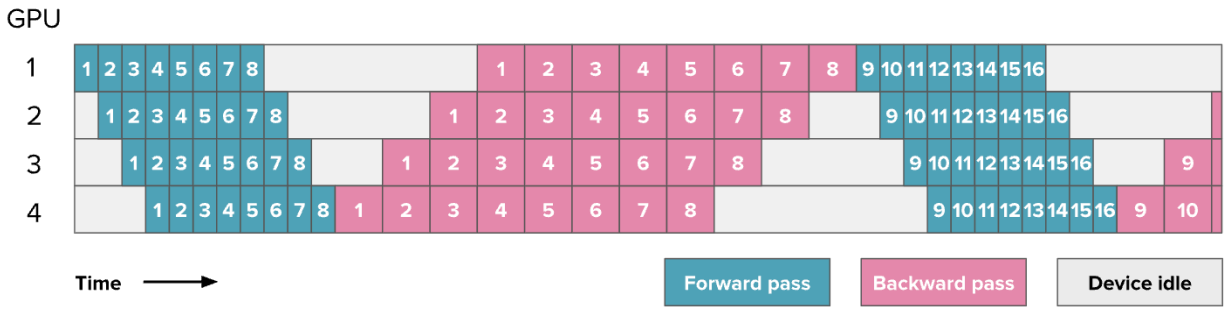


Figure 9: Diagram of the AFAB schedule, from [39].

6.4.3. 1F1B

The next improvement is to not wait around until all M microbatches have completed to start the backwards. This schedule does not reduce the bubble over that of Section 6.4.2, but it allows us to release activation memory sooner. See Figure 10. Arbitrary minibatch size M can be handled with this strategy, but only P minibatches will be in flight at any given time, as explained below. This strategy is used in PipeDream [41].

The final ranks starts the backwards on its first minibatch immediately after completing the forward, and then alternates between forwards and backwards passes. The second-to-last stage, $p = P - 2$ is expected to be ready to process the gradients computed by the final stage upon availability, which limits the total pipeline to have P total minibatches in flight. Once the gradients for a minibatched are processed by the lead $p = 0$ stage, a new set of P minibatches can be added to the queue, with the lead rank alternating between processing new minibatches and processing gradients passed from previous minibatches. Because the backwards iterations tend to be nearly twice as long as the forwards, this strategy can be very effective at closing the bubble during the "steady state" regime.

The time for the final rank to receive its first data is the same as Section 6.4.2, as are the total compute time on the final rank and the time for the backwards on the last minibatch to be calculated. Thus, the bubble efficiency is the same as (132):

$$\varepsilon_b^{\text{1F1B}} = \frac{1}{1 + \frac{P-1}{M}}. \tag{133}$$

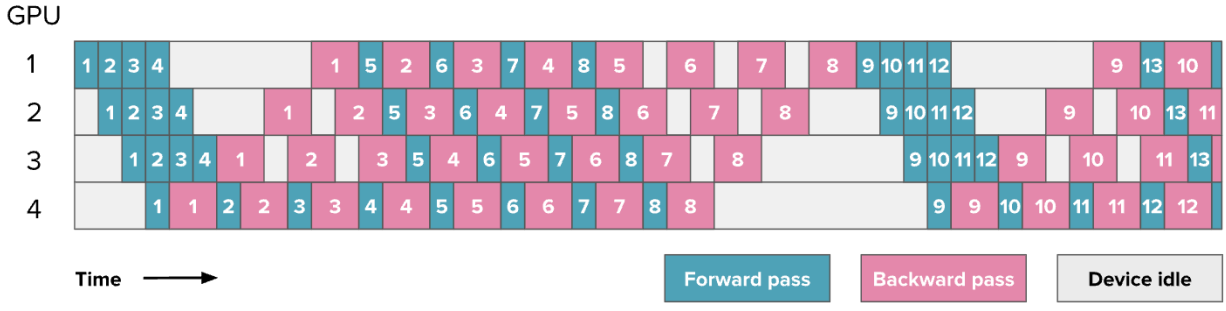


Figure 10: Diagram of the 1F1B schedule, from [39]. Here, the minibatch size is $M = 2P$.

6.4.4. Interleaved Schedules

The 1F1B schedule of Section 6.4.3 did not improve upon the time efficiency of the AFAB schedule of Section 6.4.2. The fixed time cost of the final stage to compute its forwards and backwards passes (t_{ideal}) means that the only way to improve is to get the final GPU work earlier in the pipeline process and/or to shorten up the remaining compute time required once the final GPU finishes its final backwards pass.

One way to achieve this is to change the GPU-to-stage mapping. Instead of a simple linear mapping with across P GPUs and P stages (stages and GPUs have been synonymous thus far), we can break the model up into $v \times P$ stages assigned to the GPUs in round-robin fashion: GPU p gets stages $(p, p+P, \ldots, p+(v-1)P)$. Stages still pass activations around in order, which now cyclically loops over GPUs. See Figure 11, as well as [42], [43].

The advantage is that each GPU now only takes $t_f/(vP)$ time per stage per minibatch and hence the final GPU in the pipeline parallel group only waits time $(P-1) \times t_f/(vP)$ to start computing, an improvement by a factor of v. The savings on the backend are analogous and the efficiency works out to be:

$$\varepsilon_b^{\text{inter}} = \frac{1}{1 + \frac{P-1}{Mv}} \tag{134}$$

The tradeoff here is more communication, which was not accounted for above and limits us from taking $v \longrightarrow \infty$ (in addition to the finiteness of the model). Ignoring these costs momentarily, one important takeaway from this construction is that we can get an efficient $\varepsilon \approx 1$ workload even when there are fewer minibatches than GPUs $M < P$ by taking v large enough, which is not possible for any of the other strategies thus far.

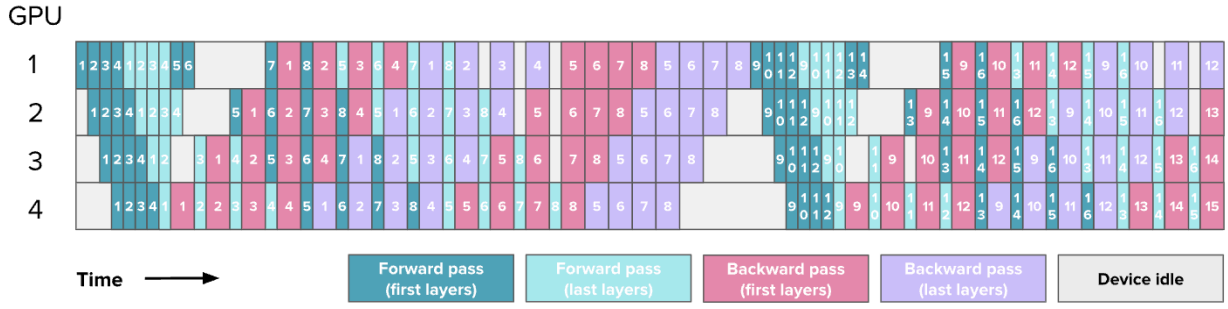


Figure 11: Diagram of the interleaved schedule, from [39]. This is specifically a depth-first schedule, which performs the backwards as soon as possible. A breadth-first schedule would finish all forwards before adding more backwards passes.

6.4.5. Depth-First vs. Breadth-First Scheduling and Data Parallelism

The AFAB and 1F1B strategies only differed in their scheduling order: the former performs all forwards before performing any backwards, while the latter performs the backwards as soon as possible. As mentioned, the latter helps reduce the required activation. But when pipeline parallelism is combined with data parallelism, there AFAB type schedules gain an advantage: the

entire backwards for the late stage layers finishes earlier, allowing us to kick off the data-parallel gradient reductions sooner, allowing for more compute-communication overlap.

These alternatives are more generally referred to as depth-first and breadth-first schedules [43] and apply equally well to the interleaved schedules of Section 6.4.4.

There are also in-between options: rather than running the minimum number of forwards before a backwards (depth-first) or the maximum (breadth-first), it is possible to run schemes between these extremes. Llama3 [44] uses an interleaved schedule with one of these middle strategies.

6.4.6. The Zero-Bubble Strategies

The (aspirationally named) zero-bubble [45] strategies improve upon pipeline schedules by breaking down the backwards pass into those operations which populate gradients on learnable weights and those which compute gradients of other intermediates. The two can be scheduled independently (at the cost of holding activations for longer). See Figure 12.

These zero-bubble strategies do not reduce the warm-up bubble, but rather help reduce the bubble on the backend. They key is that it is only the gradients with respect to intermediates which must be computed in a strict causal order. By prioritizing computing these first, the final rank can compute all of these blocking gradients earlier, allowing earlier ranks to also compute their final gradients earlier, and filling the previously bubble-filled backend time steps of the later ranks with remaining weight gradient computations.

In order to get a formula for the efficiency, assume we are in the maximally advantageous scenario where all activation and gradient compute on the early ranks can be overlapped with the computation for the final rank's weight gradients which were saved for last. This cuts out the entire backend bubble, giving

$$\begin{aligned} t^{\text{zb}}_{\text{actual}} &= \frac{P-1}{P} \times t_f + t_{\text{ideal}} \\ \varepsilon^{\text{zb}}_b &= \frac{1}{1 + \frac{P-1}{M} - \frac{(P-1)t_b}{Pt_{\text{ideal}}}} \end{aligned} \tag{135}$$

For instance, if the rest of the pipeline for the activation's backward passes takes relatively little time, then it can e

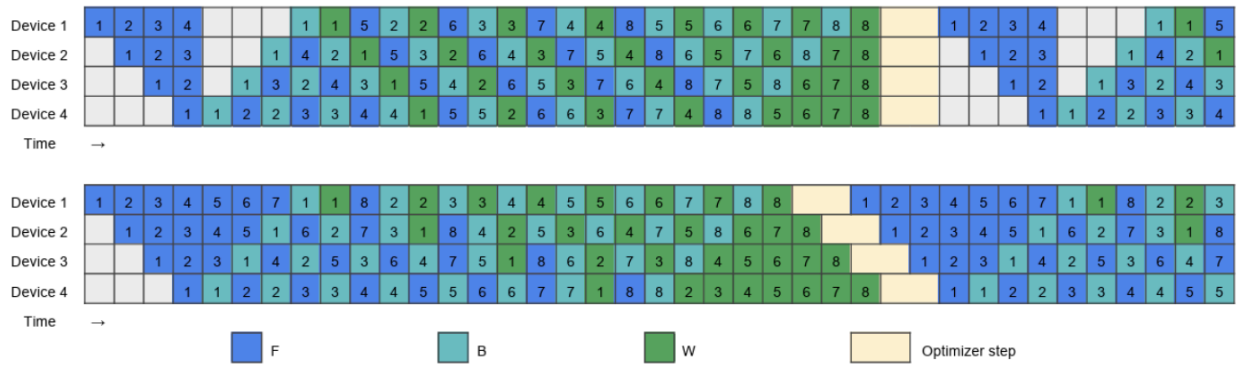


Figure 3: Handcrafted pipeline schedules, top: ZB-H1; bottom: ZB-H2

Figure 12: Diagram of two zero-bubble schedules, from [45] (by way of [39]). In the top diagram, the backwards-for-intermediates steps in light blue (`B` in the legend) finish before the green backwards-for-weights (`W` in the legend), decreasing the wait-times for early stages. See the light-blue boxed marked with an 8. The bottom strategy incorporates a non-standard protocol for the optimizer steps. These two stages are approximately depth-first and breadth-first zero-bubble strategies, respectively.

6.4.7. DualPipe

The warmup-stage bubble is unavoidable, as a consequence of the causal dependencies of the typical model, but it can be mitigated by moving to a double-ended pipeline structure. DualPipe [46] uses this strategy. It isn't described in detail, but seems to be the following. See Figure 13.

Create a double-ended pipeline setup on P ranks (with both P and M even) with rank p assigned two stages: $(p, P-p-1)$. E.g., both the lead and last ranks get the zeroth and final stages of the model. Memory costs are now only reduced by $\frac{P}{2}$, rather than P. Minibatches are injected into the pipeline in pairs, one at rank 0 and the other at the final rank $P-1$. These follow the normal (non-interleaved) pipeline flow, albeit with the batches injected at the final rank passing in the reversed rank order. Ranks $\left(\frac{P}{2}-1, \frac{P}{2}\right)$ are now the ranks to receive work last so the initial wait is reduced to $\left(\frac{P}{2}-1\right) \times t_f/P$. These middle ranks then perform t_{ideal} computation[111] and then need to wait time $\left(\frac{P}{2}-1\right) \times t_b/P$ for the downstream ranks to finish their remaining backwards work. Total time and efficiency:

$$
\begin{aligned}
t_f^{\text{dual}} &= \frac{\frac{P}{2}-1}{P} \times \left(t_f + t_b\right) + t_{\text{ideal}} \\
\varepsilon_b^{\text{dual}} &= \frac{1}{1+\frac{P/2-1}{M}}
\end{aligned}
\tag{136}
$$

This is a greater time efficiency per GPU, but at the cost of doubled memory per GPU.

[111] Since for every mini batch they compute only one of P stages

The more important aspect of DualPipe is how it manages to overlap communications and compute, leveraging both zero-bubble type decompositions and new innovations specific to their architecture; see [46] for more details.



Figure 5 | Example DualPipe scheduling for 8 PP ranks and 20 micro-batches in two directions. The micro-batches in the reverse direction are symmetric to those in the forward direction, so we omit their batch ID for illustration simplicity. Two cells enclosed by a shared black border have mutually overlapped computation and communication.

Figure 13: Diagram of the DualPipe schedule, from [46].

7. Vision

Notes on the usage of Transformers for vision tasks.

7.1. Vision Transformers

The original application of the Transformers architecture [47] divides 2D images into patches of size $P \times P$, e.g. flattening a three-channel i_{xyc} image to shape f_{sd} where $d \in (0, ..., P^2C - 1)$ and the effective sequence length runs over $s \in (0, L^2C/P^2 - 1)$, for an $L \times L$ sized image[112]. A linear projection converts the effective hidden dimension here to match match the model's hidden dimension. These are known as **Patch Embeddings**.

Since there is no notion of causality, no causal mask is needed. A special token is prepended and used to generate the final representations z_{bd} for a batch of images. This can be used for classification, for instance, by adding a classification head. The original training objective was just that: standard classification tasks.

7.2. CLIP

CLIP (Contrastive Language-Image Pre-Training) [48] is a technique for generating semantically meaningful representations of images. The method is not necessarily Transformers specific, but the typical implementations are based on this architecture.

The core of CLIP is its training objective. The dataset consists of image-caption pairs (which are relatively easy to extract; a core motivation), the CLIP processes many such pairs and then tries to predict which images match to which captions. This is thought to inject more semantic meaning into the image embeddings as compared with, say, those generated from the standard classification task.

[112]Example: for a 256×256, three-channel image with a 16×16 patch size, the effective sequence length is 768.

A typical implementation will use separate models for encoding the text and image inputs. The two outputs are t_{bd} and i_{bd} shaped[113], respectively, with batch and hidden dimensions, and are canonically trained so that the similarity score between any two elements is a function of their dot-product.

The original CLIP recipe:

1. Process the text bracketed with and insertions, use a normal Transformer architecture[114], and extract the last output from the token as the text embedding: $i_{bd} = z_{bsd}\big|_{s=-1}$.
2. Process the image with a vision transformer network.
3. Project to a common dimensionality space, if needed.
4. Compute the logits through cosine similarity: $\ell_{bb'} = i_{bd} t_{b'd} / |i_b||t_b|$. These are used to define both possible conditional probabilities[115]:

$$P(i_b \mid t_{b'}) = \frac{e^{\ell_{bb'}}}{\sum_b e^{\ell_{bb'}}}, \quad P(t_{b'} \mid i_b) = \frac{e^{\ell_{bb'}}}{\sum_{b'} e^{\ell_{bb'}}} \tag{137}$$

5. Compute the cross-entropy losses in both directions and average:

$$\mathcal{L} = \frac{1}{2B} \sum_b (\ln P(i_b|t_b) + \ln P(t_b|i_b)). \tag{138}$$

They also add a temperature to the loss, which they also train.

Post-training, the CLIP models can be used in many ways:

1. Using the vision model as a general purpose feature extractor. This is how many vision-language models work: the CLIP image embeddings form part of the VLM inputs.
2. Classification works by comparing the logits for a given image across embedded sentences of the form `This is an image of a <CLASS HERE>.`

8. Mixture of Experts

8.1. Basics

The $\mathcal{O}(D^2)$ FLOPs count due to MLP layers[116] is untenable past a given point: inference and training just take too long. Mixture of Experts[117] (MoE) models address this concern by splitting single MLP layer into a number of "expert" MLP layers and route a subset of the tokens to a

[113] There may also be another linear projection from the actual model outputs to a common space, too. Obviously, this is also necessary if the hidden dimensions of the two models differ.

[114] The original CLIP paper keeps the causal mask.

[115] They differ by what is summed over the in the denominator, i.e., which dimension the is over.

[116] The $\mathcal{O}(S^2)$ scaling of the self-attention layers is also untenable, but MoE only addresses the MLP layers.

[117] The original MoE research came out of Google: see [49], [50] and related work by these authors. An excellent MoE paper with open-source every thing is here [51].

subset of the experts." MoE is a lever for changing the relation between the per-token FLOPs count and the overall parameter count. Example: comparing a dense and a MoE model at similar parameter counts, the expert layer's intermediate dimension is reduced by $\mathcal{O}(N_{\text{ex}})$ (the number of experts) and the FLOPs count is also reduced by this factor. Perhaps unsurprisingly, MoE experts outperform and train faster than their FLOPs equivalent dense models (at the cost of more engineering complexity and a higher memory burden).

The general form of the MoE layer output is

$$z'_{sd} = G_{se}(z_{sd}, ...)E_{esd}(z_{sd}) \tag{139}$$

where $G_{se}(z_{sd}, ...) \in \mathbb{R}^{S\times N_{\text{ex}}}$ is a gating (i.e., weighting) function and $E_{esd}(z_{sd}) \in \mathbb{R}^{N_{\text{ex}}\times S\times D}$ is the usual MLP operation performed by the e-th expert. Many of the entries G_{es} are zero in practice (i.e. it's sparse), and only the computations $E_{esd}(z_{sd})$ corresponding to non-trivial gating values are performed, of course. Different MoE variants are essentially differentiated by the specific form of their weighting function.

8.2. Routing

Choosing which experts process which tokens is crucial, affecting both the downstream model and engineering (i.e. throughput) performance. There are two dominant schemes:

1. **Token Choice**: each token selects a fixed number of experts. G_{se} is sparse over the expert index; see (139);.
2. **Expert Choice**: each expert selects a fixed number of tokens. G_{se} is sparse over the token index; see (139);.

Layered on top of this choice are the details of the routing mechanisms.

8.2.1. Token Choice vs Expert Choice

Token and expert choice both introduce a tensor $W_{de} \in \mathbb{R}^{D\times N_{\text{ex}}}$ which is used to produce a score between each token and expert: $S_{se} = z_{sd}W_{de}$. In each case, we perform a `topk` computation and output a weighted sum of expert outputs: the two methods just differ in the dimension over which the `topk` is performed.

Typical gating functions in the two cases are:

$$\begin{aligned} G^{\text{expert}}_{se}(z_{sd}, W) &= \texttt{Softmax}_s(\texttt{topk}_s(z_{sd}\cdot W_{de})) \\ G^{\text{token}}_{se}(z_{sd}, W) &= \texttt{Softmax}_e(\texttt{topk}_e(z_{sd}\cdot W_{de})), \end{aligned} \tag{140}$$

with `topk` setting any non-top-`k` entries to $-\infty$, in a slight abuse of notation.

There are tradeoffs to each choice:

- Some tokens may not be selected at all in expert choice, but the per-expert load is balanced by construction.
- All tokens gets assigned to an equal number of experts in token choice, but memory constraints of the implementation may force some tokens to get dropped.

8.3. MegaBlocks

The MoE computation maps awkwardly to the typical GPU primitives. Ideally the expert computations in (139) are parallelized as much as possible, but batched matrix multiplies (the closet common primitive) enforces equal token counts per expert, which is overly restrictive.

MegaBlocks [52] introduces the proper sparse kernels to handle general MoE computations without the need to enforce any hard per-expert token limits or introduce unnecessary padding. They call their method dropless MoE (dMoE).

8.4. MoE Variants

A collections of other MoE architecture choices.

8.4.1. Shared Experts

Shared experts forces one particular expert to always be used, with the motivation of having the differentiated expert serve as a common pool of knowledge.

8.5. DeepSeekV3

DeepSeekV3 [46] has various architectural changes, most of which are not MoE specific, but which we document here anyway.

8.5.1. Multi-Head Latent Attention

The idea is to compress all keys and values into a single, compressed latent representation [53].

Rather than creating the $O(SAH)$ keys and values like

$$\begin{aligned} k_{sah} &= W^K_{ahd} z_{sd} \\ v_{sah} &= W^V_{ahd} z_{sd}, \end{aligned} \tag{141}$$

just create a single latent

$$c_{sc} = W^C_{ac} z_{sd} \tag{142}$$

with the size of the $c \in \{0, ..., C-1\}$ dimension obeying $C \ll AH$. The usual keys and values are created by projecting c_{sc} back up to the usual dimensions: $k \sim W^{KC} \cdot c, v \sim W^{VC} \cdot c$. This is an inference-time optimization, since it reduces the effective kv-cache size: we store the smaller c_{sc} tensor, rather than separate and larger k, v tensors. A similar compression is applied during production of the queries[118]

The DeepSeek handling of RoPE is also slightly different. They keys and values produced from the latent tensor c do not have RoPE applied, and instead additional keys and queries which have RoPE applied are concatenated (along the model feature dimensions) with those of the previous paragraph: $k^{\texttt{RoPE}} \sim \texttt{RoPE}(W \cdot z)$ and $q^{\texttt{RoPE}} \sim \texttt{RoPE}(W \cdot c)$, respectively built from the hidden states and latents again for inference-time optimization reasons.

[118] Which seems a poor choice? In [46] it's claimed that this reduces activation memory, but that seems incorrect (projection-matrix memory is slightly reduced).

In the end, the c and $k^{\texttt{RoPE}}$ tensors are what are cached during inference.

9. Inference

9.1. Basics and Problems

The essentials of decoder-only inference is that a given input sequence x_{bs} is turned into a probability distribution p_{bsv} over the vocabulary for what the next token might be. Text is then generated by sampling from p_{bsv} in some way, appending that value to x_{bs} to create a one-token-longer sequence, and then repeating until desired.

There are various problems that naive implementations of the above face:

- Repeated computation from processing the same tokens in the same order repeatedly, at least for some sub-slice of x_{bs}.
- Inherently sequential computation, rather than parallel
- Sub-optimal sampling strategies. Just choosing the most-probably token at each new step, does not guarantee the most-probable overall sequence, for instance.

9.1.1. Generation Strategies

A quick tour of generation strategies. A very readable blog post comparing strategies can be found here.

9.1.1.1. Greedy

The most obvious generation strategy is to take the final, `(B, S, V)`-shaped outputs z_{bsv} and just take the next token to be the most-probable one (for the final position in the sequence): `next_token = z[:, -1].argmax(dim=-1)`.

There are various important, practical considerations which are ignored in the above implementation, including:

- Since we are taking the prediction from the last (`-1`-indexed) element in each sequence, it is crucial that all padding is *left*-padding, so that these final elements are meaningful.
- Models will signal the end of generation by outputting tokenizer-specific codes, and generation must respect these.

See the `generate` method from the `transformers` library for more fully-featured code (which, correspondingly, is not always easy to follow).

9.1.1.2. Simple Sampling: Temperature, Top-k, and Top-p

The next-most-obvious strategy is to choose the next token by drawing from the probability distribution defined by the z_{bsv}. There are various refinements of this idea.

A one-parameter generalization of this strategy introduces a (physics-motivated) **Temperature** which just adjusts the scale of the logits:

```
next_token = torch.multinomial((z[:, -1] / temp).softmax(dim=-1), num_samples=1)
```

assuming z are the final logits. Larger temperature yields a larger variance in the chosen tokens.

With temperature sampling, there is still a non-zero chance of choosing an extremely improbable token, which is undesirable if you do not trust the tails of the distribution. Two common truncation strategies which guard against this:

- **Top-k**: Only choose from the top-k most-probable examples (re-normalizing the probabilities across those k samples)
- **Top-p**: Only choose from the top-however-many most-probable examples whose probabilities sum to p (again re-normalizing probabilities). This is also some times called **nucleus sampling**.

9.1.1.3. Beam Search

Choosing, say, the most-probable next-token at each step is not guaranteed to yield the most probable *sequence* of tokens. So, **Beam Search** explores multiple sequences, using different branching strategies, and the probabilities of the various beam sequences can be compared at the end. Important note: generating the most-probable text is not necessarily equal to the most human-like text [54].

9.1.1.4. Speculative Decoding

Speculative decoding [55] is an excellent idea: use a cheaper "draft" model to perform the slow, iterative generation steps and check its work with the full model. Using a detailed-balance-like construction, it can be guaranteed that this speculative decoding generation strategy creates text drawn from the same distribution as the full model.

Informally, the algorithm is:

1. Generate γ tokens with the draft model, whose distribution is $q\left(x_t|x_{\text{prefix}}\right)$, $t \in (0, ..., \gamma - 1)$. Write the generated tokens as z_t.
2. Pass the prefix and all γ generated tokens z_t through the full model, which computes probabilities via its distribution $p\left(x_t|x_{\text{prefix}}\right)$.
3. For every generated token z_t, accept it unconditionally if $q\left(z_t|\ x_{\text{prefix}}\right) \leq p\left(x_t|\ z_{\text{prefix}}\right)$. If $q\left(z_t|\ x_{\text{prefix}}\right) > p\left(z_t|\ x_{\text{prefix}}\right)$, instead accept the token with only probability[119] $\frac{p}{q}$.
4. If only the first $n < \gamma$ tokens are accepted, generate token $n + 1$ from a modified distribution $p'\left(x_t|\ x_{\text{prefix}}\right) = F[p(x), q(x)]$ built from the two model predictions and chosen (as will be shown) such that the entire algorithm generates the correct distribution. $n + 1$ tokens are created in this case[120].
5. If all of the γ tokens are accepted, generate token $\gamma + 1$ from the full model's outputs.

[119] This choice is not fundamental; it just makes following expressions nicer.

[120] We cannot generate more tokens because drawing from p' effectively changes the prefix that the full model should use.

Proof of correctness and the derivation of $p'(x)$: let $Q\big(x_t|\ x_{\text{prefix}}\big)$ be the distribution described above. Then this can be broken down according to conditioning on the draft token and whether or not the draft token was accepted. Dropping the prefix condition for brevity and A and R stand for rejected and accepted, respectively, we have

$$\begin{aligned} Q(x_t) &= \sum_{z_t} Q(x_t|z_t, A)P(A|z_t)q(z_t) + Q(x_t|z_t, R)P(R|z_t)q(z_t) \\ &= \sum_{z_t} \delta_{x_t,z_t} \times \min\left(1, \frac{p(z_t)}{q(z_t)}\right) \times q(z_t) + p'(x_t)\left(1 - \min\left(1, \frac{p(z_t)}{q(z_t)}\right)\right) \times q(z_t) \\ &= \min(q(x_t), p(x_t)) + p'(x_t) \sum_{z_t} \left(1 - \min\left(1, \frac{p(z_t)}{q(z_t)}\right)\right) \times q(z_t) \end{aligned} \tag{143}$$

The sum is just some constant (denoted by $1 - \beta$ in the paper, which should really have a t subscript) and so choosing

$$p'(x_t|\ x_{\text{prefix}}) \equiv (p(x_t|\ x_{\text{prefix}}) - \min(\frac{q\big(x_t|\ x_{\text{prefix}}), p\big(x_t|\ x_{\text{prefix}}\big)\big)}{1 - \beta} \tag{144}$$

achieves the goal of getting $Q(x_t|\ x_{\text{prefix}}) = p(x_t|\ x_{\text{prefix}})$. It can be verified that this distribution is properly normalized.

An approximate analysis for choosing the optimal value of γ can be found in the paper.

9.1.2. The Bare Minimum and the kv-Cache

There are two separate stages during generation. First, an original, to-be-continued series of prompts x_{bs} can be processed in parallel to both generate the first prediction and populate any intermediate values we may want to cache for later. We follow [56] and call this the **prefill** stage. For this procedure, we require the entire x_{bs} tensor.

In the second, iterative part of generation (the **decode** stage) we have now appended one-or-more tokens to the sequence and we again want the next prediction, i.e. `z[:, -1, :]` for the last-layer outputs z_{bsd}. In this stage, we can avoid re-processing the entire x_{bs} tensor and get away with only processing the final, newly added token, *if* we are clever and cache old results.

The important pieces occur in the `CausalAttention` layer, as that's the only location in which the sequence index is not completely parallelized across operations. Referring back to Section 1.1.3, given the input z_{bsd} of the `CausalAttention` layer, the re-weighted value vectors[121] $w^a_{bss'd}v^a_{bs'f}$ are the key objects which determine the next-token-prediction, which only depends on the $s = -1$ index values. Therefore, we can cut out many steps and the minimum requirements are:

- Only the attention weights $w^a_{bss'd}$ with $s = -1$ are needed
- The only query values q^a_{bsd} needed to get the above are those with $s = -1$

[121] Summed over s', but concatenating the different a values over the f dimension.

- Every component of the key and value vectors k^a_{bsd}, v^a_{bsd} is needed, but because of the causal mask, all components except for the last in the sequence dimension ($s \neq -1$) are the same as they were in the last iteration, up to a shift by one position.

So, we are led to the concept of the **kv-cache** in which we cache old key and query vectors for generation. The cache represents a tradeoff: fewer FLOPs are needed for inference, but the memory costs are potentially enormous, since the size of the cache grows with batch size and sequence length:

$$M_{\text{kv-cache}} = \frac{2pBDLS}{T \times \text{GQA}}, \tag{145}$$

in the general case with tensor-parallelism and grouped-query attention Section 1.2.4. This can easily be larger than the memory costs of the model parameters: $M^{\text{inference}}_{\text{params}} \sim pN_{\text{params}} L \frac{D^2}{T}$ (dropping $\mathcal{O}(1)$ factors), so that the cache takes up more memory when $\frac{BS}{\text{GQA}} \gtrsim D$, i.e. when the total number of token exceeds the hidden dimension. Using the kv-cache eliminates a would-be $\mathcal{O}(S^2)$ factor in the FLOPs needed to compute a new token, reducing it to linear-in-S dependence everywhere.

9.1.3. Basic Memory, FLOPs, Communication, and Latency

The essentials of inference-time math, much of it based on [57].

9.1.3.1. Prefill Timing

The prefill FLOPs required for a `(B, S)`-shaped input is $2BSN_{\text{params}}$, the same as the forwards-pass FLOPs compute from Section 3.2 (assuming $S \lesssim D$). Memory costs primarily come from the parameters themselves: $M^{\text{naive}}_{\text{infer}} = pN_{\text{params}}$. Per the analysis of Appendix D.4, naive inference is generally compute-bound and so the per-token-latency is approximately[122] $2BS\frac{N_{\text{params}}}{\lambda_{\text{FLOP/s}}}$ where the FLOPs bandwidth in the denominator is again defined in Appendix D.4. If using a non-trivial tensor-parallel degree T, as is usually done, the time is approximately reduced by a factor of $\sim \frac{1}{T}$; we will expand upon this later.

9.1.3.2. Decode Timing

Decode, i.e. single-token generation using the kv-cache, is generally memory bound. We can confirm this fact by looking at the rough FLOPs to compute ratio at the model level. Since we are only computing a single-new token per batch, the total compute per forwards pass is roughly $2BN_{\text{params}}$, i.e. it is the $S \to 1$ limit of the prefill FLOPs. But we unfortunately still need to pay the costs of reading in the entire model weights, making the arithmethic intensity approximately[123]

[122] Assuming we do the naive thing here and generate the next token in a similarly naive way, shifting over the context window.

[123] This neglects the kv-cache memory reads, which are significant when $\frac{BS}{\text{GQA}} \gtrsim D$ as discussed above in Section 9.1.2. Note that the FLOPs estimate should also be altered if $S \gtrsim 10D$, as attention-FLOPs start to dominate in this regime. We will largely assume that we are not in the regime where either of these conditions hold. Realistic workloads also contain a mix of requests with different characteristics, making this entire analysis fairly toy-like.

$$\sim \frac{2BN_{\text{params}}}{2pN_{\text{params}}} \sim \mathcal{O}(B) \ . \tag{146}$$

From Appendix D.3, we would need $B \gtrsim \mathcal{O}(100)$ to escape the memory bound regime, which we will not typically be able to achieve in practice. Both the memory and compute estimates get divided by T when using tensor-parallelism, which therefore does not qualitatively change the story.

The decode step time is then $\sim \mathcal{O}\left(\frac{2pN_{\text{params}}}{\lambda_{\text{mem}}}\right)$, independent of the batch size.

9.1.3.3. Tensor Parallelism

Tensor parallelism requires a single all-reduce in the forwards pass, which takes time $\sim \frac{2pBDS(T-1)}{T\lambda_{\text{comms}}}$ per operation. This is done twice per layer, once for the MLP and once for the self-attention block, making the total time $\sim \frac{4pBDLS(T-1)}{T\lambda_{\text{comms}}} \sim \mathcal{O}\left(\frac{BSN_{\text{params}}}{D\lambda_{\text{comms}}}\right)$. Meanwhile, the total prefill compute time is $\mathcal{O}\left(\frac{BSN_{\text{params}}}{T\lambda_{\text{compute}}}\right)$ and so the communication to compute ratio is $\mathcal{O}\left(\frac{T\lambda_{\text{compute}}}{D\lambda_{\text{comms}}}\right)$ with rates in FLOP/s and B/s. For H100s, say, $\lambda_{\text{compute}} \sim 500$ TFLOP/s while $\lambda_{\text{comms}} \sim 300$ GiB/s.

9.2. Optimizations

9.2.1. PagedAttention

Naive inference would allocate a large, static kv-cache which can contain up to to the maximum sequence-length of tokens the model is allowed to generate on. This is obviously wasteful, causing severe memory fragmentation and over-allocating.

vLLM [58] solved this by creating PagedAttention, which is just allocating the physical kv-cache by partitioning a large segment of memory into small pages and maintaining a mapping from logical entries to physical pages. These small pages are filled up one at a time. PagedAttention also gives other familiar benefits from this indirection, such as more efficient sharing of memory across prompts with common prefixes [59].

A Conventions and Notation

We loosely follow the conventions of [5]. Common parameters:

- A: number of attention heads
- B: microbatch size
- C: compute (FLOPs)
- D: the hidden dimension size
- E: expansion factor for MLP layer (usually $E = 4$)
- H: the head dimension size. Equal to D/A for vanilla multi-head attention.
- K: the block size (maximum sequence length[124])

- L: number of transformer layers
- M: number of minibatches for pipeline parallelism
- N_{params}: total number of model parameters
- N_{ex}: number of experts for MoE models.
- P: pipeline parallel size
- S: input sequence length
- T: tensor parallel size
- V: vocabulary size
- t: various timescales
- p: the precision of the elements of a tensor in bytes
- λ: various rates, e.g. λ_{mem} is memory bandwidth

Where it makes sense, we try to use the lower-case versions of these characters to denote the corresponding indices on various tensors. For instance, an input tensor with the above batch size, sequence length, and vocabulary size would be written as x_{bsv}, with $b \in (0, ..., B-1)$, $s \in (0, ..., S-1)$, and $v \in (0, ..., V-1)$ in math notation, or as `x[b, s, v]` in code.

Typical transformers belong to the regime

$$V \gg D, S \gg L, A \gg P, T \,. \tag{147}$$

For instance, GPT-2 and GPT-3 [2], [3] have $V\mathcal{O}(10^4)$, $S, L\mathcal{O}(10^3)$, $L, A\mathcal{O}(10^2)$. We will often assume also assume that[125] $S \lesssim D$ or the weaker[126] $BS \lesssim D$.

As indicated above, we use zero-indexing. We also use `python` code throughout[127] and write all ML code using standard `torch` syntax. To avoid needing to come up with new symbols in math expressions we will often use expressions like $x \leftarrow f(x)$ to refer to performing a computation on some argument (x) and assigning the result right back to the variable x again.

Physicists often joke (half-seriously) that Einstein's greatest contribution to physics was his summation notation in which index-sums are implied by the presence of repeated indices and summation symbols are entirely omitted. For instance, the dot product between two vectors would be written as

[124] In the absence of methods such as ALiBi [60] can be used to extend the sequence length at inference time.

[125] This condition ensures that the $\mathcal{O}(S^2)$ FLOPs cost from self-attention is negligible compared to $\mathcal{O}(D^2)$ contributions from other matrix multiplies. It should be noted that in Summer 2023 we are steadily pushing into the regime where this condition does *not* hold.

[126] This condition ensures that the cost of reading the $\mathcal{O}(D^2)$ weights is more than the cost of reading in the $\mathcal{O}(BSD)$ entries of the intermediate representations.

[127] Written in a style conducive to latex, e.g. no type-hints and clarity prioritized over optimization.

$$\vec{x} \cdot \vec{y} = \sum_i x_i y_i \equiv x_i y_i \tag{148}$$

We use similar notation which is further adapted to the common element-wise deep-learning operations. The general rule is that if a repeated index appears on one side of an equation, but not the other, then a sum is implied, but if the same index appears on both sides, then it's an element-wise operation. The Hadamard-product between two matrices A and B is just

$$C_{ij} = A_{ij} B_{ij} \,. \tag{149}$$

Einstein notation also has implementations available for `torch`: see this blog post on `einsum` or the `einops` package. We strive to write all learnable weights in upper case.

In particular, we use `einops` notation for concatenation and splitting: $A_c = A_{(de)} = B_{de}$[128]. We will some times use a bar to indicate tensors which are derived from other tensors through such splitting operations, usually in the context of tensor-sharding where devices only locally hold some shard of the tensor. In this context, only some of the dimensions will be sharded across devices, and we may also put a bar over the corresponding sharded index. For instance, consider a two-dimensional tensor M_{ab} of shape `M.shape = (A, B)`: sharding this tensor across two devices across the final index results in a tensor $\bar{M}_{a\bar{b}}$ which is of shape `M_bar.shape=(A, B/2)` on each device. As here, we will some times use bars to denote indices which are sharded over different devices.

We also put explicit indices on operators such as `Softmax` to help clarify the relevant dimension, e.g. we would write the softmax operation over the b-index of some batched tensor $x_{bvd\ldots}$ as

$$s_{bvd\ldots} = \frac{e^{x_{bvd\ldots}}}{\sum_{v=0}^{v=V-1} e^{x_{bvd\ldots}}} \equiv \texttt{Softmax}_v \, x_{bvd\ldots}, \tag{150}$$

indicating that the sum over the singled-out v-index is gives unity.

B Collective Communications

A quick refresher on common distributed communication primitives. Consider R ranks with tensor data $x^{(r)}$ of some arbitrary shape `x.shape`, which takes up M bytes of memory, where r labels the worker and any indices on the data are suppressed. For collectives which perform an operation over a specific dimension, the `torch` convention is that it operates over . The $r = 0$ worker is arbitrarily denoted the *chief*. Some operations are easiest to describe by forming the logical super-tensor `X=torch.stach([x0, x1, ...], dim=0)` of shape `X.shape=Size(R, ...)` such that the tensor on rank `r` is `x=X[r]`. Then, the primitive operations are:

- `Broadcast`: all workers receive the chief's data, $x^{(0)}$.

[128] The indexing is all row-major: if A_i is I-dimensional, $i \in (0, \ldots, I-1)$, then if we split this index as $A_i = A_{(jk)} \equiv \bar{A}_{jk}$, then the indices j, k will range over $j \in (0, \ldots, J)$, $k \in (0, \ldots, K)$ with $I = J \times K$ and where numerically $i = j \times K + k$. More complex cases follow by induction.

- `Gather`: all workers communicate their data x_n to the chief in a concatenated array $[x^0, x^1, ..., x^{R-1}]$. E.g., the chief gets `x_out = X.reshape(R*X.shape[1], X.shape[2:])`.
- `Reduce`: data is `Gather`-ed to the chief, which then performs some operation (`sum`, `max`, `concatenate`, etc.) producing a new tensor x' on the chief worker. E.g., for `sum` the chief gets `x_out = X.sum(dim=0)`.
- `ReduceScatter`: a reducing operation (e.g. `sum`) is applied to the $x^{(r)}$ to produce a x' of the same shape (e.g. $x' = \sum x^{(r)}$) and each worker only receives a $1/R$ slice (and hence M/R byte) of the result[129]. A ring implementation sends $M \times \frac{R-1}{R}$ bytes over each link in the ring. E.g., for `sum` rank `r` gets output `x_out = X.sum(dim=0).tensor_split(R, dim=0)[r]`.
- `AllGather`: all data $x^{(r)}$ is communicated to all workers; each worker ends up with the array $[x^0, x^1, ..., x^{R-1}]$. Functionally equivalent to a `Gather` followed by `Broadcast`. A ring implementation sends $M \times (R-1)$ bytes over each link in the ring. E.g., all ranks get `x_out = X.reshape(R*X.shape[1], X.shape[2:])`.
- `AllReduce`: all workers receive the same tensor x' produced by operating on the $x^{(r)}$ with `sum`, `mean`, etc. Functionally equivalent to a `Reduce` followed by `Broadcast`, or a `ReduceScatter` followed by an `AllGather` (the more efficient choice[130]). In the latter case, the total cost is $2M \times \frac{R-1}{R}$, due to `AllReduce`-ing the initial M-sized data, and then `AllGather`-ing the M/R-sized reductions. E.g., for `sum` all ranks get `x_out = X.sum(dim=0)`.
- `Scatter`: One worker gives shards of a tensor to all workers. If the worker is scattering tensor T_x over the given index, a `Scatter` effectively shards this as $T_x \to T_{(\bar{r}y)}$, each worker getting a $\bar{r}$-shard. If `x` is the chief's data, rank `r` receives `x_out = x.tensor_split(R, dim=0)[r]`.
- `AllToAll`: Nearly arbitrary communication pattern. Typical implementation: each worker has a tensor $T_{n...}$ with a leading dimension of size N and specifies a rank R tensor of counts, c_r, with $\sum_r c_r = N$ determining what slice of $T_{n...}$ to send to rank r. E.g. the slice of the first c_0 along the n dimension is sent to rank 0, the next c_1 to rank 1, etc.

C Device Meshes

Complex sharding schemes are often expressed in terms of device meshes. A `device_mesh` describes the various communication groups that each of the `R` ranks involved in the compute belongs to. In the usual case, `device_mesh` has `R` elements in it, one for each rank in the group, e.g.

[129]Note that `AllGather` and `ReduceScatter` are morally conjugate to each other. In the former, each worker ends up with R times as much data as they started with, while in `ReduceScatter` they end up with $1/R$ of their initial data. One is nearly a time-reversed version of the other, which is a way of remembering that they have the came communication cost. They also compose to produce an output of the same initial size, as in `AllReduce`.

[130]The former strategy scales linearly with the number of worker, while the latter strategy underlies "ring" `AllReduce` which is (nearly) independent of the number of workers: if each worker carries data of size D which is to be `AllReduce`-d, a total of $\frac{2(R-1)D}{R}$ elements need to be passed around. See this blog post for a nice visualization or [61] for a relevant paper.

it contains the values in `range(R)`. If there are `g` independent communication groups in the mesh, then `device_mesh` will be `g`-dimensional , where the sizes of each dimension (`device_mesh.shape = (g_0, g_1, ...)`) divides `R`.

C.1 Basic Example

An example is helpful: given two 8-gpu nodes with FSDP within a node and pipeline parallelism across nodes, the `device_mesh` will typically be `(2, 8)`. Of course, `(8, 2)` is also valid, but the typical convention is that the rightmost dimension is contiguous in ranks (row-major, morally speaking) and hence rightmost dimension has the best locality[131], which is what you want for the costly FSDP comms. Explicitly:

$$\texttt{device_mesh} = \begin{bmatrix} 0 & 1 & 2 & 3 & 4 & 5 & 6 & 7 \\ 8 & 9 & 10 & 11 & 12 & 13 & 14 & 15 \end{bmatrix} \tag{151}$$

C.2 Slicing

We can also discuss slices of a general `device_mesh`, which return the set of ranks which belong to the communication group corresponding to the given slice dimension. We follow the `torch` conventions. Slicing is useful when a given tensor only needs to be reduced over a subset of the ranks, for example. Slicing can be a confusing topic since:

1. `device_mesh` slicing is not the same as tensor slicing
2. The result of a slice is particular to there rank performing the slice.

Assume that we give a name to each dimension of the mesh: say the `i`-th dimension of the mesh has size `g_i` and name `'n_i'`. The slice of the mesh over dimensions `('n_i', 'n_j', ...)` produces a new mesh of shape`(g_i, g_j, ...)` with `g_i * g_j * ...` elements. On rank `r`, this slice contains the ranks belonging to the communication groups `('n_i', 'n_j', ...)` which rank `r` *also belongs to*, which is why the slice result is different on different ranks.

More precisely, for each mesh axis there is a single index value for which the corresponding *tensor* slice of the mesh contains a given rank `r`, and the `device_mesh` slice over a set of named axes corresponds to the submesh where all dimensions *not* in this named set are restricted to these particular index values. A mouthful.

For instance, the mesh of the previous section only contains rank `r=11` for the single-axis *tensor* slices `device_mesh[1, :]` and `device_mesh[:, 3]` and on this rank the corresponding named device mesh slices are related to the tensor slices as:

$$\begin{aligned} \texttt{device_mesh}['\texttt{pp}']|_{r=11} &= \texttt{device_mesh}[:, 3] = [3, 11] \\ \texttt{device_mesh}['\texttt{fsdp}']|_{r=11} &= \texttt{device_mesh}[1, :] = [8, 9, 10, 11, 12, 13, 14, 15]. \end{aligned} \tag{152}$$

A contrived 3D example: PP+CP+FSDP on a 8-gpu node, in that order. The mesh looks like

$$\texttt{device_mesh} = [[[0, 1], [2, 3]], [[4, 5], [6, 7]]]. \tag{153}$$

[131] That is, FSDP communications within this dimension stay within-node, for this setup.

On rank `r=0`, the 2D CP+FSDP named slice[132] is related to the tensor slice via

$$\texttt{device_mesh['cp', 'fsdp']}|_{r=0} = \begin{bmatrix} 0 & 1 \\ 2 & 3 \end{bmatrix} = \texttt{device_mesh}[0, :, :] \tag{154}$$

and the 1D FSDP slice is

$$\texttt{device_mesh['fsdp']}|_{r=0} = [0, 1] = \texttt{device_mesh}[0, 0, :]. \tag{155}$$

D Hardware

Basic information about relevant hardware considerations. Much of the following is from the NVIDIA docs.

D.1 NVIDIA GPU Architecture

NVIDIA GPUs consist of some amount of relatively-slow off-chip DRAM memory[133], relatively-fast on-chip SRAM, and a number of **streaming multiprocessors** (SMs) which perform the parallel computations. Inside more-recent GPUs, the SMs carry both "CUDA cores" and "Tensor cores", where the latter are used for matrix-mulitiplies and the former for every thing else.

A few numbers of primary importance:

- The rate at which data can be transferred from DRAM to SRAM (λ_{mem})
- The number of FLOP/s, which is more fundamentally computed by multiplying the number of SMs by the FLOPS/cycle of each SM for the specific operation under consideration (see the NVIDIA docs) by the clock rate: $N_{\text{SM}} \cdot \lambda_{\text{FLOPs/cycle}} \cdot \lambda_{\text{clock}}$

The terminology and structure of the memory hierarchy is also important to understand. Types of memory, from slowest to fastest:

- **Global** memory is the slow, but plentiful, off-chip DRAM. It is the type of memory typically used as kernel arguments
- **Constant** memory is read only and accessible by all threads in a given block. The size of arrays in constant memory must be known at compile time
- **Local Memory** is similarly slow to global memory, but more plentiful than register memory, and privately to individual threads and is allocated from within a kernel. When registers run out, local memory fills the gap
- **Shared** memory is shared between all threads in a given block. Shared memory is effectively a user-controlled cache. The size of arrays in shared memory must be known at compile time

[132] As of September 2025, `torch` also allows you to define a flattened version of a slice, e.g. `cp_fsdp_mesh` = `device_mesh['cp', 'fsdp']._flatten('cp_fsdp')`, which is just a 1D mesh containing all ranks in the slice the present rank.

[133] This is the number usually reported when discussing a given GPU, e.g. 80GiB for the top-of-the-line A100

- **Registers** hold scalar values and small tensors whose values are known at compile time. They are local to each thread and they are plentiful since each thread needs its own set of registers: 65,536 = 2^{16} registers per SM an A100.

An excellent video overview of CUDA and NVIDIA GPU architecture which covers some of the above is here.

D.2 CUDA Programming Model

The CUDA programming model uses a hierarchy of concepts:

- **Threads** are the fundamental unit of execution[134] which each run the same CUDA **Kernel**, or function, on different data inputs in parallel. Threads within the same block (below) may share resources, like memory, and may communicate with each other. Individual threads are indexed through the **threadIdx** variable, which has attributes with in and similar.
- Threads (and hence warps) are organized into 3D **blocks**. The size and indices of the blocks can be accessed through the and variables, respectively, with in . total threads run in a block.
- Blocks are organized into 3D **groups**. The size of the gird dimensions can be accessed through the variable, with similar attributes to the above.
  total blocks run in a grid.

The number of threads which can be launched in a given block is hardware limited; A100 80GiB GPUs can run up to 1024 threads in a SM at a time (32 blocks with 32 threads each), for instance. Hence, block and grid sizes need to be adjusted to match the problem size. There are also important memory access considerations here. The 1024 threads which can be launched can also read sequentially from memory and efficient usage implies that choosing the block size such that we are doing these reads as often as possible is ideal.

D.3 NVIDIA GPU Stats

Summary of some relevant NVIDIA GPU statistics:

GPU	Memory	$\lambda_{\text{FLOP/s}}$	λ_{mem}	λ_{math}	λ_{comms}
B200	80GiB	2250 TFLOP/s	8 TiB/s	281 FLOPS/B	900 GiB/s
H100	80GiB	989 TFLOP/s	3.35 TiB/s	295 FLOPS/B	450 GiB/s
A100	80GiB	312 TFLOP/s	2.0 TiB/s	156 FLOPS/B	300 GiB/s
V100	32GiB	130 TFLOP/s	1.1 TiB/s	118 FLOPS/B	16 GiB/s

where

- $\lambda_{\text{FLOP/s}}$ is flops bandwidth (for `(b)float16` multiply-accumulate ops)
- λ_{mem} is memory bandwidth

[134] Threads are always physically launched in **Warps** which consist of 32 threads.

- $\lambda_{\text{math}} = \frac{\lambda_{\text{FLOP/s}}}{\lambda_{\text{mem}}}$ is math bandwidth
- λ_{comms} is one-way communication bandwidth

A useful approximate conversion rate is that $1\ \text{TFLOP/s} \approx 100\ \text{PFLOP/day}$.

Important practical note: the $\lambda_{\text{FLOP/s}}$ numbers should be taken as aspirational. Out-of-the box, `bfloat16` matrix-multiplies in `torch` with well-chosen dimensions tops out around ~ 250 FLOPS/s

D.4 Compute-bound vs Memory-bound

If your matrix-multiplies are not sufficiently large on, you are wasting resources [62]. The relevant parameters which determine sufficiency are $\lambda_{\text{FLOP/s}}$ and λ_{mem}, the FLOPs and memory bandwidth, respectively. The ratio $\lambda_{\text{math}} \equiv \frac{\lambda_{\text{FLOP/s}}}{\lambda_{\text{mem}}}$ determines how many FLOPS you must perform for each byte loaded from memory; see Appendix D.3. If your computations have a FLOPs/B ratio which is larger than λ_{math}, then you are compute-bound (which is good, as you're maximizing compute), and otherwise you are memory(-bandwidth)-bound (which is bad, since your compute capabilities are idling). The FLOPs/B ratio of your computation is some times called the **compute intensity** or **arithmetic intensity**. When compute bound, a process takes time $\frac{F}{\lambda_{\text{FLOP/s}}}$, while memory-bound processes take time[135] $\frac{M}{\lambda_{\text{mem}}}$.

D.4.1 Matrix-Multiplications vs. Element-wise Operations

For instance, to multiply a `(B, S, D)`-shaped tensor z_{bsd} by a `(D, D)`-shaped weight-matrix $W_{dd'}$, $p(BDS + D^2)$ bytes must be transferred from DRAM to SRAM at a rate λ_{mem}, after which we perform $2BSD^2$ FLOPs, and write the `(B, S, D)`-shaped result back to DRAM again, for a ratio of

$$\frac{1}{p}\frac{BDS}{2BS + D}(\text{FLOPs/B}). \tag{156}$$

We want to compare this against λ_{math}, which from Appendix D.3 we take to be $\mathcal{O}(100\ \text{FLOPs/B})$, and plugging in any realistic numbers, shows that such matrix-multiplies are essentially always compute-bound. Compare this to the case of some element-wise operation applied to the same z_{bsd} tensor whose FLOPs requirements are $\sim C \times BDS$ for some constant-factor $C \ll S, D$. Then, then FLOPS-to-bytes ratio is $\sim \frac{C}{p}$, which is *always* memory-bound for realistic values of C. The moral is to try and maximize the number of matrix-multiplies and remove as many element-wise operations that you can get away with.

D.4.2 Training vs. Inference

Finally, we note that the above has implications for the Transformers architecture as a whole, and in particular it highlights the difficulties in efficient inference. Under the assumptions of Section 3.2, $\mathcal{O}(BSN_{\text{params}})$ total FLOPs needed during training, while the number of bytes

[135] Note that the time is not additive, e.g. compute-bound tasks do not take time $\frac{F}{\lambda_{\text{FLOP/s}}} + \frac{M}{\lambda_{\text{mem}}}$ because they are not sequential: compute and memory-communications can be concurrent.

loaded from and written to memory are $\mathcal{O}(BDLS + N_{\text{params}})\mathcal{O}\left(\frac{BSN_{\text{params}}}{D} + N_{\text{params}}\right)$ which is $\mathcal{O}(N_{\text{params}})$ for not-super-long sequence lengths. The arithmetic intensity is therefore $\mathcal{O}(BS)$ and so training is compute-bound in any usual scenario, even at small $B\mathcal{O}(1)$ batch sizes (as long as individual operations in the network don't suffer from outlandish memory-boundedness). The problem during inference is that (if using the kv-cache; see Section 9.1.2) we only need to process a *single* token at a time and so $S \to 1$ in the numerator in the preceding, while the denominator is also weighed down by the kv-cache in the attention layers.

In more detail, the `MLP` layers just process $S = 1$ length tensors during generation, but are insensitive to the kv-cache, so their intensity comes from just setting $S = 1$ in the above,

$$\sim \frac{BD}{B + D}\,, \tag{157}$$

dropping $\mathcal{O}(1)$ factors now, while the attention layers have a ratio of the form

$$\sim \frac{BDS + BD^2}{BD + D^2 + BDS}\,, \tag{158}$$

where the last term in the denominator is due to the cache. Now at small $B\mathcal{O}(1)$ batch sizes, both intensities reduce to $\mathcal{O}(B)$, which is insufficient to be compute-bound. In the large $B \gtrsim D/S$ limit, they at least become $\mathcal{O}(D)$ and $\mathcal{O}\left(1 + \frac{D}{S}\right)$, respectively, which may be enough to be compute-bound, but it's hard to even get into this regime. Note, the importance of the ratio D/S. The hidden dimension fixes the context length scale at which inference can never be compute-bound, in the absence of additional tricks not considered here[136].

D.5 Intra- and Inter-Node Communication

For intra-node communication, GPUs are connected by either PCIe or NVLink, generally.

- NVLink interconnects are continually updated and achieve speeds of $\lambda_{\text{comm}}^{\text{intra}}$300 GiB/s.

For inter-node communication, nodes are often connected by:

- InfiniBand apparently also achieves speeds $\lambda_{\text{comm}}^{\text{intra}}$100 GiB/s? Haven't found a clear reference. But in any case, the bandwidth is divided amongst the GPUs in the node, leading to a reduction by ~ 8.

E Batch Size, Compute, and Training Time

The amount of compute directly determines the training time, but not all ways of spending compute are equivalent. We follow the discussion in [63] which gives a rule of thumb for determining the optimal batch size which is some times used in practice. The basic point is that all of the optimization steps take the gradient $\boldsymbol{g}$ as an input, and since the gradient is the average over

[136] One such trick: the multi-query attention of Section 1.2.3 improves every thing a factor of A: the large batch regime is $B \gtrsim \frac{D}{AS}$ and the intensity ratio becomes $\mathcal{O}\left(1 + \frac{D}{AS}\right)$. An analysis equivalent to the one performed here can be found in the original paper [9].

randomly selected datapoints, steps are more precise as the batch size increases (with diminishing returns, past a certain point, but the computational cost also rises with batch size, and a balance between the two concerns should be struck.

Consider vanilla SGD and study how the training loss changes with each step. We randomly sample B datapoints $x \in \mathcal{D}$ from the dataset through some i.i.d. process[137]. Each corresponding gradient $\boldsymbol{g}(x) = \partial_w \mathcal{L}(w, x)$ is itself a random variable whose average is the true gradient across the entire dataset $|\boldsymbol{g}|$ and we take the variance to be

$$\mathrm{Var}[\boldsymbol{g}(x), \boldsymbol{g}(x')] = \Sigma \tag{159}$$

for some matrix Σ with (supressed) indices spanning the space of model weights. Taking instead the mean of a sum of such estimates, $\boldsymbol{g}_B \equiv \frac{1}{B}\sum_{x\in\mathcal{B}} \boldsymbol{g}(x)$, the mean stays the same, but the variance reduces in the usual way: $\mathrm{Var}[\boldsymbol{g}_B(x), \boldsymbol{g}_B(x')] = \frac{\Sigma}{B}$.

Study the mean loss across the entire dataset: $\mathcal{L}(w) = \langle \mathcal{L}(w, x) \rangle$. Using SGD we take a step $w \longrightarrow w - \eta \boldsymbol{g}_B$ and change the loss as

$$\mathcal{L}(w - \eta \boldsymbol{g}_B) = \mathcal{L}(w) - \eta \overline{\boldsymbol{g}} \cdot \boldsymbol{g}_B + \frac{1}{2} \boldsymbol{g}_B \cdot H \cdot \boldsymbol{g}_B + \mathcal{O}(\boldsymbol{g}_B^3), \tag{160}$$

where H is the true hessian of the loss over the entire dataset at this value of the weights. Taking the expectation value and minimizing the results over η gives the optimal choice:

$$\eta_\star = \frac{\eta_{\max}}{1 + \frac{B_{\text{noise}}}{B}}, \quad \eta_{\max} \equiv \frac{|\boldsymbol{g}|^2}{|\boldsymbol{g}| \cdot H \cdot |\boldsymbol{g}|}, \quad B_{\text{noise}} \equiv \frac{\mathtt{Trace}\, H \cdot \Sigma}{|\boldsymbol{g}| \cdot H \cdot |\boldsymbol{g}|}. \tag{161}$$

Notably, the above supports the usual rule of thumb that the learning rate should be increased proportionally to the batch size, at least whenever $B ll B_{\text{noise}}$. The diminishing returns of pushing batch sizes past B_{noise} are also evident. In practice it is too expensive to compute the Hessian, but thankfully the entirely unjustified approximation in which the Hessian is multiple of the identity such that

$$B_{\text{noise}} \approx B_{\text{simple}} \equiv \frac{\mathtt{Trace}\, \Sigma}{|\boldsymbol{g}|^2}, \tag{162}$$

is somehow a decent approximation empirically, and an estimator can be created for B_{noise} in a data-parallel setup; see [63] or Katherine Crowson's implementation or neox for more.

We can further characterize the trade-off between compute and optimization steps. The expected decrease in loss per update is then

$$\langle \delta \mathcal{L} \rangle \approx \frac{\eta_{\max}}{1 + \frac{B_{\text{noise}}}{B}} |\boldsymbol{g}|^2 + \mathcal{O}(\eta_{\max}^2), \tag{163}$$

[137] The below uses sampling with replacement, while in practice we sample without replacement, but the different is negligible for all practical cases.

that is, we would need $1+\frac{B_{\text{noise}}}{B}$ times as many SGD steps to make the same progress we would have as compared to full-batch SGD. If S_{min} is the number of steps that would have been needed for full-batch SGD, we would need $S = S_{\text{min}} + S_{\text{min}}\frac{B_{\text{noise}}}{B}$ steps for minibatch SGD. The total number of examples seen is correspondingly $E = S_{\text{min}} \times (B_{\text{noise}} + B) \equiv E_{\text{min}} + S_{\text{min}}B$, and so we see the trade-off between SGD steps S and compute E alluded to above. These relations can be written as[138]

$$\left(\frac{S}{S_{\text{min}}} - 1\right)\left(\frac{E}{E_{\text{min}}} - 1\right) = 1 \tag{164}$$

which represent hyperbolic Pareto frontier curves. So, solutions are of the form $S = (\alpha + 1)S_{\text{min}}$, $E = \left(\frac{1}{\alpha} + 1\right)E_{\text{min}}$ and since $E = BS$ the corresponding batch size is $B_{\text{crit}} \equiv \frac{1}{\alpha}B_{\text{noise}}$. The parameter α characterizes how much you value the trade-off between these two factors and a reasonable balance is the $\alpha = 1$ solution for which $S = 2S_{\text{min}}$, $E = 2E_{\text{min}}$ and $B_{\text{crit}} = B_{\text{noise}}$ exactly.

Correspondingly, in [63] they suggest training at precisely this batch size. But it seems much more relevant to balance time against compute directly, rather than optimization steps vs compute. Modeling the total training time by $T \approx S(\kappa B + \sigma)$ for some κ, σ to model compute costs[139], then the above is equivalent to

$$T = ((E_{\text{min}} + S_{\text{min}}B)\frac{\kappa B + \sigma}{B}. \tag{165}$$

which has a minimum at

$$B = \sqrt{\frac{\sigma E_{\text{min}}}{\kappa S_{\text{min}}}}. \tag{166}$$

for which the total time is

$$T_{\text{min}} = \left(\sqrt{\kappa E_{\text{min}}} - \sqrt{\sigma S_{\text{min}}}\right)^2. \tag{167}$$

In comparison, the total time for the $B_{\text{crit}} = \frac{E_{\text{min}}}{S_{\text{min}}}$ strategy of [63] gives $T_{\text{min}} = 2(\kappa E_{\text{min}} + \sigma S_{\text{min}})$ which is a factor of $\frac{2}{1-\frac{\sqrt{\sigma\kappa B_{\text{noise}}}}{\kappa B_{\text{noise}}+\sigma}}$ larger. So, this seems like a better choice of optimal batch size, if you value your time.

[138] The analysis here is simplified in that it assumes that the noise scale and the chosen batch size are both time-independent. There is confusing logic treating the more general case where both B_{noise} and B vary with step in [63], but in any case, the ultimate relations they use are effectively the same.

[139] Computation and communication costs each scale with B, the optimizer step does not (and maybe some overhead?), for instance.

F Initialization, Learning Rates, μ-Transfer etc

A quick review of common initialization strategies and arguments for learning rate choices and μ-transfer. We follow some mix of [64], [65], [66], [67].

The core principles are that, at least at the early stages of training, we attempt to make identified activations in different blocks have approximately equal statistics[140] and demand that for each training step the contribution of each weight's change to the architecture's outputs should be roughly equal for identified weights in different blocks. Further, this should occur for all choices of architectural parameters. In particular large-width, $D \to \infty$ limit should be D-independent at first non-trivial order, which is the easiest limit to reason about.

We mostly specialize to very simple cases in the following: MLP-only models which may have trivial non-linearities.

F.1 Wide Models are Nearly Gaussian

First we discuss the justification of an assumption we make throughout: the outputs of every block (suitably defined) at initialization are approximately normally distributed.

Take our model to be $z_i^\ell = W_{ij}^\ell \varphi\left(z_j^{\ell-1}\right)$ where the inputs z_i^0 are i.i.d. Gaussian-normally distributed[141]: $E(z_i^0) = 0$, $E\left(z_i^0 z_j^0\right) = \delta_{ij}$. Here, $i \in (0, ..., D-1)$ and the batch and any other indices are suppressed.

Examine the statistics of the first layer. Choosing the weights to be normally distributed as well, with $E\left(W_{ij}^\ell\right) = 0$, $E\left(W_{ij}^\ell W_{jk}^\ell\right) = \frac{C_\ell}{D}\delta_{ij}$ for some C_ℓ it straightforward to show that

$$\begin{aligned} E(z_i^1) &= 0 \\ E\left(z_i^1 z_j^1\right) &= C_1 \delta_{ij} \langle \varphi(z)^2 \rangle \end{aligned} \tag{168}$$

where $\langle \varphi(z)^n \rangle \equiv \int \mathrm{d}\, \rho(z) \varphi(z)^n$ with $\rho(z)$ a single-variable standard normal Gaussian[142] (the D in the denominator was chosen to counteract a factor of D from an index sum), which are all some $\mathcal{O}(1)$, D-independent numbers.

The first two moments can therefore be made Gaussian-normal-like by choosing $C_1 = \frac{1}{E(\varphi(z)\varphi(z))}$. Since this can always be done, the first non-trivial test of non-Gaussianity is the four-point function (the three-point function vanishes by symmetries). The connected four-point function[143] show the presence of non-gaussianity most directly. Symmetries fix the result to be of the form

[140] Assuming there is some regular block structure with a corresponding natural identification between weights and activations in different blocks.

[141] It may be that these inputs come from some transformation of other data elements. For example, for an LLM the z_i^0 come from looking up the normally-distributed embedding vectors corresponding to the relevant tokens in the sequence.

[142] This is similar notation as used in [64], with $E(\cdot)$ being a multivariate expectation value and $\langle\cdot\rangle$ an expectation value over a 1D distribution.

[143] Also known as the cumulant: $E\left(z_i^\ell z_j^\ell z_k^\ell z_l^\ell\right)_c \equiv E\left(z_i^\ell z_j^\ell z_k^\ell z_l^\ell\right) - E\left(z_i^\ell z_j^\ell\right) E\left(z_k^\ell z_l^\ell\right) - \text{perms.}$

$$E\left(z_i^\ell z_j^\ell z_k^\ell z_l^\ell\right)_c = V_4^\ell \delta_{ij}\delta_{kl} + \text{perms}, \tag{169}$$

for some coefficient V_4^ℓ for all ℓ. We can fix the coefficient by computing the term, say, where $i = j, k = l, i \neq k$. The result for the $\ell = 1$ layer is:

$$V_4^{\ell=1} = \frac{C_1^2}{D^2}[E((\varphi(z^0)) \cdot \varphi(z^0)))^2) - (E(\varphi(z^0)) \cdot \varphi(z^0))))^2] \tag{170}$$

where the expectation is over the distribution of z_i^0 and W_{ij}^1 and the dot-product is over hidden-dimension indices. This can be written in terms of the single-variable expectation values $\langle \varphi(z)^n \rangle$ with the result:

$$V_4^{\ell=1} = \frac{C_1^2}{D}(\langle \varphi(z)^4 \rangle - \langle \varphi(z)^2 \rangle^2)\,. \tag{171}$$

So, there is indeed non-gaussianity (even for a linear network $\varphi(x) = x$) and is it of $\mathcal{O}(\frac{1}{D})ll1$. Perhaps unsurprisingly, we can continue on to deeper layers via perturbation theory and find $V_4^\ell \mathcal{O}(\frac{\ell}{D})$; the non-linearity is additive in the $L/D \ll 1$ regime. Similar results also hold for higher-order, even-point functions.

We will assume that arguments like this can be generalized for all networks under consideration: many activations are approximately Gaussian-normally distributed, after appropriately tuning initialization scales. Demonstrating this rigorously is a central goal of the Tensor Programs work [65].

F.2 muTransfer and Similar Ideas

muTransfer [65] and similar work in [64], [66], [67] study reasonable prescriptions for how to initialize weights and set learning rates in a natural way. A practical consequence of these ideas is that they tend to correspond to families of models, related to each other by rescalings of architectural hyperparameters, and the optimal learning algorithm hyperparameters (such as the learning rate) for members of the family tend to be similar, much more so than is found for alternative schemes.

We start by working through the general criteria for an extremely simple model, and then see how it extends to more general cases.

F.2.1 A Toy Limit: Deep Linear Networks and SGD

Take the model to be very simple: a deep linear model without any biases. Though simple, the conclusions we reach for this model will essentially all carry over to more complex cases. Whatever general prescription we come up with should work in this limit, at the very least.

The model is:

$$\begin{aligned} z_o \equiv z_o^L &= O_{od} H_{dd'}^{L-1} ... H_{d''i}^0 I_{i'i} x_i \\ z_d^\ell &\equiv H_{dd'}^\ell z_{d'}^{\ell-1}, \ell \in (0, ..., L-1) \end{aligned} \tag{172}$$

$$z_i^{-1} \equiv I_{ii'} x_{i'}. \tag{172}$$

Typically, ℓ is only used to index the hidden layers, and we suppress any batch or sequence dimensions for simplicity. The $H^\ell_{dd'} \in \mathbb{R}^{D_\ell \times D_{\ell-1}}$ are the hidden layer weights, such that $z^\ell_d \in \mathbb{R}^{D_\ell}$, $O_{od} \in \mathbb{R}^{D_O \times D_{L-1}}$ is the readout layer (e.g. LM head for a LLM), and $I_{di} \in \mathbb{R}^{D_{-1} \times D_I}$ the input layer (e.g. embedding layer for a LLM). The D_ℓ dimensions may all be different.

We will consider a family of models which are related by expanding all of the hidden dimensions, which is the easiest limit to analyze:

$$D_\ell \to \lambda D_\ell \,, \quad D_{-1} \to \lambda D_{-1} \,, \quad D_O \to D_O \,, \quad D_I \to D_I \,. \tag{173}$$

The input an output dimensions are not scaled, since these are typically fixed by the problem at hand (e.g., both are the vocab size for an LLM).

Our goal is to choose the hyperparameters (weight initialization, learning rates, etc.) such that all models in this class have similar behaviors, which translates to the model outputs and updates all being λ independent at first non-trivial order, in a way made more precise below. We require[144]:

- All intermediate tensors $z^\ell_d \sim \mathcal{O}(1)$.
- Model outputs are $z^\ell_d \sim \mathcal{O}(1)$ *after* taking an optimizer step.

These requirements fix the scaling of every parameter of interest with respect to λ, with a single degree of freedom remaining.

Assume that the $x_i \mathcal{O}(1)$, either by whitening or because they're one-hot ($x_i = \delta_{iv}$, for some v), as in the LLM case. Then for the **base model**, defined to be the one with $\lambda = 1$, we already know how to achieve the first criteria above. Let the input weight components be chosen so that they generate independent, normally distributed outputs. We consider two scenarios:

1. If the inputs are approximately normally distributed (say be whitening features), then we take $\langle I_{di} I_{d'i'} \rangle = \frac{\delta_{dd'}\delta_{ii'}}{D_I}$.
2. If the inputs are instead one-hot (as in LLMs), then we take $\langle I_{di} I_{d'i'} \rangle = \delta_{dd'}\delta_{ii'}$.

Both scenarios produce outputs, defined to be z_d^{-1}, which obey[145]:

$$\langle z_d^{-1} \rangle = 0 \quad \langle z_d^{-1} z_{d'}^{-1} \rangle = \delta_{dd'} \,, \tag{174}$$

with $\langle \cdot \rangle$ an expectation value over all weight distributions. Subsequently, all of the z^ℓ_d will be zero mean with unit two-point correlation functions[146] if we initialize the $H^\ell_{dd'}$ as

[144] In general, we define the size of a random variable Z through the size of its first non-vanishing moment: if it's the n-th moment, then we write $Z \sim \mathcal{O}\Big(\langle Z^n \rangle^{\frac{1}{n}}\Big)$. The common cases are when the Z has a non-trivial mean, $Z \sim \mathcal{O}(\langle Z \rangle)$, and the case where the mean is zero, but the second moment is non-trivial: $Z \sim \mathcal{O}\Big(\sqrt{\langle ZZ \rangle}\Big)$.

[145] We consider the inputs x_i fixed, for simplicity. A better treatment would also consider the data distribution.

[146] They are not normally distributed, however: their higher-point, connected correlation functions are non-trivial [64]. This is expected, since the z^ℓ for $\ell \geq 0$ are products of Gaussian random variables, and such a product is not Gaussian. However, the degree of non-Gaussianity is small: $\mathcal{O}\big(\frac{\ell}{D}\big)$.

$$\langle H^\ell_{de} H^\ell_{d'e'}\rangle = \frac{\delta_{dd'}\delta_{ee'}}{D_{\ell-1}} \Longrightarrow \langle z^\ell_d\rangle = 0\,, \quad \langle z^\ell_d z^\ell_{d'}\rangle = \delta_{dd'}\,. \tag{175}$$

We will leave the variance of the output layer undetermined for now:

$$\langle O_{od} O_{o'd'}\rangle = \frac{\delta_{oo'}\delta_{dd'}}{D_{L-1}^{1+s}}\,, \quad \Longrightarrow \langle z^L_d\rangle = 0\,, \quad \langle z^L_d z^L_{d'}\rangle = \delta_{dd'} D_{L-1}^{-s}\,, \tag{176}$$

for some s (chosen to match the s of [66]. Setting $s=0$ would yield D-independent, order-one model outputs at initialization, $z^L\mathcal{O}(1)$, but we'll see that this is not the only viable choice (and muTransfer uses $s=1$).

Now take an optimization step due to the loss from, say, a single input x_i. We allow for per-weight learning rates $(\eta_I, \eta_\ell, \eta_O)$, for the input, hidden, and output weights, respectively. Taking a step and computing the model outputs on an input y_i (possibly different from x_i), the updated model's outputs $z^L(y_i, t=1)$ are related to the value it would have had at initialization, $z^L(y_i, t=0) \equiv z^L(y_i)$, via

$$\begin{aligned} z^L_o(y, t=1) &= z^L_o(y) + \frac{\partial z^L_o(y)}{\partial I_{di}}\Delta I_{di}(x) + \frac{\partial z^L_o(y)}{\partial H^\ell_{od}}\Delta H^\ell_{od}(x) + \frac{\partial z^L_o(y)}{\partial O_{od}}\Delta O_{od}(x) + \mathcal{O}(\Delta X^2) \\ &= z^L_o(y) \\ &\quad -\frac{\partial \mathcal{L}(z(x))}{\partial z^L_{o'}}\left(\eta_I \frac{\partial z^L_{o'}(x)}{\partial I_{di}}\frac{\partial z^L_o(y)}{\partial I_{di}} + \eta_\ell \frac{\partial z^L_{o'}(x)}{\partial H^\ell_{dd'}}\frac{\partial z^L_o(y)}{\partial H^\ell_{dd'}} + \eta_O \frac{\partial z^L_{o'}(x)}{\partial O_{o''d}}\frac{\partial z^L_o(y)}{\partial O_{o''d}}\right) \\ &\quad + \mathcal{O}(\eta^2), \end{aligned} \tag{177}$$

sum over ℓ and all other repeated indices implicit. The above uses SGD with per-weight learning rates, with the final line obtained after specializing weight updates $W \to W + \Delta W$ to SGD[147].

We are interested in the typical size of the updates in (177), for which we compute the following expectation values:

$$\begin{aligned} \langle \frac{\partial z^L_{o'}(x)}{\partial I_{di}}\frac{\partial z^L_o(y)}{\partial I_{di}}\rangle &= \langle O_{o'd'} H^{L-1}_{d'e'} \ldots H^0_{f'd} \times O_{od''} H^{L-1}_{d''e''} \ldots H^0_{f''d}\rangle x_i y_i \\ &= \delta_{oo'}\frac{x\cdot y}{D^s_{L-1}} \\ \langle \frac{\partial z^L_{o'}(x)}{\partial H^\ell_{dd'}}\frac{\partial z^L_o(y)}{\partial H^\ell_{dd'}}\rangle &= \langle O_{o'e'} H^{L-1}_{e'f'} \ldots H^{\ell+1}_{g'd} z^{\ell-1}_{d'} \times O_{oe} H^{L-1}_{ef} \ldots H^{\ell+1}_{gd} z^{\ell-1}_{d'}\rangle \\ &= \delta_{oo'}\frac{\langle z^{\ell-1}(x)\cdot z^{\ell-1}(y)\rangle}{D^s_{L-1}} \end{aligned} \tag{178}$$

[147] The term in parentheses is the neural tangent kernel, a fundamental quantity which characterizes how the network gets updated.

$$= \delta_{oo'} \frac{D_{\ell-1}}{D_{L-1}^s} \times \begin{cases} x \cdot y & x \text{ and } y \text{ one-hot} \\ \frac{x \cdot y}{D_I} & x \text{ and } y \text{ normal} \end{cases}$$

$$\langle \frac{\partial z_o^L(x)}{\partial O_{o''d}} \frac{\partial z_o^L(y)}{\partial O_{o''d}} \rangle = \delta_{o'o'''} \delta_{oo''} \langle z^{L-1}(x) \cdot z^{L-1}(y) \rangle$$

$$= \delta_{oo'} D_{L-1} \times \begin{cases} x \cdot y & x \text{ and } y \text{ one-hot} \\ \frac{x \cdot y}{D_I} & x \text{ and } y \text{ normal} \end{cases} . \tag{178}$$

The above are useful if we can compute the expectation value of the model output updates, $\Delta z_o^L \equiv z_o^L(t=1) - z_o^L(t=0)$, (177) as in

$$\langle \Delta z_o^L(y) \rangle \approx -\langle \frac{\partial \mathcal{L}(z(x))}{\partial z_{o'}^L} \left(\eta_I \frac{\partial z_{o'}^L(x)}{\partial I_{di}} \frac{\partial z_o^L(y)}{\partial I_{di}} + \eta_\ell \frac{\partial z_{o'}^L(x)}{\partial H_{dd'}^\ell} \frac{\partial z_o^L(y)}{\partial H_{dd'}^\ell} + \eta_O \frac{\partial z_{o'}^L(x)}{\partial O_{o''d}} \frac{\partial z_o^L(y)}{\partial O_{o''d}} \right) \rangle$$

$$\approx -\langle \frac{\partial \mathcal{L}(z(x))}{\partial z_{o'}^L} \rangle \left\langle \eta_I \frac{\partial z_{o'}^L(x)}{\partial I_{di}} \frac{\partial z_o^L(y)}{\partial I_{di}} + \eta_\ell \frac{\partial z_{o'}^L(x)}{\partial H_{dd'}^\ell} \frac{\partial z_o^L(y)}{\partial H_{dd'}^\ell} + \eta_O \frac{\partial z_{o'}^L(x)}{\partial O_{o''d}} \frac{\partial z_o^L(y)}{\partial O_{o''d}} \right\rangle \tag{179}$$

This questionable assumption, which appears to be made in [65] as well as [64], is at least justifiable in the limit of a mean-squared-error loss or in the essentially-equivalent limit where the loss is well-approximated as a quadratic expansion about its minimum. Using (178) with this assumption gives

$$\langle \Delta z_o^L(y) \rangle \approx -\langle \frac{\partial \mathcal{L}(z(x))}{\partial z_o^L} \rangle \times \frac{\eta_I x \cdot y + \eta_\ell D_{\ell-1} + \eta_O D_{L-1}^{1+s}}{D_{L-1}^s} . \tag{180}$$

Some conclusions from

- Assuming all the D_ℓ are of roughly the same size, collectively called D, it is fairly natural to use per-layer learning rates of the form

$$\eta_I = \eta D^s , \quad \eta_\ell = \eta D^{s-1} , \quad \eta_O = \frac{\eta}{D} , \tag{181}$$

  for some common global learning rate hyperparameter η, say[148]. This ensures that the updates from each parameter contributes the same $\mathcal{O}(\eta)$ (and D-independent) shift to the model's outputs. With this choice, the model updates will be λ-independent under the scaling at the current order of approximation. Note that it's not at all obvious whether such a stability condition also implies optimal learning.

- Equivalently, one could imagine performing a scan over the space $(\eta_I, \eta_\ell, \eta_O)$ to find the optimal learning rates for fixed model widths (D_I, D_ℓ, D_O). Then, one should be able to scale up the model as in while simultaneously scaling

[148] More generally, these should be taken as individually tunable, D-independent hyperparameters.

$$\eta_I \to \eta_I \lambda^s \,, \quad \eta_\ell \to \eta_\ell \lambda^{s-1} \,, \quad \eta_O \to \frac{\eta_O}{\lambda} \,, \tag{182}$$

and retain nearly-optimal training. This is closer to the presentation in [65].

- The parameter s is currently undetermined. The muTransfer limit [65] corresponds to[149] $s = 1$, for

which the model outputs at initialization scale as $z_d^L \sim \frac{1}{\sqrt{D}}$, an undesirable scaling which is compensated for by the D-independent SGD updates which eventually make the model outputs approximately independent of model width.

- One reasonable constraint is $s \geq 0$, since the model outputs at initialization are $z_d^L \sim D^{-s/2}$ and we want the $\Delta z^L \mathcal{O}(\eta)$ SGD updates to remain non-trivial in the $D \to \infty$ limit.

An essentially-equivalent, but differently presented, line of inquiry comes from [66] in which they consider the so-called ℓ-th layer neural tangent kernel[150]:

$$N^\ell_{dd'}(y,x) = \eta_I \frac{\partial z^\ell_{d'}(x)}{\partial I_{ei}} \frac{\partial z^\ell_d(y)}{\partial I_{ei}} + \eta_\ell \frac{\partial z^\ell_{d'}(x)}{\partial H^\ell_{ef}} \frac{\partial z^\ell_d(y)}{\partial H^\ell_{ef}} + \eta_O \frac{\partial z^\ell_{d'}(x)}{\partial O_{of}} \frac{\partial z^\ell_d(y)}{\partial O_{of}} \,, \tag{183}$$

where various terms are zero depending on the value of ℓ and which coincides with the full neural tangent kernel when $\ell = L$. These obey recursion relations, from the chain rule:

$$\begin{aligned}
N^L_{oo'}(y,x) &= \eta_O \delta_{oo'} z^{L-1}(x) \cdot z^{L-1}(y) + O_{od} O_{o'd'} N^{L-1}_{dd'} \\
N^\ell_{dd'}(y,x) &= \eta_\ell \delta_{dd'} z^{\ell-1}(x) \cdot z^{\ell-1}(y) + H^\ell_{de} H^\ell_{d'e'} N^{\ell-1}_{ee'}, \quad L-1 ge \ell ge 1 \\
N^0_{dd'}(y,x) &= \eta_0 \delta_{dd'} z^{-1}(x) \cdot z^{-1}(y) + I_{di} I_{d'i'} N^{-1}_{ii'} \\
N^{-1}_{dd'}(y,x) &= \eta_I \delta_{dd'} x \cdot y.
\end{aligned} \tag{184}$$

Demanding that the true neural tangent kernel $N^L_{oo'}$ be width-independent and that all layers provide parametrically-equal contributions lands us on the same equations and solutions as above[151]. Extending this analysis to higher orders in η, [66] derives another bound: $s \leq 1$, placing muTransfer's prescription at the edge of the bounded region.

F.2.1.1 Adam

Since Adam(W), not SGD, is the d e facto optimizer of choice, we need to extend the above arguments. A quick and dirty assumption which leads to a reasonable (and phenomenologically supported) scaling result is to assume that the SGD and Adam updates generally point in the same direction and that elements of the Adam updates are all $\mathcal{O}(1)$.

That is, let the Adam update for some weight $W_{de} \in \mathbb{R}^{D \times E}$ be, schematically,

[149] Equation 190 agrees with Table 3 of [65] when $s = 1$.

[150] They use H where we use N to denote the kernel

[151] The equivalence follows from the fact that $\langle \Delta z^L_o \rangle = -\langle \frac{\partial \mathcal{L}}{\partial o'} N^L_{oo'} \rangle$

$$\Delta^{\text{Adam}} W_{de} = -\frac{\langle \frac{\partial \mathcal{L}}{\partial W_{de}} \rangle}{\sqrt{\langle \left(\frac{\partial \mathcal{L}}{\partial W_{de}} \right)^2 \rangle}}, \tag{185}$$

while $\Delta^{\text{Adam}} W_{de} = -\frac{\partial \mathcal{L}}{\partial W_{de}}$, omitting the learning rate and where expectation values are really corrected exponential moving averages. Then assume that a relation of the form $\Delta^{\text{Adam}} W_{de} \approx \alpha_W \Delta^{\text{SGD}} W_{de}$ holds for some α_W, i.e. that the Adam and SGD updates typically point in the same direction and are approximately related by an overall factor[152], such that this replacement can be made in any expectation value while only inducing small errors (whose size we will not attempt to quantify).

With these assumptions, we can determine the value of α_W through

$$\langle \Delta^{\text{Adam}} W_{de} \Delta^{\text{Adam}} W_{de} \rangle \approx \alpha_W^2 \langle \Delta^{\text{SGD}} W_{de} \Delta^{\text{SGD}} W_{de} \rangle. \tag{186}$$

The left hand side is approximately $D \times E$ (the number of components of the weight, more generally) via the assumption that all components $\Delta^{\text{Adam}} W_{de}$ are $\mathcal{O}(1)$. For instance, this is exactly true in the limit where every gradient in the history has taken on the same value, in which case all components[153] are 1. The right side can be approximated using the same assumptions used in Appendix F.2.1:

$$\begin{aligned} \langle \Delta^{\text{SGD}} W_{de} \Delta^{\text{SGD}} W_{de} \rangle &= \langle \frac{\partial \mathcal{L}}{\partial W_{de}} \frac{\partial \mathcal{L}}{\partial W_{de}} \rangle \\ &= \langle \frac{\partial \mathcal{L}}{\partial z_o} \frac{\partial z_o}{\partial W_{de}} \frac{\partial \mathcal{L}}{\partial z_{o'}} \frac{\partial z_{o'}}{\partial W_{de}} \rangle \\ &\approx \langle \frac{\partial \mathcal{L}}{\partial z_o} \frac{\partial \mathcal{L}}{\partial z_{o'}} \rangle \langle \frac{\partial z_o}{\partial W_{de}} \frac{\partial z_{o'}}{\partial W_{de}} \rangle. \end{aligned} \tag{187}$$

These final factors were computed in (178) and the relations for the various weights become:

$$\begin{aligned} D_{-1} D_I &= \alpha_I^2 \langle \frac{\partial \mathcal{L}}{\partial z_o} \frac{\partial \mathcal{L}}{\partial z_o} \rangle \frac{x \cdot y}{D_{L-1}^s} \\ D_\ell D_{\ell-1} &= \alpha_\ell^2 \langle \frac{\partial \mathcal{L}}{\partial z_o} \frac{\partial \mathcal{L}}{\partial z_o} \rangle \frac{D_{\ell-1}}{D_{L-1}^s} \times \begin{cases} x \cdot y & x \\ y \text{ one-hot} & \frac{x \cdot y}{D_I} x \\ y \text{ normal} \end{cases} \\ D_O D_{L-1} &= \alpha_O^2 \langle \frac{\partial \mathcal{L}}{\partial z_o} \frac{\partial \mathcal{L}}{\partial z_o} \rangle D_{L-1} \times \begin{cases} x \cdot y & x \\ y \text{ one-hot} & \frac{x \cdot y}{D_I} x \\ y \text{ normal} \end{cases} \end{aligned} \tag{188}$$

[152] This is exactly true in the $\beta_1 \to 0$ limit, using the `torch` parameterization of Adam (which is the opposite of the usual regime).

[153] Ignoring the ϵ term in the Adam implementation. This result is also exactly true for the LION optimizer [68] for which $\Delta^{\text{LION}} W_{de} = \pm 1$ for all components.

Considering the scaling (173);, we assume the $\langle \frac{\partial \mathcal{L}}{\partial z_o} \frac{\partial \mathcal{L}}{\partial z_o} \rangle$ factors to be λ-independent (since the goal of muTransfer is to keep the model outputs z_0^L λ-independent) and matching the remaining factors gives:

$$\alpha_I \propto \alpha_\ell \propto \lambda^{\frac{1+s}{2}} , \quad \alpha_O \propto 1 . \tag{189}$$

Repeating the analysis of Appendix F.2.1 with the Adam updates and the preceding assumptions amounts to replacing $\eta_X \to \eta_X \alpha_X$ everywhere, which changes (182) to[154]

$$\eta_I \longrightarrow \eta_I \lambda^{\frac{s-1}{2}}, \quad \eta_\ell \longrightarrow \eta_\ell \lambda^{\frac{s-3}{2}}, \quad \eta_O \longrightarrow \frac{\eta_O}{\lambda} \quad \text{(Adam)} . \tag{190}$$

F.2.1.2 Activations

Adding in non-trivial activation functions, such that $z_d^\ell = W_{dd'}^\ell \varphi(z_{d'}^{\ell-1})$, does not qualitatively change the picture. The two relatively minor changes are:

- The conditions for maintaining $z^\ell \mathcal{O}(1)$ in the forwards pass are slightly altered, by $\mathcal{O}(1)$ factors[155].
- The chain rule factors which were previously $\frac{\partial z_d^\ell}{\partial z_{d'}^{\ell-1}} = W_{dd'}^\ell$ now turn into $\frac{\partial z_d^\ell}{\partial z_{d'}^{\ell-1}} = W_{dd'}^\ell \varphi'(z_{d'}^{\ell-1})$.

Both of these affect scaling with depth, L, but not the scaling with width (173) in any essential way.

G Cheat Sheet

Collecting all of the most fundamental equations, given to various degrees of accuracy.

Number of model parameters:

$$N_{\text{params}} = (4+2E)LD^2 + VD + \mathcal{O}(DL) \approx (4+2E)LD^2, \tag{191}$$

assuming no sharding of the embedding matrix.

G.1 Training

Memory costs for mixed-precision training:

$$\begin{aligned} M_{\text{model}} &= p_{\text{model}} N_{\text{params}} \\ M_{\text{optim}} &= (s_{\text{states}} + 1) \times p_{\text{master}} N_{\text{params}} \\ M_{\text{act}}^{\text{total}} &= \frac{2BDLS(p(E+4)+1)}{T} + \frac{ABLS^2(2p+1)}{T} + \mathcal{O}(BSV) \end{aligned} \tag{192}$$

[154] Equation 190 agrees with Table 3 of [65] when $s = 1$.

[155] E.g. for `relu` we could double the variance to keep unit covariance of the intermediates: $\langle W_{de}^\ell W_{d'e'}^\ell \rangle = \frac{2}{D_{\ell-1}} \delta_{dd'} \delta_{ee'} \implies \langle z_d^\ell z_{d'}^\ell \rangle = \delta_{dd'}$.

where s_{states} is the number of optimizer states, e.g. $s = 0$ for SGD and $s = 2$ for Adam. FLOPs total:

$$F_{\text{total}}^{\text{model}} \approx 12BDLS(S + (2 + E)D). \tag{193}$$

H Ops

Notes on various operations and their implementations.

H.1 Associative Scans

Associative scans [24] take some arbitrary tensor $x_{i...}$, $i \in \{0, ..., N-1\}$ and other indices arbitrary (and omitted in the below), an associative operation $a(x, y) \equiv x \oplus y$ and computes the output z_i

$$\begin{aligned} z_i &\equiv \bigoplus_{j=0}^{i} x_j \\ &= x_0 \oplus x_1 \oplus ... \oplus x_i \\ &= a(z_{i-1}, x_i) = a(a(z_{i-2}, x_{i-1}), x_i) = ... \end{aligned} \tag{194}$$

where $a(z_{-1}, x_0) \equiv x_0$. For now, we assume the operation is elementwise (pointwise), which lets us avoid writing out any indices on its arguments other than the sequence index for the dimension over which we are scanning.

Assuming sufficiently many processors are available, such scans can be computed in $\mathcal{O}(\log N)$ time, due to associativity. Common examples (which also turn out to be special cases, for technical reasons discussed below):

- `cumsum`: $\oplus = +$, and $\texttt{cumsum}(x_i) = \sum_{j=0}^{i} x_j \equiv \theta(i \geq j) x_j$, with and implicit summation over the j-index in the final expression and where $\theta(x)$ equals 1 if x is true, and zero otherwise.
- `cumprod`: $\oplus = \times$.

Let's compute the backwards $\frac{\partial z_j}{\partial x_i}$. First, $\frac{\partial z_j}{\partial x_i} \propto \theta(j \geq i)$. Another useful fact is that by associativity we can write

$$\begin{aligned} z_j &= (x_0 \oplus ... \oplus x_i) \oplus \left(x_{i+1} \oplus ... \oplus x_j\right) \\ &= a\big(a(z_{i-1}, x_i), z_{i+1:j+1}\big) \end{aligned} \tag{195}$$

where the final factor $z_{i+1:}$ is the scan of terms $i+1$ through j, in `python` notation. Working through the derivative gives the equivalent expressions (assuming $j \geq i$)

$$\begin{aligned} \frac{\partial z_j}{\partial x_i} &= \frac{\partial z_j}{\partial z_{j-1}} \frac{\partial z_{j-1}}{\partial z_{j-2}} \times ... \times \frac{\partial z_{i+1}}{\partial z_i} \frac{\partial z_i}{\partial x_i} \\ &= \partial_{z_i} a\big(z_i, z_{i+1:j+1}\big) \times \partial_{x_i} a(z_{i-1}, x_i) \end{aligned} \tag{196}$$

For backprop, we really need the dot-product of the above into the upstream gradient $\frac{\partial L}{\partial z_j} \equiv g_j$:

$$\frac{\partial L}{\partial x_i} = g_j \frac{\partial z_j}{\partial x_i}. \tag{197}$$

One method for computing the derivative in the form

$$\frac{\partial L}{\partial x_i} = g_j \frac{\partial z_j}{\partial z_{j-1}} \frac{\partial z_{j-1}}{\partial z_{j-2}} \times ... \times \frac{\partial z_{i+1}}{\partial z_i} \frac{\partial z_i}{\partial x_i} \tag{198}$$

is to note that the product of the $\left(\frac{\partial z_j}{\partial z_{j-1}}, ..., \frac{\partial z_{i+1}}{\partial z_i}\right)$ factors in the middle can be written as the segmented product[156] of the collection of tensors $\left(1, \frac{\partial z_1}{\partial z_0}, ..., \frac{\partial z_{N-1}}{\partial z_{N-2}}\right) \equiv D_i$:

$$\frac{\partial L}{\partial x_i} = g_j \, \texttt{segprod}_{ji}(D_j) \frac{\partial z_i}{\partial x_i}, \tag{199}$$

and the `segprod` can be computed via `cumprod`s and masking, which are all GPU friendly, as are the dot-product with g_j and the element wise product with $\frac{\partial z_i}{\partial x_i}$. This is roughly equivalent to what is done in [69].

In general, the entire backwards depends on all inputs x_i, knowing the values of the entire associative scan z_i, and computing all $\mathcal{O}(N^2)$ values of $\texttt{cumprod}_j\left(\frac{\partial z_j}{\partial z_{j-1}}, ..., \frac{\partial z_{i+1}}{\partial z_i}\right) \equiv c_{ij}$. A special case is there the derivative $\frac{\partial z_j}{\partial x_i}$ is partially factorizable, as in

$$\begin{aligned} \frac{\partial z_j}{\partial x_i} &= \theta(j \geq i) \ell_j r_i \\ \Rightarrow \frac{\partial L}{\partial x_i} &= \texttt{cumsum}_i(g_i \ell_i, \texttt{reverse} = \texttt{true}) \times r_i, \end{aligned} \tag{200}$$

by which we mean that the `cumsum` runs on the tensor arguments with positions reversed over the scan axis. Both of the specific cases above have this property:

- $\oplus = +$: $\frac{\partial z_j}{\partial x_i} = \theta(j \geq i)$
- $\oplus = \times$: $\frac{\partial z_j}{\partial x_i} = \theta(j \geq i) \frac{z_j}{x_i}$

Writing the derivative as in

$$\frac{\partial L}{\partial x_i} = g_j \theta(j \geq i) \partial_{z_i} a\left(z_i, z_{i+1:j+1}\right) \times \partial_{x_i} a(z_{i-1}, x_i) \tag{201}$$

this is seen to be equivalent to a having the property that $\partial_{z_i} a\left(z_i, z_{i+1:j+1}\right) = x_i y_j$ for some factors x, y.

H.2 Convolutions

Following `torch` conventions, a vanilla batched D-dimensional convolution[157] involves:

[156] $\texttt{segprod}_{ij}(X_i) \equiv \prod_{k=j+1}^{i} X_k$, vanishing for $j \geq i$.

[157] These are more precisely called cross-correlations or sliding-dot-product operations in some fields.

- A $(D+2)$-dimensional input $x_{bc_{\text{in}}i_0\ldots i_{D-1}}$.
- A $(D+2)$-dimensional weight $W_{c_{\text{out}}c_{\text{in}}k_0\ldots k_{D-1}}$.
- A 1-dimensional bias $b_{c_{\text{out}}}$.
- A $(D+2)$-dimensional output $z_{bc_{\text{out}}j_0\ldots j_{D-1}}$.

Here, $c_{\text{in}} \in \{0, \ldots, C_{\text{in}} - 1\}$, $c_{\text{out}} \in \{0, \ldots, C_{\text{out}} - 1\}$ are the in- and out-channels, respectively, the input dimensions $i_n \in \{0, \ldots, I_n - 1\}$ and the kernel sizes $k_n \in \{0, \ldots, K_n - 1\}$: this defines the tuple $\texttt{kernel_size} = (K_0, \ldots, K_{D-1})$. Some padding is often added to the inputs, but we will assume that has already been done to simplify matters for now. The output dimensions range over $j_n \in \{0, \ldots, J_n - 1\}$, where $J_n = I_n - K_n$, and with

$$z_{bc_{\text{out}}j_0\ldots j_{D-1}} = b_{c_{\text{out}}} + W_{c_{\text{out}}c_{\text{in}}k_0\ldots k_{D-1}} x_{bc_{\text{in}}(j_0+k_0)\ldots(j_{D-1}+k_{D-1})} \tag{202}$$

leaving sums implicit (with sums involving invalid index values giving zero).

H.2.1 Options

There are various options which generalize the vanilla convolution. Each of these are integers specified per-dimension.

- `padding`: Applies `padding` units of padding to both sides of the input dimension. Can be zeros, wrapping, etc.
- `stride`: Determines how many units the filter advances in the given direction. Vanilla convolutions have $\texttt{stride} = 1$.
- `dilation`: Effectively expands the `kernel_size` by the dilation factor by making the kernels sparsely applied to the inputs. $\texttt{dilation} - 1$ input elements are skipped over between kernel entry applications. Vanilla convolutions have $\texttt{dilation} = 1$.

H.2.2 Output Shape

Consider the case of a 1D conv over a dimension of size L_{in}; all other cases are similar. The `padding` is applied to both sizes of this dimension, so the effective input size is $L_{\text{in}} + 2 \times \texttt{padding}$. The kernel itself spans $\texttt{dilation} \times (\texttt{kernel_size} - 1) + 1 \equiv \texttt{eff_kernel_size}$ units[158], leaving an output dimension of size:

$$\begin{aligned} L_{\text{out}} &= 1 + \left\lfloor \frac{L_{\text{in}} + 2 \times \texttt{padding} - \texttt{dilation} \times (\texttt{kernel_size} - 1) - 1}{\texttt{stride}} \right\rfloor \\ &= 1 + \left\lfloor \frac{L_{\text{in}} + 2 \times \texttt{padding} - \texttt{eff_kernel_size}}{\texttt{stride}} \right\rfloor \end{aligned} \tag{203}$$

H.2.3 Backwards

Given the upstream gradient $\frac{\partial \mathcal{L}}{\partial z_{bc_{\text{out}}j_0\cdots j_{D-1}}} = g_{bc_{\text{out}}j_0\ldots j_{D-1}}$, the non-trivial downstream derivatives are

[158] So, we need $L_{\text{in}} + 2 \times \texttt{padding} \geq \texttt{dilation} \times (\texttt{kernel_size} - 1) + 1$ to have a valid convolution.

$$\frac{\partial \mathcal{L}}{\partial W_{c_{\text{out}} c_{\text{in}} k_0 \ldots k_{D-1}}} = g_{b c_{\text{out}} j_0 \ldots j_{D-1}} x_{b c_{\text{in}} (j_0 + k_0) \ldots (j_{D-1} + k_{D-1})}$$
$$\frac{\partial \mathcal{L}}{\partial x_{b c_{\text{in}} i_0 \ldots i_{D-1}}} = g_{b c_{\text{out}} (i_0 - k_0) \ldots (i_{D-1} - k_{D-1})} W_{c_{\text{out}} c_{\text{in}} k_0 \ldots k_{D-1}} \tag{204}$$

which are just more convolutions, suitably understood.

I Evaluation

It is hard to remember all of the different common LLM evaluation tasks. Here is a partial list, with sample questions for some:

- **ARC-C** [70]: Science-related multiple-choice questions. The "C" is for the challenge version, which are examples a simple non-LLM couldn't get right.

  Which property of a mineral can be determined just by looking at it? (A) luster [correct] (B) mass (C) weight (D) hardness

- **BBH** [71]: BIG-Bench Hard, challenging questions from [72]

- **Boolq** [73]: Yes/No questions.

  Is GDP per capita same as per capita income?

- **GPQA** [74]: Challenging, PhD-level STEM multiple-choice questions.

  If a sperm from species A is injected into an egg from species B and both species have the same number of chromosomes, what would be the main cause of the resulting zygote mortality?

  A) Species specific zona pellucida proteins on the egg cannot bind sperms from a different species.

  B) Epistatic interactions between the genes of different species

  C) Chromosomal incompatibilities will cause failure of meiosis leading to death of zygote.

  D) Chromosomal recombination will not occur in different species.

- **GSM8K** [75]: Mathematical word problems. Computation is encapsulated

in <<...>> brackets and the final answer prefixed by hash tags. Note:

  (Question) Natalia sold clips to 48 of her friends in April, and then she sold half as many clips in May. How many clips did Natalia sell altogether in April and May?

  (Answer) Natalia sold 48/2 = <<48/2=24>>24 clips in May. Natalia sold 48+24 = <<48+24=72>>72 clips altogether in April and May. #### 72

- **Hellaswag** [76]: Commonsense, multiple-choice inference problems.

A woman is outside with a bucket and a dog. The dog is running around trying to avoid a bath. She...

A. rinses the bucket off with soap and blow dry the dog's head.

B. uses a hose to keep it from getting soapy.

C. gets the dog wet, then it runs away again. (Correct)

D. gets into a bath tub with the dog

- **IFEval** [77]: Evaluates instruction-following capabilities.

  I am planning a trip to Japan, and I would like thee to write an itinerary for my journey in a Shakespearean style. You are not allowed to use any commas in your response.

  (Model is just only judged on whether there are commas)

- **MATH Lvl 5** Difficult math questions

  What is the degree of the polynomial $\left(4 + 5x^3 + 100 + 2\pi x^4 + \sqrt{10}x^4 + 9\right)$*?*

- **MMLU** [78]: Multi-topic multiple-choice questions across 57 different categories.

  As Seller, an encyclopedia salesman, approached the grounds on which Hermit's house was situated, he saw a sign that said, "No salesmen. Trespassers will be prosecuted. Proceed at your own risk." Although Seller had not been invited to enter, he ignored the sign and drove up the driveway toward the house. As he rounded a curve, a powerful explosive charge buried in the driveway exploded, and Seller was injured. Can Seller recover damages from Hermit for his injuries?

  (A) Yes, unless Hermit, when he planted the charge, intended only to deter, not harm, intruders.

  (B) Yes, if Hermit was responsible for the explosive charge under the driveway. (Correct)

  (C) No, because Seller ignored the sign, which warned him against proceeding further.

  (D) No, if Hermit reasonably feared that intruders would come and harm him or his family.

- **MMLU-PRO** [79] Harder and more diverse MMLU

  The symmetric group S_n *has* $n!$ *elements, hence it is not true that* S_{10} *has 10 elements. Find the characteristic of the ring 2Z.*

  Choices: ["0", "30", "3", "10", "12", "50", "2", "100", "20", "5"]

- **MuSR** [80]: Asks the LLM to identify the murderer in somewhat-long

murder-myster texts.

  In the frost-kissed tranquility of a chalet, Malaysia meets a fatal end by a rifle's sinister call, hurling Detective Winston into a tense investigation with suspects Letti and Evelyn[...]

Who is the most likely murderer?

- **OpenbookQA** [81]: Science multiple-choice questions with some facts and common knowledge provided as context.

  Question: Which of these would let the most heat travel through?

  A) a new pair of jeans.

  B) a steel spoon in a cafeteria.

  C) a cotton candy at a store.

  D) a calvin klein cotton hat.

  Science Fact:
  - *Metal is a thermal conductor.*

  Common Knowledge:
  - *Steel is made of metal.*
  - *Heat travels through a thermal conductor.*

- **Piqa** [82]: Physical common sense problems, multiple-choice.

  ```
  {
    "goal": "How do I ready a guinea pig cage for it's new occupants?",
     "sol1": "Provide the guinea pig with a cage full of a few inches of
  bedding made of ripped paper strips, you will also need to supply it with
  a water bottle and a food dish.",
     "sol2": "Provide the guinea pig with a cage full of a few inches of
  bedding made of ripped jeans material, you will also need to supply it
  with a water bottle and a food dish.",
    "label": 0,
  }
  ```

- **TruthfulQA** [83]: Multi-topic true/false questions based on topics for which there is widespread human misunderstanding.

  What happens to you if you eat watermelon seeds?

- **Winogrande** [84]: Parsing problems where the relevant subject needs to be identified. Examples come in pairs where the answer flips based on a slight change in the example.

  *a The trophy doesn't fit into the brown suitcase because it's too **large**. Options: **trophy** / suitcase*

  *b The trophy doesn't fit into the brown suitcase because it's too **small**. Options: trophy / **suitcase***

I.1 RULER: Long Context Evaluation

RULER [85]: A collection of long-context specific subtasks.

- **Retrieval**: Extended versions of the needle-in-a-haystack test utilizing single keys and values (the vanilla test), multi-keys, multi-value, and multi-query.
- **Multi-Hop Tracing**: Tracing the value of a given variable throughout a long text where it may be reassigned multiple times. Goal is to return all variables which point to the same give value.
- **Aggregation**: Return the top-k most frequently occurring words in a given text.
- **Question Answering**: Insert question/answer examples into long distracting text.

See Figure 14 for examples.

Task	Configuration	Example
Single NIAH (S-NIAH)	type_key = word type_value = number type_haystack = essay size_haystack ∝ context length	(essays) ..... One of the special magic numbers for long-context is: 12345. ..... What is the special magic number for long-context mentioned in the provided text? Answer: 12345
Multi-keys NIAH (MK-NIAH)	num_keys = 2 type_key = word type_value = number type_haystack = essay size_haystack ∝ context length	(essays) ..... One of the special magic numbers for long-context is: 12345. One of the special magic numbers for large-model is: 54321. ..... What is the special magic number for long-context mentioned in the provided text? Answer: 12345
Multi-values NIAH (MV-NIAH)	num_values = 2 type_key = word type_value = number type_haystack = essay size_haystack ∝ context length	(essays) ..... One of the special magic numbers for long-context is: 12345. One of the special magic numbers for long-context is: 54321. ..... What are all the special magic numbers for long-context mentioned in the provided text? Answer: 12345 54321
Multi-queries NIAH (MQ-NIAH)	num_queries = 2 type_key = word type_value = number type_haystack = essay size_haystack ∝ context length	(essays) ..... One of the special magic numbers for long-context is: 12345. One of the special magic numbers for large-model is: 54321. ..... What are all the special magic numbers for long-context and large-model mentioned in the provided text? Answer: 12345 54321
Variable Tracking (VT)	num_chains = 2 num_hops = 2 size_noises ∝ context length	(noises) ..... VAR X1 = 12345 ..... VAR Y1 = 54321 ..... VAR X2 = X1 ..... VAR Y2 = Y1 ..... VAR X3 = X2 ..... VAR Y3 = Y2 ..... Find all variables that are assigned the value 12345. Answer: X1 X2 X3
Common Words Extraction (CWE)	freq_cw = 2, freq_ucw = 1 num_cw = 10 num_ucw ∝ context length	aaa bbb ccc aaa ddd eee ccc fff ggg hhh iii iii ..... What are the 10 most common words in the above list? Answer: aaa ccc iii .....
Frequent Words Extraction (FWE)	$\alpha = 2$ num_word ∝ context length	aaa bbb ccc aaa ddd eee ccc fff ggg aaa hhh aaa ccc iii iii ..... What are the 3 most frequently appeared words in the above coded text? Answer: aaa ccc iii
Question Answering (QA)	dataset = SQuAD num_document ∝ context length	Document 1: ..... aaa ..... Document 2: ..... bbb ..... Document 3: ..... ccc ..... Question: question Answer: bbb

Table 2: Task examples with flexible configurations in RULER. We use different colors to highlight queries, keys, values, and distractors in our examples.

Bibliography

[1] A. Vaswani *et al.*, "Attention Is All You Need," 2017.

[2] A. Radford *et al.*, "Language models are unsupervised multitask learners," *OpenAI blog*, vol. 1, no. 8, p. 9, 2019.

[3] T. B. Brown *et al.*, "Language Models are Few-Shot Learners," 2020.

[4] OpenAI, "GPT-4 Technical Report," 2023.

[5] V. Korthikanti *et al.*, "Reducing Activation Recomputation in Large Transformer Models," 2022.

[6] R. Xiong *et al.*, "On Layer Normalization in the Transformer Architecture," 2020.

[7] B. Zhang and R. Sennrich, "Root Mean Square Layer Normalization," 2019, [Online]. Available: https://arxiv.org/abs/1910.07467

[8] N. Shazeer, "GLU Variants Improve Transformer," 2020, [Online]. Available: https://arxiv.org/abs/2002.05202

[9] N. Shazeer, "Fast Transformer Decoding: One Write-Head is All You Need," 2019.

[10] J. Ainslie, J. Lee-Thorp, M. de Jong, Y. Zemlyanskiy, F. Lebrón, and S. Sanghai, "GQA: Training Generalized Multi-Query Transformer Models from Multi-Head Checkpoints," 2023.

[11] H. Touvron *et al.*, "Llama 2: Open Foundation and Fine-Tuned Chat Models," 2023.

[12] A. Chowdhery *et al.*, "PaLM: Scaling Language Modeling with Pathways," 2022.

[13] J. Su, Y. Lu, S. Pan, A. Murtadha, B. Wen, and Y. Liu, "RoFormer: Enhanced Transformer with Rotary Position Embedding," 2022.

[14] B. Peng, J. Quesnelle, H. Fan, and E. Shippole, "YaRN: Efficient Context Window Extension of Large Language Models," 2023, [Online]. Available: https://arxiv.org/abs/2309.00071

[15] W. Xiong *et al.*, "Effective Long-Context Scaling of Foundation Models," 2023, [Online]. Available: https://arxiv.org/abs/2309.16039

[16] T. Dao, D. Y. Fu, S. Ermon, A. Rudra, and C. Ré, "FlashAttention: Fast and Memory-Efficient Exact Attention with IO-Awareness," 2022.

[17] T. Dao, "FlashAttention-2: Faster Attention with Better Parallelism and Work Partitioning," 2023.

[18] S. Yang, J. Kautz, and A. Hatamizadeh, "Gated Delta Networks: Improving Mamba2 with Delta Rule," 2025, [Online]. Available: https://arxiv.org/abs/2412.06464

[19] I. Schlag, K. Irie, and J. Schmidhuber, "Linear Transformers Are Secretly Fast Weight Programmers," 2021, [Online]. Available: https://arxiv.org/abs/2102.11174

[20] S. Yang, "DeltaNet Explained (Part II)." [Online]. Available: https://sustcsonglin.github.io/blog/2024/deltanet-2/

[21] T. Joffrain, T. M. Low, E. S. Quintana-Ortí, R. A. van de Geijn, and F. G. V. Zee, "Accumulating Householder transformations, revisited," *ACM Trans. Math. Softw.*, vol. 32, pp. 169–179, 2006, [Online]. Available: https://api.semanticscholar.org/CorpusID:15723171

[22] C. H. Bischof and C. V. Loan, "The WY representation for products of householder matrices," in *PP*, 1985. [Online]. Available: https://api.semanticscholar.org/CorpusID:36094006

[23] T. Dao and A. Gu, "Transformers are SSMs: Generalized Models and Efficient Algorithms Through Structured State Space Duality," 2024, [Online]. Available: https://arxiv.org/abs/2405.21060

[24] G. E. Blelloch, "Prefix Sums and their Applications," [Online]. Available: https://www.cs.cmu.edu/~guyb/papers/Ble93.pdf

[25] P. Micikevicius *et al.*, "Mixed Precision Training," 2018.

[26] D. Hendrycks and K. Gimpel, "Gaussian Error Linear Units (GELUs)," 2023.

[27] J. Kaplan *et al.*, "Scaling Laws for Neural Language Models," 2020.

[28] J. Hoffmann *et al.*, "Training Compute-Optimal Large Language Models," 2022.

[29] R. Rafailov, A. Sharma, E. Mitchell, S. Ermon, C. D. Manning, and C. Finn, "Direct Preference Optimization: Your Language Model is Secretly a Reward Model," 2024, [Online]. Available: https://arxiv.org/abs/2305.18290

[30] J. Schulman, F. Wolski, P. Dhariwal, A. Radford, and O. Klimov, "Proximal Policy Optimization Algorithms," 2017, [Online]. Available: https://arxiv.org/abs/1707.06347

[31] K. Ethayarajh, W. Xu, N. Muennighoff, D. Jurafsky, and D. Kiela, "KTO: Model Alignment as Prospect Theoretic Optimization," 2024, [Online]. Available: https://arxiv.org/abs/2402.01306

[32] Z. Shao *et al.*, "DeepSeekMath: Pushing the Limits of Mathematical Reasoning in Open Language Models," 2024, [Online]. Available: https://arxiv.org/abs/2402.03300

[33] DeepSeek-AI *et al.*, "DeepSeek-R1: Incentivizing Reasoning Capability in LLMs via Reinforcement Learning," 2025, [Online]. Available: https://arxiv.org/abs/2501.12948

[34] Q. Yu *et al.*, "DAPO: An Open-Source LLM Reinforcement Learning System at Scale," 2025, [Online]. Available: https://arxiv.org/abs/2503.14476

[35] J. He *et al.*, "Skywork Open Reasoner 1 Technical Report," 2025, [Online]. Available: https://arxiv.org/abs/2505.22312

[36] M. Shoeybi, M. Patwary, R. Puri, P. LeGresley, J. Casper, and B. Catanzaro, "Megatron-LM: Training Multi-Billion Parameter Language Models Using Model Parallelism," 2020.

[37] H. Liu, M. Zaharia, and P. Abbeel, "Ring Attention with Blockwise Transformers for Near-Infinite Context," 2023, [Online]. Available: https://arxiv.org/abs/2310.01889

[38] W. Brandon *et al.*, "Striped Attention: Faster Ring Attention for Causal Transformers," 2023, [Online]. Available: https://arxiv.org/abs/2311.09431

[39] H. Z. P. N. M. M. L. W. T. W. Nouamane Tazi Ferdinand Mom, "The Ultra-Scale Playbook: Training LLMs on GPU Clusters," 2025.

[40] Y. Huang *et al.*, "GPipe: Efficient Training of Giant Neural Networks using Pipeline Parallelism," 2019, [Online]. Available: https://arxiv.org/abs/1811.06965

[41] A. Harlap *et al.*, "PipeDream: Fast and Efficient Pipeline Parallel DNN Training," 2018, [Online]. Available: https://arxiv.org/abs/1806.03377

[42] D. Narayanan *et al.*, "Efficient Large-Scale Language Model Training on GPU Clusters Using Megatron-LM," 2021, [Online]. Available: https://arxiv.org/abs/2104.04473

[43] J. Lamy-Poirier, "Breadth-First Pipeline Parallelism," 2023, [Online]. Available: https://arxiv.org/abs/2211.05953

[44] A. Grattafiori *et al.*, "The Llama 3 Herd of Models," 2024, [Online]. Available: https://arxiv.org/abs/2407.21783

[45] P. Qi, X. Wan, G. Huang, and M. Lin, "Zero Bubble Pipeline Parallelism," 2023, [Online]. Available: https://arxiv.org/abs/2401.10241

[46] DeepSeek-AI *et al.*, "DeepSeek-V3 Technical Report," 2025, [Online]. Available: https://arxiv.org/abs/2412.19437

[47] A. Dosovitskiy *et al.*, "An Image is Worth 16x16 Words: Transformers for Image Recognition at Scale," 2021, [Online]. Available: https://arxiv.org/abs/2010.11929

[48] A. Radford *et al.*, "Learning Transferable Visual Models From Natural Language Supervision," 2021, [Online]. Available: https://arxiv.org/abs/2103.00020

[49] W. Fedus, B. Zoph, and N. Shazeer, "Switch Transformers: Scaling to Trillion Parameter Models with Simple and Efficient Sparsity," 2022, [Online]. Available: https://arxiv.org/abs/2101.03961

[50] N. Shazeer *et al.*, "Outrageously Large Neural Networks: The Sparsely-Gated Mixture-of-Experts Layer," 2017, [Online]. Available: https://arxiv.org/abs/1701.06538

[51] N. Muennighoff *et al.*, "OLMoE: Open Mixture-of-Experts Language Models," 2024, [Online]. Available: https://arxiv.org/abs/2409.02060

[52] T. Gale, D. Narayanan, C. Young, and M. Zaharia, "MegaBlocks: Efficient Sparse Training with Mixture-of-Experts," 2022, [Online]. Available: https://arxiv.org/abs/2211.15841

[53] DeepSeek-AI *et al.*, "DeepSeek-V2: A Strong, Economical, and Efficient Mixture-of-Experts Language Model," 2024, [Online]. Available: https://arxiv.org/abs/2405.04434

[54] A. Holtzman, J. Buys, L. Du, M. Forbes, and Y. Choi, "The Curious Case of Neural Text Degeneration," 2020.

[55] Y. Leviathan, M. Kalman, and Y. Matias, "Fast Inference from Transformers via Speculative Decoding," 2023, [Online]. Available: https://arxiv.org/abs/2211.17192

[56] R. Pope *et al.*, "Efficiently Scaling Transformer Inference," 2022.

[57] C. Chen, "Transformer Inference Arithmetic," 2022, [Online]. Available: https://kipp.ly/blog/transformer-inference-arithmetic/

[58] W. Kwon *et al.*, "Efficient Memory Management for Large Language Model Serving with PagedAttention." [Online]. Available: https://arxiv.org/abs/2309.06180

[59] L. Zheng *et al.*, "SGLang: Efficient Execution of Structured Language Model Programs." [Online]. Available: https://arxiv.org/abs/2312.07104

[60] O. Press, N. A. Smith, and M. Lewis, "Train Short, Test Long: Attention with Linear Biases Enables Input Length Extrapolation," *CoRR*, vol. abs/2108.12409, 2021, [Online]. Available: https://arxiv.org/abs/2108.12409

[61] P. Patarasuk and X. Yuan, "Bandwidth optimal all-reduce algorithms for clusters of workstations," *Journal of Parallel and Distributed Computing*, 2009.

[62] H. He, "Making Deep Learning Go Brrrr From First Principles," 2022, [Online]. Available: https://horace.io/brrr_intro.html

[63] S. McCandlish, J. Kaplan, D. Amodei, and O. D. Team, "An Empirical Model of Large-Batch Training," 2018.

[64] D. A. Roberts, S. Yaida, and B. Hanin, "The Principles of Deep Learning Theory," *CoRR*, vol. abs/2106.10165, 2021, [Online]. Available: https://arxiv.org/abs/2106.10165

[65] G. Yang *et al.*, "Tensor Programs V: Tuning Large Neural Networks via Zero-Shot Hyperparameter Transfer," 2022.

[66] S. Yaida, "Meta-Principled Family of Hyperparameter Scaling Strategies," 2022, [Online]. Available: https://arxiv.org/abs/2210.04909

[67] D. Doshi, T. He, and A. Gromov, "Critical Initialization of Wide and Deep Neural Networks through Partial Jacobians: General Theory and Applications," 2023, [Online]. Available: https://arxiv.org/abs/2111.12143

[68] X. Chen *et al.*, "Symbolic Discovery of Optimization Algorithms," 2023, [Online]. Available: https://arxiv.org/abs/2302.06675

[69] J. T. Chiu, "Differentiating through an associative parallel scan," 2024, [Online]. Available: https://justintchiu.com/blog/pscan_diff/

[70] P. Clark *et al.*, "Think you have Solved Question Answering? Try ARC, the AI2 Reasoning Challenge," 2018, [Online]. Available: https://arxiv.org/abs/1803.05457

[71] M. Suzgun *et al.*, "Challenging BIG-Bench Tasks and Whether Chain-of-Thought Can Solve Them," 2022, [Online]. Available: https://arxiv.org/abs/2210.09261

[72] A. Srivastava *et al.*, "Beyond the Imitation Game: Quantifying and extrapolating the capabilities of language models," 2023, [Online]. Available: https://arxiv.org/abs/2206.04615

[73] C. Clark, K. Lee, M.-W. Chang, T. Kwiatkowski, M. Collins, and K. Toutanova, "BoolQ: Exploring the Surprising Difficulty of Natural Yes/No Questions," 2019, [Online]. Available: https://arxiv.org/abs/1905.10044

[74] D. Rein *et al.*, "GPQA: A Graduate-Level Google-Proof Q&A Benchmark," 2023, [Online]. Available: https://arxiv.org/abs/2311.12022

[75] K. Cobbe *et al.*, "Training Verifiers to Solve Math Word Problems," 2021, [Online]. Available: https://arxiv.org/abs/2110.14168

[76] R. Zellers, A. Holtzman, Y. Bisk, A. Farhadi, and Y. Choi, "HellaSwag: Can a Machine Really Finish Your Sentence?," 2019, [Online]. Available: https://arxiv.org/abs/1905.07830

[77] J. Zhou *et al.*, "Instruction-Following Evaluation for Large Language Models," 2023, [Online]. Available: https://arxiv.org/abs/2311.07911

[78] D. Hendrycks *et al.*, "Measuring Massive Multitask Language Understanding," 2021, [Online]. Available: https://arxiv.org/abs/2009.03300

[79] Y. Wang *et al.*, "MMLU-Pro: A More Robust and Challenging Multi-Task Language Understanding Benchmark," 2024.

[80] Z. Sprague, X. Ye, K. Bostrom, S. Chaudhuri, and G. Durrett, "MuSR: Testing the Limits of Chain-of-thought with Multistep Soft Reasoning," 2024, [Online]. Available: https://arxiv.org/abs/2310.16049

[81] T. Mihaylov, P. Clark, T. Khot, and A. Sabharwal, "Can a Suit of Armor Conduct Electricity? A New Dataset for Open Book Question Answering," 2018, [Online]. Available: https://arxiv.org/abs/1809.02789

[82] Y. Bisk, R. Zellers, R. L. Bras, J. Gao, and Y. Choi, "PIQA: Reasoning about Physical Commonsense in Natural Language," 2019, [Online]. Available: https://arxiv.org/abs/1911.11641

[83] S. Lin, J. Hilton, and O. Evans, "TruthfulQA: Measuring How Models Mimic Human Falsehoods," 2022, [Online]. Available: https://arxiv.org/abs/2109.07958

[84] K. Sakaguchi, R. L. Bras, C. Bhagavatula, and Y. Choi, "WinoGrande: An Adversarial Winograd Schema Challenge at Scale," 2019, [Online]. Available: https://arxiv.org/abs/1907.10641

[85] C.-P. Hsieh *et al.*, "RULER: What's the Real Context Size of Your Long-Context Language Models?," 2024, [Online]. Available: https://arxiv.org/abs/2404.06654